
Physics-enhanced Deep Surrogate Models for Neutron Transport Solvers

Master thesis

by CATERINA FRAU



Master of Science in Nuclear Engineering

ÉCOLE POLYTECHNIQUE FÉDÉRALE DE LAUSANNE
in collaboration with University of California, Berkeley

EPFL academic supervisor: Dr. Mathieu Hursin

External supervisor: Dr. Daniel Siefman

14 AUGUST 2026

Abstract

Neutron transport calculations are central to reactor analysis and design, but high-fidelity Monte Carlo simulations are computationally demanding when repeated evaluations are required. This thesis investigates the application of Physics Enhanced Deep Surrogates (PEDS) to neutron transport eigenvalue calculations, with the objective of obtaining more accurate predictions while preserving the efficiency of a low-fidelity solver. The proposed framework combines a neural network with a neutron diffusion solver, while OpenMC simulations are used to generate the high-fidelity training data. Rather than directly predicting the effective multiplication factor k_{eff} , the neural network adjusts the multigroup cross sections supplied to the solver. This correction is learned to compensate for its physical and spatial approximations, bringing the result closer to the reference. The initial cross section estimate is obtained from a fast polynomial regression model fitted to the multigroup cross sections extracted from the OpenMC evaluations. A dedicated one dimensional diffusion solver was developed and integrated into the neural network. The required gradients are computed via an analytically derived vector-Jacobian product based on the adjoint flux solution, bypassing the need to differentiate through the solver itself. The method was evaluated over a six-parameter reactor design space including geometric and material properties. On the test set, the PEDS model reduced the discrepancy between the coarse diffusion solver and the OpenMC reference by approximately 90% on average, while being evaluated four orders of magnitude faster than the high-fidelity calculations. Optimal performance was achieved with a training set of about 1000 samples, demonstrating that the data requirements remain compatible with repeated design applications. The correction was found to be most effective for near critical and supercritical configurations, while larger residual errors remained for strongly subcritical systems. Subsequent sensitivity analyses suggest that these originate from the underlying physical limitations of the diffusion model. A forward design study demonstrated that screening candidate geometries with the trained surrogate substantially accelerates the identification of optimal reactor configurations, with the initial training cost fully recovered after four design searches. The proposed formulation establishes a first basis for physics enhanced surrogate models in neutron transport problems, successfully combining the rapid execution of a deterministic diffusion solver with improved agreement to Monte Carlo simulations.

Table of Contents

Abstract	i
List of Figures	iv
List of Tables	vii
1 Introduction	1
2 Physics-Enhanced Deep Surrogates applied to neutron transport	4
2.1 The PEDS methodology	4
2.1.1 Existing PEDS applications	7
2.1.2 Adjoint method	9
2.2 PEDS application to neutron transport	10
2.2.1 Boltzmann NT equation and diffusion approximation	13
2.2.2 Monte Carlo codes	15
2.2.3 Derivation of vJp for gradients definition	17
3 Application to diffusion equation specifics	20
3.1 Low-Fidelity diffusion solver	20
3.1.1 Governing equations	21
3.1.2 Solution method	22
3.1.3 Adjoint calculation	23
3.1.4 Analytical solution check	23
3.2 High-Fidelity OpenMC solver	24
3.3 PEDS network implementation	25
3.3.1 External solver	25
3.3.2 Custom VJP rule	26
3.3.3 Gradient verification of the custom vJp	27
3.3.4 Network architecture	30
4 Results and discussion	36
4.1 Geometry description and implementation	36
4.2 Training data generation	38

4.3	Preliminary results	40
4.3.1	Training-validation gap and correction attempts	41
4.3.2	Physical validation and overall framework efficacy	43
4.4	Model tuning and sensitivity analysis	47
4.4.1	Influence of training dataset size	48
4.4.2	Subdivision of training data into subcritical and supercritical regimes	50
4.4.3	Impact of a uniform k_{eff} training distribution	51
4.4.4	Study over the parameters space	54
4.5	Forward design study	58
4.5.1	Baseline design study: High-Fidelity LHS search	59
4.5.2	Surrogate-assisted LHS screening with a PEDS ensemble	60
5	Conclusion and Outlook	63
5.1	Future Work	64
A	Polynomial regression evaluation metrics	66
B	Matrix formulation of the 1D diffusion solver	68
C	Verification of numerical sources of error	70
C.1	OpenMC convergence	70
C.2	Influence of diffusion solver spatial discretization	72
D	Parametric study performance metrics	74
E	Detailed design study geometries	76
E.1	High Fidelity LHS Baseline	76
E.2	Surrogate Assisted LHS Screening	76
	Bibliography	78

List of Figures

Figure	Page
2.1 Schematic representation of the PEDS computational graph, adapted from [1] with supplemental labels added to align with the mathematical definitions presented in this chapter. A high-fidelity geometry is downsampled to provide a physical baseline, while a neural network generator concurrently produces an optimized geometric correction. These representations are combined via a learned mixing weight and passed through an embedded low-fidelity solver to yield the final property prediction.	6
2.2 Flowchart of the PEDS architecture adapted for neutron transport calculations. The generator network outputs a set of multiplicative corrections to refine a baseline cross-section estimate, producing an optimized multigroup cross-section tensor to be evaluated by the low-fidelity diffusion solver.	12
3.1 Comparison of numerical and analytical solutions for the two-group forward and adjoint fluxes in a 1D homogeneous slab.	24
3.2 Comparison of the effective multiplication factor, k_{eff} , predicted by the OpenMC high-fidelity solver and the low-fidelity diffusion solver over the sampled configurations. The left panel shows the results using the diffusion solver only, whereas the right panel shows the results after the PEDS applied its cross-section correction.	29
3.3 Flowchart of the PEDS training pipeline, showing the flow from the raw geometry parameters to the eigenvalue prediction, the loss evaluation, and the parameter update through the custom VJP rule.	35
4.1 Schematic cross section of the three region cylindrical geometry: B ₄ C control rod, fuel annulus, and water moderator, centre to outside.	37
4.2 OpenMC computed normalised two-group flux profile, with dotted lines marking the control rod and fuel annulus outer boundaries.	38
4.3 Diffusion solver computed normalised two-group flux profile for the same reactor configuration as Figure 4.2, with dotted lines marking the control rod and fuel annulus outer boundaries.	38
4.4 Distribution of the effective multiplication factor (k_{eff}) for the complete generated dataset, resulting from uniform Latin Hypercube Sampling across the defined geometric and material parameter bounds.	40
4.5 Evolution of MAE and MSE metrics over 70 training epochs for the baseline PEDS configuration.	41

4.6	Evolution of the reactivity difference, expressed in pcm, over training epochs for selected validation samples. The dashed black line shows the mean error across samples, while the horizontal blue line marks the reference value $\beta_{\text{eff}} = 650$ pcm.	43
4.7	Comparison of the predicted k_{eff} against the OpenMC reference for the validation samples. The initial diffusion solver results are shown in (a), while the neural network corrected predictions after 70 epochs are presented in (b). The point colors represent the reactivity difference $\Delta\rho$ expressed in pcm.	44
4.8	Nominal values and relative magnitude of neural network corrections applied to the baseline macroscopic cross-sections (from the polynomial regression) for a representative sample at epoch 70.	45
4.9	Normalized spatial flux distributions for the fast and thermal energy groups of a representative validation sample, whose parameters are annotated within the figure. The plot compares the high-fidelity OpenMC target, the uncorrected baseline diffusion solution, and the diffusion solution utilizing PEDS-corrected cross-sections.	46
4.10	Distribution of validation samples evaluated by their reactivity difference ($\Delta\rho$) and mapped against the magnitude of the target k_{eff} . The plots compare the error distribution prior to the neural network correction (a) and at the final epoch 70 (b).	46
4.11	Kernel Density Estimation (KDE) of the target k_{eff} distribution for a 1000-sample training set across 10 independent random seeds. The validation and test distributions remain fixed, while the training distribution exhibits minor statistical fluctuations.	49
4.12	Scaling of test set performance metrics with respect to training dataset size. Data points represent the average across 10 random seeds, with shaded regions indicating the standard deviation.	49
4.13	Kernel Density Estimation (KDE) of the k_{eff} distribution for the balanced dataset splits, constrained to the $[0.85, 1.15]$ range.	52
4.14	Prediction error against target k_{eff} found in the test set, for the unbalanced and balanced configurations, before and after the NN correction.	53
4.15	PEDS performance across stratified k_{eff} bins for the training, validation and test datasets. The upper, middle, and lower panels show the sample distribution, mean absolute reactivity discrepancy before and after correction, and fraction of samples satisfying $ \Delta\rho < \beta_{\text{eff}} = 650$ pcm, respectively. The middle panel uses a logarithmic scale; error bars denote the standard deviation over five runs.	55
4.16	Parallel-coordinates plot of the six geometry parameters for a 200 test-set case, colored by absolute reactivity error before neural-network correction (reference case, 1000 training samples).	56
4.17	Parallel-coordinates plot of the same six geometry parameters and test cases as Figure 4.16, after neural-network correction.	56
4.18	Scatter plots comparing the absolute reactivity error against individual geometry parameters for the validation set. The Pearson correlation coefficient (r) is presented for each parameter subplot. The worst 5% of samples are highlighted to illustrate the dominant criticality regimes driving the highest errors before (a) and after (b) correction.	57

4.19	Parity plots comparing the predicted k_{eff} and PPF against high fidelity OpenMC results for the 30 verified designs. The performance of the PEDS ensemble is contrasted with the uncorrected diffusion solver.	62
C.1	Shannon entropy over Monte Carlo batches for a representative geometry. A stable plateau is reached before the transition from inactive to active batches at batch 100.	71
C.2	Signed reactivity differences relative to OpenMC for the ten validation geometries with the largest residual errors after PEDS correction. Results are shown for mesh widths from 2.0 cm to 0.01 cm. Solid and dashed lines indicate the mean signed and mean absolute errors, respectively.	72
C.3	Signed reactivity differences relative to OpenMC for the same geometries using the uncorrected polynomial cross sections. Results are shown for mesh widths from 2.0 cm to 0.01 cm. Solid and dashed lines indicate the mean signed and mean absolute errors, respectively.	73

List of Tables

Table	Page
3.1 Gradient verification comparing the analytical VJP against the central finite differences (FD) estimate across all material regions of the selected geometry, for all cross-section components with a non-zero value. Subscripts 1 and 2 denote respectively the fast and thermal energy groups.	28
4.1 Ranges considered for the six geometric and material design parameters.	39
4.2 Summary of mean execution times for key PEDS operations during the baseline training run.	41
4.3 Training metrics at selected epochs. Epoch 1 represents the state after the first network update, distinct from the uncorrected baseline.	42
4.4 Validation metrics at selected epochs, highlighting the lower fraction of samples meeting the 650 pcm threshold compared to the training set.	42
4.5 Test metrics as a function of training dataset size. The mean error includes the standard deviation across 10 random seeds.	49
4.6 Test metrics for isolated subcritical and supercritical configurations, before and after NN correction. The mean error includes the standard deviation across random seeds ($n = 10$ seeds for the corrected results; single evaluation for the uncorrected baseline).	51
4.7 Test metrics for unbalanced and balanced k_{eff} bins, training size 500, before and after NN correction. Corrected results report mean $\pm$ standard deviation across 10 seeds; uncorrected values are single-seed, with no variance.	52
4.8 High-fidelity quality metrics for 28 HF-feasible designs ($ \Delta\rho(k_{\text{HF}}, 1) \leq 1500$ pcm).	61
A.1 Polynomial regression evaluation metrics for each multigroup cross-section, sorted by R^2 in descending order. RMSE denotes root mean squared error; χ^2 denotes the chi-squared goodness-of-fit statistic. Subscripts 1 and 2 denote the fast and thermal energy groups, respectively, while $S_{gg'}$ denotes the scattering transfer from group g to g'	67
C.1 Physical parameters for the five selected geometries.	70
C.2 Comparison of k_{eff} values between original and converged OpenMC runs, including standard deviations and diffusion solver results with updated cross sections.	71

D.1	PEDS correction performance across k_{eff} bins for train, validation, and test splits (train size = 1000 and validation size = 500, averaged over 5 runs; test size = 300 and kept locked). The fraction satisfying $ \Delta\rho < \beta_{\text{eff}}$ is reported after correction, with $\beta_{\text{eff}} = 650$ pcm. Before correction, this fraction was zero in 22 of the 30 split-bin cases and did not exceed 2.44%, hence it was deemed negligible to report.	75
E.1	Top 10 ranked feasible designs from the high fidelity LHS baseline. All designs share $\sigma_{k_{\text{eff}}} \approx 40$ pcm, the fixed convergence trigger used to stop each run.	76
E.2	Detailed parameters and metrics for the ten best geometries identified by the PEDS ensemble, ranked by their high fidelity verified PPF.	77

Chapter 1

Introduction

In recent years, nuclear energy has received renewed attention as a reliable low-carbon source that can support decarbonization and energy security goals. This interest concerns both the extended operation of the existing reactor fleet, whose role remains important in electricity production, and the active development of advanced reactor technologies intended for future deployment. As such, nuclear energy is increasingly viewed as a vital component of sustainable modern energy systems [2].

Within this context, numerical simulations are extensively utilized across all fields of nuclear engineering. For current power plants, reliable modeling tools are used to support routine operational checks, including fuel management and risk assessment. For new reactor technologies, comprehensive simulations are foundational for iterative design studies and optimization, sensitivity analyses, and safety evaluations, needed to advance the licensing procedures.

At the core of these assessments is the simulation of neutron transport, which fundamentally dictates the physical behaviour of neutrons in the reactor core, strongly affecting its performance and safety-related quantities. However, modern engineering workflows require these studies to be repeated across extensive parameter spaces, including varied physical geometries, material properties, and operational states, making the computational cost associated with the simulations a major bottleneck. Consequently, advancing reactor analysis requires tools that can successfully balance physical accuracy with computational efficiency. In practice, this balance is typically achieved by relying on different categories of neutronic solvers, each characterized by a distinct compromise between fidelity and computational expense.

High-fidelity methods such as continuous-energy Monte Carlo codes [3–5] are considered reference tools for reactor analysis, because they resolve neutron interactions with detailed cross-section data and minimal modeling approximations. Their use, however, is associated with substantial computational cost, which becomes prohibitive when many evaluations are required, as for design exploration or optimization.

Deterministic methods, based on multigroup transport or diffusion formulations [6–8], provide a faster alternative by solving a discretized approximation of the transport equation in energy, space, and angle. These approaches are therefore well suited to workflows where large numbers of core calculations are required. However, the gain in efficiency is obtained at the price of additional approximations in the physical model and its numerical representation, which can impact the accuracy of key reactor

performance and safety indicators.

This motivates the search for new strategies for neutron transport analysis that can retain much of the predictive strength of high-fidelity simulations while approaching the speed of lower-cost deterministic tools. A possible solution to this accuracy–cost trade-off is the use of surrogate models, that is, fast approximate models trained to reproduce the behaviour of a more expensive simulator. Such models have been widely investigated in many areas of computational science and engineering, to accelerate design optimization and inverse problems [9, 10]. More recently, scientific machine learning has emerged as a class of methods that combines data-driven models with knowledge of the governing physical laws. Rather than replacing the governing physics entirely, these approaches incorporate equations, constraints, or simplified solvers into the learning process, with the aim of improving robustness, reduce the amount of training data required and increase trust in the predictions [11–14].

This shift has also reached nuclear engineering, where physics-informed neural networks and related surrogate architectures have been proposed for various applications. Recent studies have demonstrated their use in the surrogate modeling of accidental scenarios [15], and in the direct prediction of neutron flux by embedding transport equations and boundary conditions into the network loss function [16]. In parallel, data-based and hybrid surrogate models have been developed to accelerate routine reactor core simulations [17] and transient kinetics calculations [18]. These studies show that embedding physical structure into the learning process can improve accuracy and generalization compared with purely data-driven models, while also being of interest for licensing, where physically consistent predictions are required. A comprehensive overview of the current status, application scenarios, and future prospects of machine learning methods in reactor physics can be found in [19].

A recent development in the Machine Learning community is the physics-enhanced deep Surrogate (PEDS) framework, introduced as a way to combine a Low-Fidelity numerical solver with a Neural Network generator, trained end-to-end to match a High-Fidelity reference [20]. Rather than predicting the final solution directly, the method uses the network to learn the input for a fast physics solver, which remains embedded inside the model and produces the final prediction. This hybrid construction allows the surrogate to preserve a stronger physical basis than a purely black-box reduced-order model, while still approaching the accuracy of a high-fidelity solver at much lower cost, while requiring significantly less training data. The PEDS framework has so far been demonstrated only on a small number of PDE-governed problems, most recently on the phonon Boltzmann transport equation [1].

Neutron transport is an attractive candidate for this application: it is a field where the accuracy–cost trade-off is particularly significant, and in which well-established physical principles can be conveniently integrated directly into the surrogate model. To the best of current knowledge, the PEDS methodology has not yet been extended to neutron transport calculations. The goal of this Master thesis work is hence to investigate whether it can be successfully used for nuclear engineering applications, and whether its advantages in terms of accuracy and data efficiency can be transferred to this new field.

A remarkable difference with respect to these first applications is that, rather than a partial differential equation, the underlying physical problem is here formulated as an eigenvalue problem. This does not pose any issue for the network: instead of backpropagating through every iteration, the framework uses

a custom gradient rule derived analytically, based on the adjoint flux formulation commonly used in nuclear sensitivity analysis, providing an accurate calculation at low additional computational cost.

Building on this formulation, the low-fidelity solver embedded in the network is a fast deterministic code based on the diffusion approximation of the neutron transport equation, developed in Python specifically for this application. Training data is instead generated using a Monte Carlo code, serving as the high-fidelity reference for its accuracy in reactor physics calculations. The target property of interest is the effective multiplication factor k_{eff} ; the network is trained to output a set of multigroup cross sections, the physical input required by the diffusion solver, so that its resulting eigenvalue is brought progressively closer to the Monte Carlo reference. To validate this approach, a simplified model is built and tested, providing a first assessment of whether the methodology can be effectively extended to design problems in nuclear engineering.

Beyond this initial application, the successful extension of PEDS to an eigenvalue-based transport problem would carry broader significance for nuclear engineering. Neutronic calculations of this kind are not limited to reactor core design, but also underpin criticality safety analysis, fuel cycle studies, and shielding calculations, all of which routinely require balancing computational cost against physical accuracy. A validated physics-enhanced surrogate could therefore offer a practical tool for accelerating iterative design and optimization workflows across these areas, while retaining a level of physical trust that purely data-driven models cannot guarantee.

This thesis is structured as follows: Chapter 2 introduces the Physics-Enhanced Deep Surrogate (PEDS) methodology and establishes its theoretical adaptation to neutron transport calculations. Building on this foundation, Chapter 3 details the computational implementation of the architecture, including the custom low-fidelity diffusion solver, the high-fidelity OpenMC model used for data generation, and the neural network’s training integration. Chapter 4 then evaluates the model’s performance on a 1D cylindrical reactor core, offering a comprehensive sensitivity analysis and demonstrating its utility through a practical forward design study. Finally, Chapter 5 summarizes the core findings, evaluates the overall efficacy of extending PEDS to nuclear engineering, and outlines directions for future research.

Chapter 2

Physics-Enhanced Deep Surrogates applied to neutron transport

This chapter introduces the physics-enhanced deep surrogate (PEDS) framework and describes how it is adapted to neutron transport calculations. Section 2.1 presents the general PEDS methodology as presented in the original literature, describing the roles of the high- and low-fidelity solvers, the construction of the network, and the general strategy used to obtain gradients through the embedded solver. A brief summary of the existing example applications on which the framework has so far been demonstrated is also reported. Section 2.2 then gives a general, intuitive description of how the PEDS framework is adapted to neutron transport, motivating the choice of solvers, design parameters, and network structure adopted in this thesis. Subsection 2.2.1 introduces the governing neutron transport equation and the diffusion approximation used as the low-fidelity model, both for the forward and adjoint problem, while Subsection 2.2.2 presents the Monte Carlo codes used to generate the high-fidelity reference data. Finally, Subsection 2.2.3 derives the analytical gradient rule that allows the network to be trained without differentiating through the solver directly.

2.1 The PEDS methodology

PEDS (Physics-Enhanced Deep Surrogate) is a scientific machine learning framework designed for the fast simulation of physical systems, initially developed for partial differential equations (PDEs) [20]. The construction of a PEDS surrogate involves two solvers, defined in this context as the specific numerical methods or computational codes utilized to resolve the governing equations of the physical system. A high-fidelity solver $f^{hf}(\cdot)$ is used to generate the reference data required for training and validation. Given a design parameter vector $\mathbf{p}$, encoding the geometry, material properties, boundary conditions, or other configuration specifications of the system, the high-fidelity solver computes the target property $y = f^{hf}(hf(\mathbf{p}))$ by solving the PDE on a fine discretization. This corresponds to the high-fidelity representation of the problem based on the parametrization vector, defined as $hf(\mathbf{p})$. This result is accurate, but computationally expensive to obtain; by varying $\mathbf{p}$ across many simulations, the dataset representative of the design space over which the surrogate is then trained to generalize is built.

A low-fidelity solver $f^{lf}(\cdot)$ is instead used within the surrogate itself. It operates on a coarser representation

of the same problem $lf(\mathbf{p})$, and is orders of magnitude faster than f^{hf} , though substantially less accurate. The solver may share the same numerical method as f^{hf} at reduced resolution, or introduce simplifications directly in the physics; crucially, it retains the key structural properties of the problem, such as conservation laws or symmetries, that a purely data-driven model would not inherently have.

For PEDS to approximate the high fidelity target, the low-fidelity solver must be able to reproduce the relevant range of high fidelity outputs when supplied with admissible coarse inputs. This requirement is expressed by the inclusion criterion,

$$\text{span} \{ f^{hf}(hf(\mathbf{p})) \mid \mathbf{p} \in \mathcal{P} \} \subseteq \text{span} \{ f^{lf}(\mathbf{x}) \mid \mathbf{x} \in \mathcal{X}^{lf} \}, \quad (2.1)$$

where $\mathcal{X}^{lf}$ denotes the set of admissible inputs to the low-fidelity solver. If this condition is not satisfied, no generator can compensate for the missing range of outputs, since the low-fidelity solver cannot produce the required target property for any possible generated input. Thus, it does not necessarily need to be accurate for the directly downsampled geometry, but it must be capable of producing the high fidelity response after a physically admissible correction of its input.

A distinguishing feature of PEDS lies in how the input $\mathbf{x}$ to $f^{lf}(\cdot)$ is constructed. The high-fidelity geometry $hf(\mathbf{p})$ is first downsampled to the coarse representation $downsample(\mathbf{p})$ required by the low-fidelity solver (for example, the previous applications used sub-pixel averaging), to produce a baseline field. A generator network, $generator_{NN}(\mathbf{p})$, is trained to produce an alternative coarse representation of the same geometry through a nonlinear transformation, designed to compensate for the errors introduced by the low-fidelity discretization and physics. Both components are built on the same design vector $\mathbf{p}$ used for $hf(\mathbf{p})$, but they represent the system at different levels of abstraction. The two representations are combined through a learned mixing weight $w \in [0, 1]$, independent of $\mathbf{p}$, that balances the physical baseline with the NN output:

$$G(\mathbf{p}) = w \cdot generator_{NN}(\mathbf{p}) + (1 - w) \cdot downsample(\mathbf{p}) \quad (2.2)$$

The generator therefore does not predict the target property itself, but instead learns an optimized input field for the low-fidelity solver. The surrogate property is finally obtained by applying the low-fidelity solver to this mixed geometry:

$$\tilde{f}(\mathbf{p}) = f^{lf}(P[w \cdot generator_{NN}(\mathbf{p}) + (1 - w) \cdot downsample(\mathbf{p})]) = f^{lf}(\mathbf{x}) \quad (2.3)$$

Where P is a projection operator that can be used to impose any additional physical constraints or symmetries on the geometry, ensuring that the input to the solver remains physically admissible.

Figure 2.1 presents the diagram proposed for PEDS in [20] to give a visual understanding of the computational graph, further labeled to connect to the definitions given throughout the chapter. Both the high-fidelity solver and the full surrogate model aim to predict the same target property from the same design parameters $\mathbf{p}$; the former does so accurately but at high computational cost, while the latter

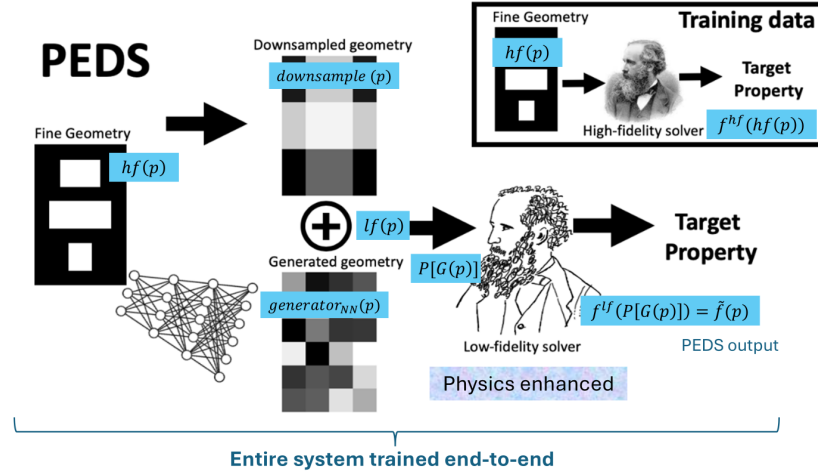


Figure 2.1: Schematic representation of the PEDS computational graph, adapted from [1] with supplemental labels added to align with the mathematical definitions presented in this chapter. A high-fidelity geometry is downsampled to provide a physical baseline, while a neural network generator concurrently produces an optimized geometric correction. These representations are combined via a learned mixing weight and passed through an embedded low-fidelity solver to yield the final property prediction.

achieves a close approximation at a fraction of the cost. The role of the neural network is to improve the input fed to the low-fidelity solver, guiding its output towards the high-fidelity reference.

The full model is trained end-to-end, minimizing the discrepancy between $\tilde{f}(\mathbf{p})$ and the high-fidelity reference $f^{hf}(\mathbf{p})$ across the design space. This requires gradients of the loss with respect to the network weights to be propagated through the low-fidelity solver, which acts as a layer of the computational graph. If the solver is simple enough to be expressed directly in the syntax of the machine learning framework, standard reverse-mode automatic differentiation can trace the operations performed and provide the required gradients; however, this becomes impractical for more complex solvers, or when many iterations are required, as the cost of backpropagating through each step accumulates. Finite differences offer an alternative that avoids this issue, but at the cost of one additional solver evaluation per network parameter, which quickly becomes prohibitive for realistically sized networks. In [20], this is instead handled through the adjoint method, a well-established technique in sensitivity analysis [21] that provides the required vector-Jacobian product without repeated solver evaluations; its mathematical formulation is presented in Subsection 2.1.2. Defining an adjoint rule for the solver removes the need for it to be differentiable in the first place, which considerably broadens the range of low-fidelity solvers that can be embedded in the PEDS framework.

The resulting framework is physics-enhanced because the final prediction is obtained by passing the learned geometry through the low-fidelity solver, rather than by letting a neural network produce the output directly. In this way, physically meaningful constraints such as conservation laws and geometric symmetries are incorporated into the surrogate architecture itself, instead of being learned only from data. Thus, the solver layer enforces structure, while the network is free to correct the remaining discrepancy between the coarse and high-fidelity descriptions. This division of labor is what allows PEDS to combine

the interpretability and reliability of physics-based models with the accuracy of deep learning, while remaining far less expensive than repeated high-fidelity simulation.

2.1.1 Existing PEDS applications

Since its introduction, PEDS has been demonstrated on a small number of PDE-governed problems. The original formulation was applied to the linear diffusion equation, the nonlinear reaction-diffusion equation, and Maxwell’s equations for optical metamaterials [20]. More recently, the framework has been extended to the phonon Boltzmann transport equation for nanoscale thermal materials design, coupling a differentiable Fourier solver with a neural generator to capture the transition between ballistic and diffusive heat transport [1]. Both applications implement the same general structure described above, but differ in the parametrization used, the physics solved by the low- and high-fidelity solvers, and the way gradients are obtained during training.

2.1.1.1 Diffusion, reaction-diffusion, and Maxwell surrogates

In [20], the design vector $\mathbf{p}$ encodes the widths of a grid of air holes cut into a background medium, producing a parameterized geometry. For the diffusion and reaction-diffusion cases, $\mathbf{p}$ has 16 or 25 entries corresponding to a 4×4 or 5×5 grid of hole widths, while for the Maxwell case $\mathbf{p}$ has 10 entries describing the widths of ten layers of holes in a metasurface unit cell.

The high-fidelity solver builds a fine finite-difference discretization $hf(\mathbf{p})$ of this geometry and solves either the linear diffusion equation, the nonlinear reaction-diffusion equation, or the 2D Maxwell equations, depending on the case. The target property is, respectively, the thermal flux, the reaction-diffusion flux, or the complex transmission coefficient. The low-fidelity solver uses the same finite-difference method as the high-fidelity solver, but at a much coarser resolution (as low as one pixel per hole), and, in the reaction-diffusion case, further simplifies the physics by discarding the nonlinear term of the PDE.

The baseline $downsample(\mathbf{p})$ is obtained by sub-pixel averaging: the fine geometry is coarsened onto the resolution required by the low-fidelity solver, with each coarse pixel assigned the fractional area of solid versus pore material it contains.

2.1.1.2 Phonon Boltzmann Transport surrogate

In [1], PEDS is applied to the design of nanostructured thermal materials, where the goal is to predict the effective thermal conductivity of a material perforated with small pores. The design focus is this time a topology: a grid representation of the material in which each cell is assigned a value indicating whether that region is solid or empty (a pore). The design vector $\mathbf{p}$, denoted G in [1], is a 25-dimensional binary vector describing a 5×5 grid of embedded pores in a silicon domain, where the width of the pores does not vary as in the previous example. The high-fidelity solver builds a fine mesh from this topology and solves the phonon BTE under the relaxation-time approximation, returning the effective thermal conductivity κ of the nanostructured material as the target property.

The low-fidelity solver is a differentiable Fourier (heat-conduction) solver, which approximates the BTE in the diffusive limit and can be in error by several hundred percent for strongly ballistic geometries,

but is roughly three orders of magnitude faster to evaluate. The baseline $\text{downsample}(G)$ is a coarse conductivity map built directly from the binary topology vector, mapping solid and pore cells onto a coarse $N \times N$ conductivity field that the Fourier solver can use; this map plays the same role as sub-pixel averaging in the earlier study, but is adapted to a binary porosity representation rather than a continuous geometry.

The neural generator $\text{generator}_{NN}(G)$ learns a correction to this baseline conductivity field, and the mixing weight w_ϕ is itself learned as a function of G rather than treated as a single global constant, allowing the balance between the Fourier baseline and the network correction to adapt locally between the diffusive and ballistic regimes.

2.1.1.3 Overall benefits

Across both applications, PEDS delivers three main advantages compared to purely data-driven surrogates: higher accuracy for a given training budget, large reductions in the amount of expensive high-fidelity data required, and physically consistent outputs that also offer some insight into the underlying process. In terms of accuracy, the PEDS surrogate significantly improves both predictive performance compared to a pure neural-network and the outcome of using the solver alone. On the diffusion, reaction-diffusion, and Maxwell benchmarks, PEDS reduces the fractional error by up to a factor of three compared to a neural-network-only baseline with the same number of parameters, and by up to 28% compared to the low-fidelity solver alone [20]. Similarly, for the phonon Boltzmann transport surrogate, PEDS reduces the test error by up to 76% relative to a purely data-driven baseline trained on the same number of samples, and improves on the low-fidelity Fourier solver alone by a much larger margin, bringing an error of up to almost 700% (200% on average) down to the target 5% [1].

The use of this coupled framework also leads to a significant gain in training-data efficiency. Because the low-fidelity solver already encodes a first approximation of the governing physics, the neural network only needs to learn a correction, rather than the full input-to-output mapping. This reduces the number of costly high-fidelity simulations needed to reach a target error by at least two orders of magnitude in the diffusion, reaction-diffusion, and Maxwell cases [20], and allows the thermal-conductivity surrogate to reach a 5% target error with only 300 high-fidelity BTE evaluations when combined with active learning. In an inverse-design setting, this data efficiency is significant enough that the upfront cost of training the surrogate is fully recovered after only four design evaluations [1].

Combining PEDS with active learning amplifies these benefits further, since new training points can be selected where the surrogate is most uncertain, rather than sampled at random. This combination lowers the fractional error on the Maxwell surrogate by almost a factor of three in the low-data regime, and allows the model to reach errors that a purely data-driven baseline cannot match even with two orders of magnitude more data [20]. The same pattern is observed for the phonon transport case, where active learning reduces the error by roughly 70% compared to PEDS trained on randomly sampled data [1].

Beyond accuracy and data efficiency, keeping the low-fidelity solver inside the model also preserves physically meaningful properties of the output, such as conservation of mass or energy, which are not automatically guaranteed by a purely data-driven surrogate. Because the neural network only modifies

the input to this solver, its output can also be inspected after training, offering some physical insight into the correction being learned, rather than remaining a black box. Finally, since the low-fidelity solver is typically two to four orders of magnitude faster than the high-fidelity reference, the resulting surrogate remains inexpensive to evaluate at inference time, making it attractive for large-scale design and optimization tasks

2.1.2 Adjoint method

As discussed above, the low-fidelity solver embedded in the network is generally not expressed in a form that automatic differentiation can trace efficiently, and repeated finite-difference evaluations scale poorly with the number of network parameters. The adjoint method, a well-established technique in sensitivity analysis [21–23], resolves this issue by providing the required gradient at the cost of a single additional solve, independent of the number of parameters. This subsection derives the method in general form; the specific application to the eigenvalue problem of neutron transport follows in Subsubsection 2.2.1.1. Consider a parameterized linear system $\mathbf{M}(\mathbf{p})\mathbf{x} = \mathbf{s}(\mathbf{p})$, where $\mathbf{x} \in \mathbb{R}^M$ is the solution vector to be determined and $\mathbf{p} \in \mathbb{R}^P$ is a vector of parameters upon which the system matrix $\mathbf{M}$ and source term $\mathbf{s}$ depend. Written as a residual, the problem is

$$\mathcal{R}(\mathbf{x}, \mathbf{p}) = \mathbf{M}(\mathbf{p})\mathbf{x} - \mathbf{s}(\mathbf{p}) = \mathbf{0}. \quad (2.4)$$

Differentiating Eq. (2.4) with respect to p and applying the product rule gives

$$\frac{\partial \mathbf{M}}{\partial \mathbf{p}} \mathbf{x} + \mathbf{M} \frac{\partial \mathbf{x}}{\partial \mathbf{p}} - \frac{\partial \mathbf{s}}{\partial \mathbf{p}} = \mathbf{0}, \quad (2.5)$$

where each term $\in \mathbb{R}^{M \times P}$, noting that in $\frac{\partial \mathbf{M}}{\partial \mathbf{p}} \mathbf{x}$ each column corresponds to $\frac{\partial \mathbf{M}}{\partial p_j} \mathbf{x}$ for every parameter j in $\mathbf{p}$. The state Jacobian formulation follows:

$$\frac{\partial \mathbf{x}}{\partial \mathbf{p}} = \mathbf{M}^{-1} \left(\frac{\partial \mathbf{s}}{\partial \mathbf{p}} - \frac{\partial \mathbf{M}}{\partial \mathbf{p}} \mathbf{x} \right). \quad (2.6)$$

Forming $\partial \mathbf{x} / \partial \mathbf{p} \in \mathbb{R}^{M \times P}$ explicitly would require P solves of an $M \times M$ system, one for every parameter, which becomes prohibitive as P grows. In practice, however, the network training only requires the chain-rule product:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} \cdot \frac{\partial \mathbf{x}}{\partial \mathbf{p}}, \quad (2.7)$$

where $\mathcal{L}$ is the loss function, and $\partial \mathcal{L} / \partial \mathbf{x}$ is a fixed, known row vector that multiplies the Jacobian. Defining the column vector $\mathbf{v} \equiv (\partial \mathcal{L} / \partial \mathbf{x})^T \in \mathbb{R}^{M \times 1}$, this product is a vector-Jacobian product (VJP):

$$\mathbf{v}^T \frac{\partial \mathbf{x}}{\partial \mathbf{p}} = \mathbf{v}^T \mathbf{M}^{-1} \left(\frac{\partial \mathbf{s}}{\partial \mathbf{p}} - \frac{\partial \mathbf{M}}{\partial \mathbf{p}} \mathbf{x} \right). \quad (2.8)$$

The key step is to group the terms so that $\mathbf{M}^{-1}$ is applied to $\mathbf{v}^T$, which is a row vector, rather than to the $M \times P$ matrix on the right. To do this, the adjoint variable $\boldsymbol{\lambda} \in \mathbb{R}^{M \times 1}$ is introduced as the solution of a single linear system, constituting the adjoint equation:

$$\mathbf{M}^T \boldsymbol{\lambda} = \mathbf{v}. \quad (2.9)$$

Substituting λ back into Eq. (2.8) gives the final gradient in closed form:

$$\mathbf{v}^T \frac{\partial \mathbf{x}}{\partial \mathbf{p}} = \lambda^T \left(\frac{\partial \mathbf{s}}{\partial \mathbf{p}} - \frac{\partial \mathbf{M}}{\partial \mathbf{p}} \mathbf{x} \right). \quad (2.10)$$

This result is the reason the adjoint method is attractive for training: when calculating λ first, the gradient with respect to all P parameters is obtained from one solve of Eq. (2.9), of the same size and structure as the original problem, rather than from P separate solves of the state Jacobian. This general derivation applies directly to the neutron transport case, where the governing equations take the form of an eigenvalue problem rather than a simple linear system, and the adjoint equation coincides with the physical adjoint neutron flux, further discussed in Subsubsection 2.2.1.1.

2.2 PEDS application to neutron transport

The PEDS methodology was introduced only recently in the scientific machine learning literature, and has so far been demonstrated on a small number of PDE problems. Because it embeds a physical solver directly into the model, allowing the network only a controlled and limited influence over the final prediction, the framework is particularly well suited to fields where physical trust in the surrogate output is important, such as neutron transport calculations in nuclear engineering. This section maps the general PEDS structure introduced above onto this new setting, identifying the main framework elements adaptation.

Reactor design and operational studies rely on Monte Carlo simulations (further described in Subsection 2.2.2) for their inherent accuracy, since these methods resolve particle behaviour using continuous-energy (CE) cross-section data with minimal approximation. This accuracy, however, comes at a substantial computational cost, often on the order of hours or days per calculation, making it a perfect fit for the role of the high-fidelity solver f^{hf} of the PEDS method. To avoid this cost, deterministic methods solve a simplified form of the neutron transport equation, presented in Subsection 2.2.1. Their usage requires to discretise space and energy to use multigroup cross sections (MGXS) to use on a grid, on which the equation is calculated. These methods differ in how they treat the angular dependence: some rely on spherical harmonics expansions, others on the method of characteristics, and others further simplify the problem through the diffusion approximation [24, 25].

This latter method will be used as the low-fidelity solver f^{lf} for this first application of PEDS to neutron transport, further described in Section 3.1. In this context, "low-fidelity" refers not only to physical simplifications, such as the diffusion assumption, but also to numerical resolution used in the calculation, like employing a coarse spatial mesh or a reduced number of energy groups. These choices deliberately trade some baseline precision to achieve the rapid simulation speeds required for extensive design studies, relying on the neural network correction phase to effectively bridge the resulting accuracy gap.

With the HF and LF solvers identified, the design vector $\mathbf{p}$ is defined as a set of region-wise material parameters (enrichment, moderator type, etc.) and geometric parameters (reactor size, pin radius, pitch, etc.), from which cross sections at different levels of fidelity are built. The resulting application is a reactor system constructed from $\mathbf{p}$, whose property of interest is the effective multiplication factor k_{eff} .

The HF solver builds its problem definition by applying CE cross sections $\sigma_{CE}(E; \mathbf{p})$ to the geometry specified by $\mathbf{p}$, producing a finely resolved representation of the system and, from it, the reference solution.

In the general PEDS formulation, the low-fidelity solver input is built combining a downsampled version of the geometry with the generator network output through the mixing weight (Equation 2.2). This baseline field acts as a first, physics-based estimate, later refined by the network to bring the output closer to the target. For the neutron transport application, the conceived analogue of it is a coarse estimate of the multigroup cross sections (MGXS), corresponding to the input required by the diffusion solver. Computing this baseline for the exact geometry parametrization, however, would require a full Monte Carlo calculation (Subsubsection 2.2.2.1) for every new sample, reintroducing the same computational bottleneck that PEDS is meant to avoid. This is circumvented by approximating the baseline rather than computing it exactly. Since this baseline serves only as a guide, what really matters is primarily the order of magnitude of the MGXS set, while the network is responsible for further refining the input data. A fast degree-2 polynomial regression model is hence trained to map the parameter vector $\mathbf{p}$ directly to an approximate cross-section set, with each cross section modeled independently as

$$\Sigma_{base}^{MGXS}(\mathbf{p}) = \boldsymbol{\beta}^\top \boldsymbol{\xi}(\mathbf{p}), \quad (2.11)$$

where $\boldsymbol{\xi}(\mathbf{p})$ is the degree-2 polynomial expansion of $\mathbf{p}$ and $\boldsymbol{\beta}$ is fit by ordinary least squares. Training data comes from the Monte Carlo runs already generated for the surrogate, so no additional high-fidelity evaluations are needed. Fit quality is not critical, since the model only needs to provide a reasonable starting point for the network to correct.

Having defined the role of the baseline, the way it is combined with the network's output differs slightly from the original formulation of the PEDS. Rather than mixing two independently generated representations through a weighted sum, here the regression baseline directly serves as the network's first guess, which is then corrected by the network. It outputs a set of multiplicative ratios that modify the baseline, producing a tensor with the same shape as the MGXS field, representing an adjusted cross-section set to bring the low-fidelity solver result closer to the target.

$$\Sigma_{final}^{NN}(\mathbf{p}) = \Sigma_{base}^{MGXS}(\mathbf{p}) \times \text{NN}_\theta(\mathbf{p}) \quad (2.12)$$

This ratio form is better suited to cross sections, whose magnitudes span several orders of magnitude across groups and reactions, than the additive mixture used in the original formulation; the underlying principle, however, is unchanged: the network corrects the baseline more or less strongly depending on how far it already is from the high-fidelity result. Further architectural details of the generator network are given in Subsection 3.3.4. Figure 2.2 presents a visual scheme of how the elements described above are combined, in this adaptation of PEDS to neutron transport.

As explained in Section 2.1, the gradient required to train the network can be obtained through a manually derived adjoint rule rather than through automatic differentiation of the solver. In this neutron transport

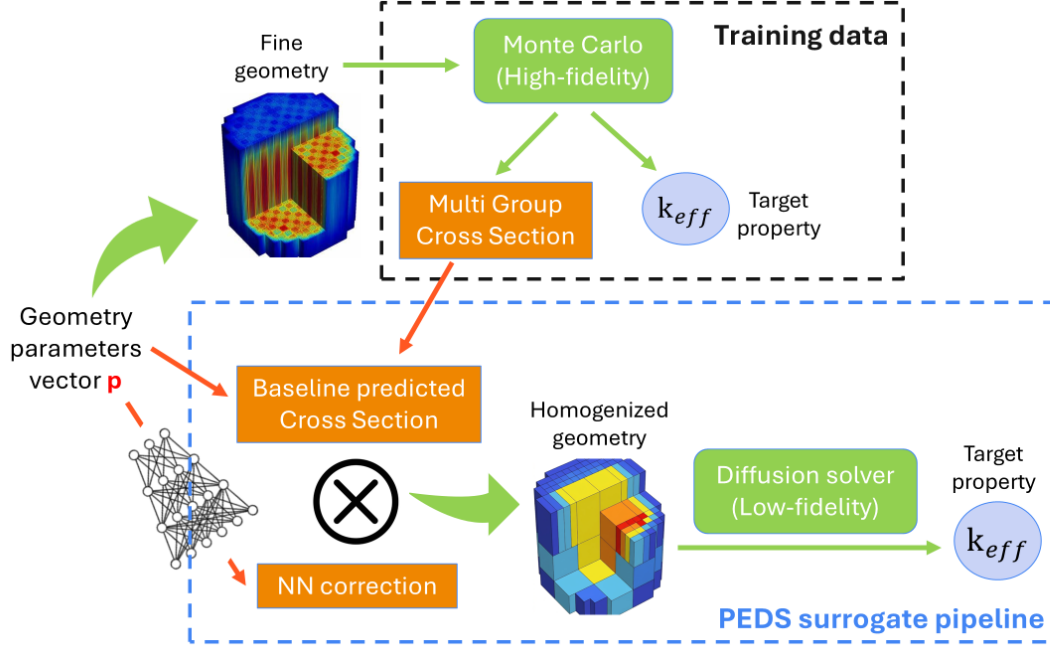


Figure 2.2: Flowchart of the PEDS architecture adapted for neutron transport calculations. The generator network outputs a set of multiplicative corrections to refine a baseline cross-section estimate, producing an optimized multigroup cross-section tensor to be evaluated by the low-fidelity diffusion solver.

application, the quantity of interest is the sensitivity of the solver output, k_{eff} , with respect to its input, the XS set, that is, the Jacobian of k_{eff} with respect to the cross sections. Computing this sensitivity is a standard practice in reactor physics sensitivity analysis [26], which makes deriving a custom gradient rule for the solver a natural step for this application (derivation presented in Subsection 2.2.3). Removing the need to trace the solver’s internal operations gives a significant advantage to the PEDS methodology: because the LF solver is treated as an external callback, it could in principle be replaced with any other deterministic solver, provided it accepts an MGXS-based input and returns k_{eff} , without requiring that solver to be rewritten in a differentiable form.

One thing that differs from previous PEDS applications is the scale of the required correction on the output property. In the phonon Boltzmann transport case, the low-fidelity solver was in error by 200% relative to the high-fidelity reference, hence the resulting improvement from PEDS was correspondingly large. In the present application, the diffusion approximation typically deviates from the Monte Carlo reference by less than 10% in k_{eff} . Despite being numerically small, this discrepancy is highly significant: reactor systems are extremely sensitive to reactivity, and an error of only a few percent in k_{eff} can be enough to shift a system between subcritical and supercritical states. Correcting this smaller, but crucial for safety, gap is therefore the target of the present application, rather than the large accuracy gain seen in the phonon transport case.

This section aim was to present a high-level description of the PEDS adaptation to neutron transport; detailed specifications on the actual code-level network development can be found in Chapter 3.

2.2.1 Boltzmann NT equation and diffusion approximation

Given this application focus on neutron transport, the following section presents the equation governing it, and traces the derivation of the neutron diffusion approximation, on which the low-fidelity solver used in the PEDS network is based on. The rigorous description of neutron distribution within a reactor core is governed by the steady-state Boltzmann transport equation, which, in the presence of multiplying media, takes the form of an eigenvalue problem [24, 27]:

$$\begin{aligned} \hat{\Omega} \cdot \nabla \psi(\vec{r}, \hat{\Omega}, E) + \Sigma_t(\vec{r}, E) \psi(\vec{r}, \hat{\Omega}, E) = \\ \int_0^\infty dE' \int_0^{4\pi} d\hat{\Omega}' \Sigma_s(\vec{r}, E' \rightarrow E, \hat{\Omega}' \rightarrow \hat{\Omega}) \psi(\vec{r}, \hat{\Omega}', E') \\ + \frac{\chi(E)}{4\pi k_{\text{eff}}} \int_0^\infty dE' \int_0^{4\pi} d\hat{\Omega}' \nu \Sigma_f(\vec{r}, E') \psi(\vec{r}, \hat{\Omega}', E') \end{aligned} \quad (2.13)$$

where the variables are defined as follows:

- $\psi(\vec{r}, E, \hat{\Omega})$: angular neutron flux (neutrons/cm²/s/sr/MeV).
- Σ_t : total macroscopic cross section (1/cm).
- Σ_s : scattering kernel.
- Σ_f, ν : fission cross section and average neutrons per fission.
- $\chi(E)$: fission spectrum.
- k_{eff} : multiplication eigenvalue.

where $\psi(\vec{r}, E, \hat{\Omega})$ denotes the angular neutron flux (neutrons/cm²/s/sr/MeV), Σ_t is the total macroscopic cross section (1/cm), and Σ_s denotes the scattering kernel. Furthermore, Σ_f and ν are the fission cross section and the average number of neutrons produced per fission, respectively. The fission spectrum is represented by $\chi(E)$, while k_{eff} denotes the multiplication eigenvalue.

Grouping all the elements in a more compact operator form yields:

$$\mathcal{L}(\psi) = \mathcal{S}(\psi) + \frac{1}{k_{\text{eff}}} \mathcal{F}(\psi), \quad (2.14)$$

where $\mathcal{L}$ represents the streaming + removal operator, $\mathcal{S}$ the scattering operator, and $\mathcal{F}$ the fission production operator. Note that the operators are determined by the materials and geometry (via the macroscopic cross-sections), and they operate on the angular flux. This compact form makes the eigenvalue structure of the problem more explicit, while the equation remains continuous in energy, space, and angle. To make the problem computationally tractable, two levels of approximation are applied. First, the energy variable is discretised using the multigroup approximation, which divides the continuous energy spectrum into G discrete groups and replaces the continuous energy integrals with group-averaged quantities [7]. Second, the angular dependence is simplified. While high-fidelity methods retain the full transport equation by discretising angles (e.g., via discrete ordinates or the Method of Characteristics), lower-fidelity deterministic solvers often apply the diffusion approximation.

This consists of integrating the transport equation over all directions $\hat{\Omega}$ and employing Fick's Law, $\vec{J} = -D\nabla\phi$, to relate the neutron current $\vec{J}$ to the gradient of the scalar flux $\phi(\vec{r}, E)$.

For Fick's Law and the resulting diffusion approximation to be physically valid, several conditions regarding the physical system must be satisfied:

1. the medium must be weakly absorbing, meaning the absorption cross-section is significantly smaller than the scattering cross-section ($\Sigma_a \ll \Sigma_s$).
2. The neutron flux must be a slowly varying function of space. Consequently, the diffusion approximation breaks down in regions with steep flux gradients, such as within a few mean free paths of strong localized absorbers (e.g., control rods), external vacuum boundaries, or sharp material interfaces.
3. Neutron scattering is assumed to be primarily isotropic or linearly anisotropic.

When these assumptions hold, the steady-state neutron diffusion equation in its multigroup formulation can be written as:

$$-\nabla \cdot D_g(\vec{r})\nabla\phi_g(\vec{r}) + \Sigma_{r,g}(\vec{r})\phi_g(\vec{r}) = \frac{1}{k}\chi_g \sum_{g'=1}^G \nu\Sigma_{f,g'}(\vec{r})\phi_{g'}(\vec{r}) + \sum_{g' \neq g} \Sigma_{s,g' \rightarrow g}(\vec{r})\phi_{g'}(\vec{r}) \quad (2.15)$$

Where:

- $\phi_g(\vec{r}) = \int_g \int_{4\pi} \psi(\vec{r}, \hat{\Omega}, E) d\hat{\Omega} dE$ is the scalar flux obtained by integrating the angular flux over all directions within energy group g .
- $D_g(\vec{r}) = [3\Sigma_{tr}(\vec{r})]^{-1}$ is the diffusion coefficient.
- $\Sigma_{r,g} = \Sigma_{a,g} + \sum_{g' \neq g} \Sigma_{s,g \rightarrow g'}$ is the removal cross-section (accounting for absorption plus scattering out of group g).
- $\Sigma_{s,g' \rightarrow g}$ is the macroscopic scattering cross-section from group g' into group g .
- χ_g quantifies the fraction of fission neutrons born into group g .

This simplification eliminates the angular variable entirely, at the cost of reduced accuracy in regions where the neutron flux is highly anisotropic. Finally, spatial discretisation (e.g., via finite difference, finite volume, or finite element methods) transforms the remaining continuous spatial integral operators into a discrete matrix system, which can be solved utilizing standard numerical linear algebra techniques. The equation thus becomes a discrete generalized eigenproblem of the form [24]:

$$A(\Sigma)\phi = \frac{1}{k_{\text{eff}}} F(\Sigma)\phi \iff (A - \lambda F)\phi = 0 \quad (2.16)$$

where:

- $\phi \in \mathbb{R}^M$: the discrete forward scalar flux vector (encompassing space and energy groups), representing the eigenvector of the problem.
- $A(\Sigma)$: the discrete loss matrix representing leakage, absorption, and out-scattering, which depends on the macroscopic cross-section tensor Σ .

- $\mathbf{F}(\Sigma)$: the discrete fission production matrix, also dependent on Σ .
- $\lambda = 1/k_{\text{eff}}$: the principal eigenvalue.
- Σ : the multigroup cross-section tensor defining the neutronic properties of the discrete spatial cells (e.g., $\Sigma_t, \Sigma_s, \nu\Sigma_f$).

For a system with N spatial mesh points and G energy groups, this yields a set of $N \times G$ simultaneous linear algebraic equations to determine the $N \times G$ flux elements and the multiplication eigenvalue k_{eff} .

2.2.1.1 Adjoint formulation of the NT equation

Having established the forward diffusion approximation to describe the physical neutron distribution, to complete the theoretical framework it is relevant to define its mathematical counterpart: the adjoint problem. In reactor physics, the adjoint flux is presented as an importance function for a specific system response (such as the multiplication factor or a localized detector reaction rate). Physically, it quantifies how heavily a neutron born at a given phase-space coordinate will contribute to that response. The adjoint transport equation for a steady-state multiplying system is derived by applying the mathematical adjoint to the direct Boltzmann operator (eq. 13.17 in [24]):

$$\begin{aligned} -\hat{\Omega} \cdot \nabla \psi^\dagger(\vec{r}, E, \hat{\Omega}) + \Sigma_t(\vec{r}, E) \psi^\dagger(\vec{r}, E, \hat{\Omega}) \\ = \int_0^\infty dE' \int_0^{4\pi} d\hat{\Omega}' \Sigma_s(\vec{r}, E \rightarrow E', \hat{\Omega} \rightarrow \hat{\Omega}') \psi^\dagger(\vec{r}, E', \hat{\Omega}') \\ + \frac{\nu(E) \Sigma_f(\vec{r}, E)}{4\pi k_{\text{eff}}} \int_0^\infty dE' \int_0^{4\pi} d\hat{\Omega}' \chi(E') \psi^\dagger(\vec{r}, E', \hat{\Omega}') \end{aligned} \quad (2.17)$$

where $\psi^\dagger(\mathbf{r}, E, \hat{\Omega})$ represents the continuous adjoint angular flux. Following the same multigroup energy and spatial discretisation procedures applied to the forward equation, the adjoint problem reduces to a corresponding discrete generalized eigenproblem:

$$\mathbf{A}(\Sigma)^\dagger \phi^\dagger = \frac{1}{k_{\text{eff}}} \mathbf{F}(\Sigma)^\dagger \phi^\dagger \iff (\mathbf{A}(\Sigma)^\dagger - \lambda^\dagger \mathbf{F}(\Sigma)^\dagger) \phi^\dagger = 0 \quad (2.18)$$

where $\phi^\dagger \in \mathbb{R}^M$ is the discrete adjoint scalar flux vector, and $\mathbf{A}(\Sigma)^\dagger$ and $\mathbf{F}(\Sigma)^\dagger$ are the transposes of the discrete loss and fission production matrices, respectively.

It is a fundamental property of these linear operators that the continuous adjoint and forward eigenvalues are strictly identical ($\lambda^\dagger = \lambda$). Consequently, both systems yield the exact same effective multiplication factor, k_{eff} , while their corresponding eigenvectors, being the forward and adjoint fluxes, differ in their physical interpretation.

2.2.2 Monte Carlo codes

Unlike deterministic solvers, Monte Carlo codes do not discretise the Boltzmann transport equation in space, angle, or energy. Instead, they estimate its solution stochastically, by simulating the histories

of a large number of individual particles whose interactions (streaming, scattering, absorption, fission) are sampled from probability distributions built from the underlying cross-section data. Because each particle history is sampled independently and the physical model is not discretised, the result carries no truncation error from spatial, angular, or energy approximations, but is instead subject to a statistical uncertainty that decreases with the number of simulated particles. More information on this methods can be found in [28].

Several Monte Carlo codes are widely used in reactor physics. MCNP [4], is one of the most established general-purpose codes for coupled neutron, photon, and electron transport. Serpent is a continuous-energy code specialised in lattice physics and the generation of homogenised group constants for deterministic solvers [3]. This work employs OpenMC, a more recent community-developed transport code built for modern high-performance computing architectures, supporting both continuous-energy and multigroup transport with a fully open-source codebase [5]. Its increasing adoption in the reactor physics community make it a practical choice as the high-fidelity reference solver for the PEDS network adaptation to neutron transport.

2.2.2.1 Multigroup cross-sections generation

As previously mentioned, deterministic solvers require a precomputed multigroup cross-section (MGXS) set as input, condensed onto the chosen energy-group structure and spatial mesh. Because Monte Carlo codes resolve neutron interactions using continuous-energy data with minimal approximation, they are commonly used to generate these group constants. In this work, OpenMC is used for this purpose: since it solves the transport equation stochastically rather than on a discretised group structure, the multigroup constants of Equation 2.13 cannot be read directly from the code output. Instead, they must be reconstructed as a post-processing step from tallied reaction rates and fluxes [29].

A general macroscopic multigroup cross section for reaction x , spatial region k , and energy group g is defined as the ratio between the tallied groupwise reaction rate and the tallied groupwise flux:

$$\hat{\Sigma}_{x,k,g} = \frac{\langle \Sigma_x, \psi \rangle_{k,g}}{\langle \psi \rangle_{k,g}}, \quad (2.19)$$

where $\langle \cdot, \cdot \rangle_{k,g}$ denotes the inner product over region k and energy group g , and $\langle \psi \rangle_{k,g}$ is the corresponding energy- and space-integrated flux tally. A key requirement for these tallies is that condensing the continuous-energy problem onto the group structure must preserve its reaction rates, so that the fission source and every other reaction term in Equation 2.13 remain consistent with the reference simulation once used by the diffusion solver.

The tallies and arithmetic operations needed to compute all multigroup constants used in this work, including the treatment of scattering matrices and the transport correction, are implemented in OpenMC through the `openmc.mgxs` Python module. Further practical detail on generating multigroup cross sections with this module is available in the official OpenMC documentation [30].

2.2.3 Derivation of vJp for gradients definition

Training the generator network requires the gradient of the loss with respect to the network weights θ , obtained via the chain rule as

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial \Sigma} \cdot \frac{\partial \Sigma}{\partial \theta}, \quad (2.20)$$

where $\Sigma \in \mathbb{R}^M$ is the macroscopic multigroup cross-section tensor produced by the network (Equation 2.12), and $\partial \Sigma / \partial \theta$ is obtained through standard automatic differentiation of the network itself. As motivated previously, the term $\partial \mathcal{L} / \partial \Sigma$ is instead obtained analytically via the adjoint method, since the solver is treated as an external, non-differentiable callback. This subsection derives an analytical backward pass for the neutron transport eigenvalue problem, which requires only a single additional adjoint solve.

Since $\mathcal{L}$ depends on Σ only through the eigenvalue k_{eff} , the required gradient reduces to $\partial k_{\text{eff}} / \partial \Sigma$. Because k_{eff} depends on Σ both directly, through the operators $\mathbf{A}(\Sigma)$ and $\mathbf{F}(\Sigma)$, and indirectly, through the flux $\phi(\Sigma)$ that solves Equation 2.16, the full chain rule gives a direct and an indirect contribution [26]:

$$\frac{dk_{\text{eff}}}{d\Sigma_m} = \underbrace{\frac{\partial k_{\text{eff}}}{\partial \phi} \frac{\partial \phi}{\partial \Sigma_m}}_{\text{indirect effect}} + \underbrace{\frac{\partial k_{\text{eff}}}{\partial \Sigma_m}}_{\phi \text{ fixed}}, \quad m = 1, \dots, M \quad (2.21)$$

where Σ_m denotes the m -th entry of the macroscopic cross-section tensor Σ , a convention used to perform the derivation for a single scalar component in order to preserve notational clarity. The indirect term captures how a change in Σ_m shifts the flux ϕ , which in turn affects k_{eff} ; computing it directly would require solving the state Jacobian $\partial \phi / \partial \Sigma_m$ for every one of the M entries of Σ , which is the cost the adjoint method is meant to avoid.

This cost can be avoided because k_{eff} admits a variational property at the solution. Expressing k_{eff} as the Rayleigh quotient of the discretised operators, [24, p. 500, Eq. (13.63)]:

$$k_{\text{eff}} = \frac{\langle \phi^\dagger, \mathbf{F}\phi \rangle}{\langle \phi^\dagger, \mathbf{A}\phi \rangle} = \frac{\phi^{\dagger T} \mathbf{F}\phi}{\phi^{\dagger T} \mathbf{A}\phi} \quad (2.22)$$

Differentiating the Rayleigh quotient with respect to ϕ via the quotient rule and assuming $\phi^\dagger$ is the associated adjoint eigenfunction, one obtains:

$$\frac{\partial k_{\text{eff}}}{\partial \phi} = \frac{2 \left[(\phi^{\dagger T} \mathbf{A}\phi) \mathbf{F}\phi - (\phi^{\dagger T} \mathbf{F}\phi) \mathbf{A}\phi \right]}{(\phi^{\dagger T} \mathbf{A}\phi)^2} \quad (2.23)$$

Substituting $\mathbf{F}\phi = k_{\text{eff}} \mathbf{A}\phi$ from Equation 2.16:

$$\frac{\partial k_{\text{eff}}}{\partial \phi} = \frac{(\phi^{\dagger T} A \phi) k_{\text{eff}} A \phi - (\phi^{\dagger T} k_{\text{eff}} A \phi) A \phi}{(\phi^{\dagger T} A \phi)^2} = \mathbf{0} \quad (2.24)$$

To evaluate the indirect effect from Equation 2.21, the Rayleigh quotient is differentiated with respect to the forward flux vector ϕ via the quotient rule. Noting that the derivative of the bilinear form $\phi^{\dagger T} \mathbf{M} \phi$ with respect to ϕ is the row vector $\phi^{\dagger T} \mathbf{M}$, one obtains:

$$\frac{\partial k_{\text{eff}}}{\partial \phi} = \frac{(\phi^{\dagger T} \mathbf{A} \phi) \phi^{\dagger T} \mathbf{F} - (\phi^{\dagger T} \mathbf{F} \phi) \phi^{\dagger T} \mathbf{A}}{(\phi^{\dagger T} \mathbf{A} \phi)^2} \quad (2.25)$$

By definition of the Rayleigh quotient, $\phi^{\dagger T} \mathbf{F} \phi = k_{\text{eff}} (\phi^{\dagger T} \mathbf{A} \phi)$. Substituting this into the numerator:

$$\frac{\partial k_{\text{eff}}}{\partial \phi} = \frac{(\phi^{\dagger T} \mathbf{A} \phi) \phi^{\dagger T} \mathbf{F} - k_{\text{eff}} (\phi^{\dagger T} \mathbf{A} \phi) \phi^{\dagger T} \mathbf{A}}{(\phi^{\dagger T} \mathbf{A} \phi)^2} = \frac{\phi^{\dagger T} (\mathbf{F} - k_{\text{eff}} \mathbf{A})}{\phi^{\dagger T} \mathbf{A} \phi} \quad (2.26)$$

Assuming the vector $\phi^{\dagger}$ is chosen to be the fundamental adjoint flux, it satisfies the discrete adjoint eigenvalue problem $(\mathbf{F}^T - k_{\text{eff}} \mathbf{A}^T) \phi^{\dagger} = \mathbf{0}$ from Equation 2.18. Transposing this relationship yields $\phi^{\dagger T} (\mathbf{F} - k_{\text{eff}} \mathbf{A}) = \mathbf{0}^T$. Therefore:

$$\frac{\partial k_{\text{eff}}}{\partial \phi} = \mathbf{0}^T \quad (2.27)$$

This represents the variational property of the Rayleigh quotient: the sensitivity of k_{eff} to first-order perturbations in the forward flux is exactly zero. It follows that only the direct effect has to be taken care of now, meaning that even though the flux ϕ depends on Σ (the state Jacobian $\partial \phi / \partial \Sigma_m$), that dependence does not affect the eigenvalue k_{eff} to first order:

$$\frac{dk_{\text{eff}}}{d\Sigma_m} = \left. \frac{\partial k_{\text{eff}}}{\partial \Sigma_m} \right|_{\phi \text{ fixed}}. \quad (2.28)$$

To compute the direct effect without forming the full $N \times M$ Jacobian of ϕ with respect to Σ , the adjoint method introduced in Subsection 2.1.2 is used. Even with the indirect term removed, the direct effect still depends implicitly on ϕ through the residual of the eigenvalue problem,

$$\mathcal{R}(\phi, k_{\text{eff}}, \Sigma) = \mathbf{A}(\Sigma) \phi - \frac{1}{k_{\text{eff}}} \mathbf{F}(\Sigma) \phi = \mathbf{0}. \quad (2.29)$$

Differentiating Equation 2.29 totally with respect to Σ_m :

$$\underbrace{\frac{\partial \mathcal{R}}{\partial \phi}}_{N \times N} \underbrace{\frac{\partial \phi}{\partial \Sigma_m}}_{N \times 1} + \underbrace{\frac{\partial \mathcal{R}}{\partial k_{\text{eff}}}}_{N \times 1} \underbrace{\frac{\partial k_{\text{eff}}}{\partial \Sigma_m}}_{\text{scalar}} + \underbrace{\frac{\partial \mathcal{R}}{\partial \Sigma_m}}_{N \times 1} = \mathbf{0}. \quad (2.30)$$

Pre-multiplying by $\phi^{\dagger T}$ and choosing $\phi^{\dagger}$ to satisfy the adjoint equation Equation 2.18, the adjoint property gives:

$$\phi^{\dagger T} \frac{\partial \mathcal{R}}{\partial \phi} = \phi^{\dagger T} \left(\mathbf{A} - \frac{1}{k_{\text{eff}}} \mathbf{F} \right) = \mathbf{0}^T, \quad (2.31)$$

so the first term in Equation 2.30 vanishes. What remains is:

$$\phi^{\dagger T} \frac{\partial \mathcal{R}}{\partial k_{\text{eff}}} \frac{\partial k_{\text{eff}}}{\partial \Sigma_m} + \phi^{\dagger T} \frac{\partial \mathcal{R}}{\partial \Sigma_m} = 0. \quad (2.32)$$

Computing the partial derivatives from Equation 2.29:

$$\frac{\partial \mathcal{R}}{\partial \Sigma_m} = \frac{\partial \mathbf{A}}{\partial \Sigma_m} \phi - \frac{1}{k_{\text{eff}}} \frac{\partial \mathbf{F}}{\partial \Sigma_m} \phi, \quad \frac{\partial \mathcal{R}}{\partial k_{\text{eff}}} = \frac{1}{k_{\text{eff}}^2} \mathbf{F} \phi, \quad (2.33)$$

and solving for the eigenvalue sensitivity:

$$\frac{\partial k_{\text{eff}}}{\partial \Sigma_m} = \frac{\phi^{\dagger T} \left(\frac{\partial \mathbf{A}}{\partial \Sigma_m} - \frac{1}{k_{\text{eff}}} \frac{\partial \mathbf{F}}{\partial \Sigma_m} \right) \phi}{\frac{1}{k_{\text{eff}}^2} \phi^{\dagger T} \mathbf{F} \phi}. \quad (2.34)$$

This result requires only the single adjoint solve already needed to obtain $\phi^{\dagger}$, rather than one additional solve per entry of Σ . The bilinear forms in the numerator have a direct computational interpretation as vector-Jacobian products (VJPs). These are the quantities that must be defined when customizing the backward pass for the neural network, discussed in detail later on in Subsection 3.3.2.

Chapter 3

Application to diffusion equation specifics

The adaptation of the Physics-Enhanced Deep Surrogate (PEDS) methodology to neutron transport requires a development environment in which each stage of the workflow can be inspected and verified. The high-fidelity (HF) model serves exclusively as an external tool to generate the reference training data. Although this model is based on an expensive Monte Carlo calculation, it remains external to the training loop and therefore its integration into the workflow is relatively straightforward.

Conversely, the low-fidelity (LF) solver constitutes the operational core of the PEDS architecture. It must tightly couple with the neural network, evaluating the neutron transport equation for every training sample using the cross-sections updated by the neural network at every epoch. Furthermore, the forward and adjoint fluxes produced by the solver are directly utilized in the backward pass to evaluate the analytical gradients required in the backward pass. For this reason, a simplified one-dimensional solver was developed in Python rather than relying on an existing transport code. This allowed the numerical implementation to be inspected at every stage, the relevant internal quantities to be accessed directly, and intermediate results to be verified through dedicated checks.

This chapter details the specific numerical and computational implementations required to realize this framework. Section 3.1 introduces the custom low-fidelity diffusion solver, covering its spatial discretization, iterative solution method, and the crucial derivation of its discrete adjoint. Following this, Section 3.2 describes the high-fidelity setup used for dataset generation. Finally, Section 3.3 details the software integration of the PEDS architecture. It explains how the numerical solver is embedded within the machine learning graph, the translation of the analytical adjoint into a custom backward pass for gradient propagation, and its mathematical verification. The chapter concludes with a detailed technical description of the core model elements.

3.1 Low-Fidelity diffusion solver

As the computational engine of the surrogate framework, the low-fidelity solver must balance rapid execution speed with sufficient physical fidelity to capture the fundamental neutronics of the system. To fulfill this role, a custom 1D multigroup diffusion solver was developed using the finite-volume method.

This section details the mathematical and numerical foundation of the code. It begins by establishing the governing equations and their spatial discretization across different geometries. It then outlines the numerical solution strategy, and the handling of the mathematical adjoint required for the neural network training. Lastly, it concludes with an analytical benchmark to verify the numerical robustness of the solver.

3.1.1 Governing equations

A dedicated code was developed in Python to solve numerically the steady-state, multigroup neutron diffusion equation in a domain $[0, R]$, presented for energy group g in Equation 2.15, for standard one-dimensional geometries. The spatial differential operator associated with the diffusion coefficient $D(\vec{r})$ depends on the system geometry:

$$\nabla \cdot D \nabla = \begin{cases} \frac{d}{dr} D \frac{d}{dr} & \text{slab} \\ \frac{1}{r} \frac{d}{dr} r D \frac{d}{dr} & \text{cylinder} \\ \frac{1}{r^2} \frac{d}{dr} r^2 D \frac{d}{dr} & \text{sphere} \end{cases} \quad (3.1)$$

Employing the finite volume method for the spatial discretization of the problem, the domain can hence be divided into I cells of width $\Delta r = R/I$, with cell centers $r_i = (i + \frac{1}{2})\Delta r$ and cell edges $r_{i\pm 1/2} = (i \pm \frac{1}{2} + \frac{1}{2})\Delta r$ for $i = 0, 1, \dots, I - 1$. Integrating the diffusion equation over the volume of cell i and applying the central difference formula for the neutron flux definition results in the discrete equation:

$$-\frac{1}{V_i} \left[D_{i+1/2} S_{i+1/2} \frac{\phi_{i+1} - \phi_i}{\Delta r} - D_{i-1/2} S_{i-1/2} \frac{\phi_i - \phi_{i-1}}{\Delta r} \right] + \Sigma_{a,i} \phi_i = \frac{1}{k} \nu \Sigma_{f,i} \phi_i \quad (3.2)$$

where $S_{i\pm 1/2}$ and V_i represent the cell-edge surface area and the cell volume, respectively, defined according to the specific geometry:

$$S_{i+1/2} = \begin{cases} 1 & \text{slab} \\ 2\pi r_{i+1/2} & \text{cylinder} \\ 4\pi r_{i+1/2}^2 & \text{sphere} \end{cases} \quad V_i = \begin{cases} \Delta r & \text{slab} \\ \pi (r_{i+1/2}^2 - r_{i-1/2}^2) & \text{cylinder} \\ \frac{4}{3}\pi (r_{i+1/2}^3 - r_{i-1/2}^3) & \text{sphere} \end{cases} \quad (3.3)$$

To ensure neutron current continuity across adjacent cells with different material properties, the interface diffusion coefficient is computed via harmonic averaging:

$$D_{i+1/2} = \frac{2 D_i D_{i+1}}{D_i + D_{i+1}}. \quad (3.4)$$

This formulation allows material properties to be defined for each individual cell, thereby enabling a heterogeneous material structure to be represented through cell-dependent cross-sections. For example,

assuming a case with two regions describing the fuel and reflector sections of a nuclear reactor separated at the interface position r_{div} , the macroscopic cross section $\Sigma_{x,i}$ takes the form:

$$\Sigma_{x,i} = \begin{cases} \Sigma_x^{\text{core}} & r_i \leq r_{\text{div}} \\ \Sigma_x^{\text{refl}} & r_i > r_{\text{div}} \end{cases} \quad (3.5)$$

Boundary conditions are incorporated directly into these discrete cell equations. A reflective boundary condition at the domain center ($r = 0$) is naturally enforced by setting the inner surface area $A_{-1/2} = 0$, ensuring zero net current. A vacuum boundary condition at the outer surface ($r = R$) is applied by imposing a zero incoming partial current, which modifies the leakage coefficients of the outermost cell $I - 1$.

Assembling the discretized equations for all groups and all spatial cells into a global linear system yields the generalized eigenvalue problem, presented in Equation 2.16, reported below:

$$\mathbf{A}(\Sigma) \phi = \frac{1}{k_{\text{eff}}} \mathbf{F}(\Sigma) \phi \quad (3.6)$$

To solve the generalized eigenvalue problem computationally, the discrete spatial operators are assembled into global system matrices $\mathbf{A}$ and $\mathbf{F}$. A detailed mathematical expansion of these block matrices for a representative 1D, two-energy-group problem is provided in Appendix B. The size of these system matrices depends directly on the number of spatial cells, I . The choice of this mesh size involves a practical trade-off: a finer mesh (a larger I) provides better spatial resolution of the neutron flux, but it also generates larger matrices. Because the eigenvalue solution requires repeated matrix operations, larger matrices directly increase the computational time of each run. Since this solver is designed to operate as a rapid low-fidelity surrogate model, I is chosen to balance acceptable physical accuracy with fast execution speeds.

3.1.2 Solution method

To solve this generalized eigenvalue problem, the inverse power method is utilized. This method extracts the dominant mode, which in this application corresponds to the fundamental neutron flux shape and the effective multiplication factor, k_{eff} . Instead of directly computing the full inversion for the largest eigenvalue of $\mathbf{A}^{-1}\mathbf{F}$, the method repeatedly solves a linear system and updates an iterate until it converges to the dominant eigenvector. The iteration solves the transformed standard problem $\mathbf{A}^{-1}\mathbf{F} \phi = k_{\text{eff}} \phi$, and proceeds as follows:

$$\mathbf{A} \tilde{\phi}^{(n+1)} = \mathbf{F} \phi^{(n)}, \quad k_{\text{eff}}^{(n+1)} = \left\| \tilde{\phi}^{(n+1)} \right\|, \quad \phi^{(n+1)} = \frac{\tilde{\phi}^{(n+1)}}{k_{\text{eff}}^{(n+1)}} \quad (3.7)$$

where $\tilde{\phi}^{(n+1)}$ represents the unnormalized intermediate flux vector. The algorithm is initialized with a uniform random vector $\phi^{(0)}$ and iterates until the absolute change in the eigenvalue satisfies a specified convergence tolerance ε :

$$\left| k_{\text{eff}}^{(n+1)} - k_{\text{eff}}^{(n)} \right| < \varepsilon \quad (3.8)$$

During the inner iterations, the linear system $\mathbf{A}\tilde{\phi}^{(n+1)} = \mathbf{F}\phi^{(n)}$ must be solved repeatedly. Because the intra-group operators form tridiagonal matrix blocks, the expensive explicit matrix inversion is completely bypassed by applying an LU factorization, thereby minimizing the computational overhead of the solver.

3.1.3 Adjoint calculation

As motivated in Subsection 3.1.1, deriving the analytical gradient required for the neural network training necessitates the evaluation of the mathematical adjoint. The inverse power method is therefore deployed a second time to solve the discrete adjoint generalized eigenproblem:

$$\mathbf{A}^\dagger \phi^\dagger = \frac{1}{k_{\text{eff}}} \mathbf{F}^\dagger \phi^\dagger \quad (3.9)$$

producing the neutron importance function $\phi^\dagger$ as the eigenvector, and k_{eff} as the eigenvalue, as introduced in Subsubsection 2.2.1.1. It is important to note here that, within the finite-volume formulation of the solver, the discrete adjoint operator $\mathbf{A}^\dagger$ is not simply the algebraic transpose of the forward matrix, $\mathbf{A}^T$. Because cylindrical and spherical geometry cell volumes increase with radius (as defined in Equation 3.3), the discrete transpose must be weighted appropriately:

$$\mathbf{A}^\dagger = \mathbf{V}^{-1} \mathbf{A}^T \mathbf{V} \quad (3.10)$$

where $\mathbf{V} = \text{diag}(V_0, V_1, \dots, V_{I-1})$ is the diagonal matrix of the respective cell volumes. This is why the adjoint calculation with finite-volumes needs extra care for non-slab geometries. If the plain transpose $\mathbf{A}^T$ were used, the resulting solution would be non-physical, artificially suppressed at the domain center and peaking mid-radius. Incorporating the volume-weighting correction restores the proper balance between inner and outer cells, ensuring that the spatial distribution of the adjoint flux matches theoretical expectations.

As a fundamental property of self-adjoint continuous diffusion operators, the resulting discrete adjoint eigenvalue identically matches the forward k_{eff} .

3.1.4 Analytical solution check

To verify the numerical robustness of the finite-volume formulation before embedding it into the PEDS architecture, the calculated flux and eigenvalue were benchmarked against exact analytical solutions. For a simple homogeneous domain, the continuous diffusion equation can be solved analytically, yielding well-known fundamental buckling mode shapes—such as the pure cosine distribution for a 1D slab [31, 32]. Similarly, the analytical formulation of the continuous adjoint problem is well established in the literature (e.g., [33] for the two-group diffusion equation in a bare homogeneous reactor).

Figure 3.1 illustrates the spatial distribution of both the forward and adjoint fluxes for a two-group homogeneous slab geometry. The perfect overlap between the discrete numerical output and the theoretical

continuous curves confirms the correct implementation of the boundary conditions, spatial operators, and mathematical adjoint. This guarantees that the solver provides a rigorously verified physical baseline for the complex, heterogeneous parameter spaces subsequently explored by the PEDS surrogate model.

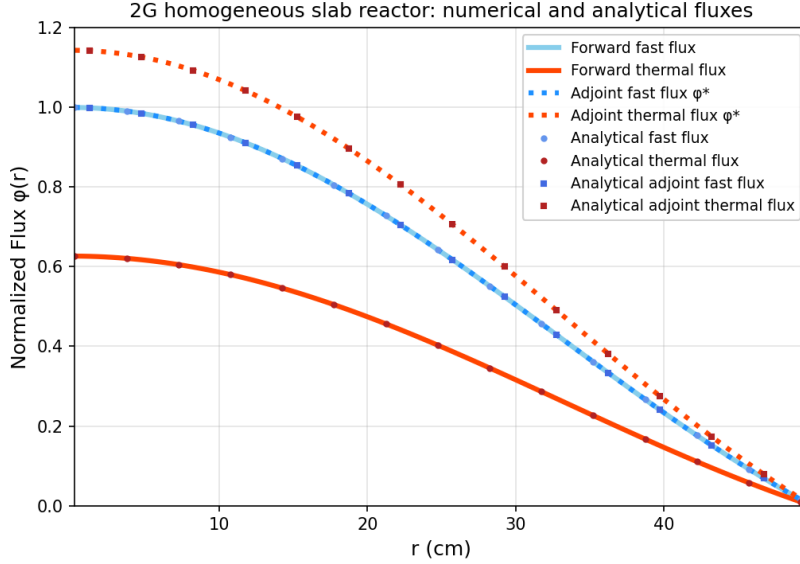


Figure 3.1: Comparison of numerical and analytical solutions for the two-group forward and adjoint fluxes in a 1D homogeneous slab.

3.2 High-Fidelity OpenMC solver

As introduced in Chapter 1, the high fidelity model was implemented using OpenMC, a Monte Carlo neutron transport code in which neutron histories are simulated by direct particle tracking. It was used to generate the reference data for the training and evaluation of the PEDS surrogate, together with the multigroup cross-sections used as targets for the polynomial regression baseline.

Mirroring what can be done with the diffusion solver for the low fidelity case, the solver handles a parametric configuration describing N concentric regions arranged with spherical, cylindrical, or slab symmetry, ordered from the centre of the domain outward. Each region is assigned a material preset together with its relevant design parameters. For example, a fuel region can be defined as a homogeneous U-235/U-238 mixture with a specified enrichment, or as a homogenised uranium water mixture with an adjustable moderation fraction. Other available presets include H₂O moderator at standard density and B₄C based absorber region, in which the B₄C fraction controls the absorption strength through a homogeneous mixture with iron. A custom preset additionally allows an arbitrary user-supplied material to be substituted into any region. The geometry builder then converts the selected regions into OpenMC surface and cell definitions, placing a vacuum boundary condition on the outermost surface and transmission reflective conditions at all internal interfaces.

Once the geometry and materials are assembled, OpenMC’s multigroup cross-section library is configured to tally G -group macroscopic cross-sections for every material region. For the following applications,

the quantities extracted are the diffusion coefficient $D_g = 1/(3\Sigma_{t,g})$, the absorption cross-section $\Sigma_{a,g}$, the fission neutron production term $\nu\Sigma_{f,g}$, the scattering matrix $\Sigma_{s,g\rightarrow g'}$ for all G^2 group pairs, and the fission spectrum χ_g . These constitute the multigroup cross-section tensor used as input by the low-fidelity diffusion solver (Section 3.1) and as regression targets for the polynomial baseline (discussed in Section 2.2).

A spatial flux tally was also defined using the same energy group boundaries. For slab geometry, uniform bins were used along the one-dimensional coordinate. For cylindrical and spherical geometries, a non-uniform radial mesh was adopted instead. The number of spatial bins was selected to match the discretisation used by the low-fidelity solver, enabling direct comparison of the two flux profiles. The tallied flux values were divided by cell volumes before storage, resulting in a volume-normalised flux that remains comparable across bins of different size.

At the end of each run, the statepoint quantity `keff`, OpenMC’s own combined eigenvalue estimator consisting of an average over all active-cycle estimates together with its associated standard deviation, is extracted and stored as the target k_{eff} for that configuration. This is the quantity the PEDS surrogate is then trained to match.

3.3 PEDS network implementation

The neutron transport application presented in this chapter builds upon the PEDS methodology previously applied to the Phonon Boltzmann Transport Equation [1]. The codebase, available on [Github](#), served as the starting point for this work. The key adaptation replaces the Gauss solver used in that prior application with a tailored implementation of the diffusion solver presented in Subsection 3.3.1. This section describes the complete PEDS pipeline for neutron transport: the integration of the external diffusion solver (Subsection 3.3.1), the custom adjoint definition for backpropagation through it (Subsection 3.3.2), and the neural network architecture that learns corrections to the cross-section inputs (Subsection 3.3.4).

3.3.1 External solver

As introduced earlier, the low-fidelity model solves the neutron diffusion equation over a finite-volume grid, exploiting the diffusion approximation. It uses the inverse power method to calculate the core eigenvalue k_{eff} and the neutron flux vectors, based on the system matrices $\mathbf{A}(\boldsymbol{\Sigma})$ and $\mathbf{F}(\boldsymbol{\Sigma})$.

The solver takes two inputs: the raw design parameters $\mathbf{p}$, which define the physical dimensions and material layout used to construct the spatial grid, and the cross-section tensor $\boldsymbol{\Sigma}$. In the full PEDS pipeline, $\boldsymbol{\Sigma}$ is supplied by the generator network rather than taken directly from a Monte Carlo simulation. To build $\boldsymbol{\Sigma}$, the neural network works alongside the solver by computing a set of targeted corrections applied on top of the initial baseline cross-sections, yielding low-fidelity simulations that closely match a high fidelity calculation.

Because the eigensolver is implemented entirely in Python using NumPy routines, its internal operations are unknown to JAX’s automatic differentiation engine. The solver is therefore wrapped in a `jax.pure_callback` for the forward pass, and a custom gradient rule describing how the output relates

to the inputs is defined separately, to handle the NN backpropagation through it. The forward physics evaluation is thus performed by the external routine, while the gradient computation for the backward pass is supplied analytically, as described in Subsection 3.3.2.

Once the calculation converges, the solver outputs the eigenvalue k_{eff} , the forward flux ϕ , and the adjoint flux $\phi^\dagger$. The predicted k_{eff} is subsequently used to evaluate the neural network's loss function (which is detailed later in Subsubsection 3.3.4.3). However, both flux vectors are determined only up to an arbitrary scaling factor. This creates a problematic ambiguity because the sensitivity formula used to calculate gradients places the inner product of these two fluxes in its denominator. If left unscaled, the arbitrary magnitude of the fluxes would directly influence the calculated gradient values, destabilizing the neural network training. To resolve this, a biorthonormalization condition is applied immediately after the solve. The adjoint flux is rescaled so that the product $\langle \phi^\dagger, \mathbf{F}\phi \rangle$ exactly equals 1, updating the adjoint flux as follows:

$$\phi^\dagger \leftarrow \frac{\phi^\dagger}{\phi^\dagger \mathbf{F}\phi}. \quad (3.11)$$

Finally, alongside returning k_{eff} , the forward pass saves k_{eff} , ϕ , $\phi^\dagger$, and the product $\mathbf{F}\phi$ as intermediate variables. By saving these states in memory, the network can efficiently reuse them during the backward pass to compute the gradients without needing to solve the system a second time.

3.3.2 Custom VJP rule

As described in the previous section, the eigenvalue solver is external, meaning its internal operations are invisible to JAX's tracing system. Consequently, gradients cannot flow through it automatically: to enable the neural network's backpropagation, a custom Vector-Jacobian Product (VJP) must be supplied manually [34]. This is achieved using the `@jax.custom_vjp` decorator utility, which separates the execution into the standard forward pass and an analytically defined backward pass, allowing the automatic differentiation engine to bypass the non-differentiable solver.

For this purpose, the adjoint method is exploited, as suggested from the previous applications of the PEDS method [1, 20]. For the neutron transport application, the analytical form of the gradient is derived in Subsection 2.2.3.

The goal of this backward pass is to compute the gradient of the loss $\mathcal{L}$ with respect to the cross-section tensor Σ . Assuming that the upstream derivative $\frac{\partial \mathcal{L}}{\partial k_{\text{eff}}}$ has already been supplied by the outer training loop, the gradient is computed via the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \Sigma} = \frac{\partial \mathcal{L}}{\partial k_{\text{eff}}} \frac{\partial k_{\text{eff}}}{\partial \Sigma}. \quad (3.12)$$

The custom rule is split into two functions. The forward function manages the execution of the external solver and, as established previously, caches the intermediate variables as JAX residuals. The backward function then retrieves these cached residuals, combines them with the upstream derivative from the loss function, and evaluates the physical sensitivity term $\frac{\partial k_{\text{eff}}}{\partial \Sigma}$. Based on the adjoint sensitivity analysis, the mathematical formulation for this derivative is (from Equation 2.34):

$$\frac{\partial k_{\text{eff}}}{\partial \Sigma} = \frac{\phi^{\dagger T} \left(\frac{\partial \mathbf{A}}{\partial \Sigma} - \frac{1}{k_{\text{eff}}} \frac{\partial \mathbf{F}}{\partial \Sigma} \right) \phi}{\frac{1}{k_{\text{eff}}^2} \phi^{\dagger T} \mathbf{F} \phi} = \frac{\phi^{\dagger T} \left(\frac{\partial(\mathbf{A}\phi)}{\partial \Sigma} - \frac{1}{k_{\text{eff}}} \frac{\partial(\mathbf{F}\phi)}{\partial \Sigma} \right)}{\frac{1}{k_{\text{eff}}^2} \phi^{\dagger T} \mathbf{F} \phi} \quad (3.13)$$

The biorthonormalization condition previously introduced simplifies the computation to the derivative of simple matrix-vector products:

$$\frac{\partial k_{\text{eff}}}{\partial \Sigma} = k_{\text{eff}}^2 \phi^{\dagger T} \left(\frac{\partial(\mathbf{A}\phi)}{\partial \Sigma} - \frac{1}{k_{\text{eff}}} \frac{\partial(\mathbf{F}\phi)}{\partial \Sigma} \right) \quad (3.14)$$

Evaluating this equation would normally require building full $N \times N$ Jacobian matrices for $\mathbf{A}$ and $\mathbf{F}$ (where N is the total number of flux values in the spatial mesh), which is slow and requires large memory. To avoid this, the code computes the products $\mathbf{A}(\Sigma)\phi$ and $\mathbf{F}(\Sigma)\phi$ directly without ever assembling the full matrices. It achieves this by replacing standard Python loops with JAX's `lax.scan` function, which iteratively applies the local diffusion operators cell-by-cell across the spatial grid.

During this backward calculation, the forward flux ϕ is treated as a constant. This is justified by the variational property from Eq. (2.27), which states that the sensitivity of the eigenvalue k_{eff} to a perturbation in ϕ is exactly zero at the converged solution. By evaluating the gradient of these implicit products, the backward pass efficiently extracts a vector of length M (the total number of cross-section parameters) that contains the exact parametric sensitivities. The vector stores one entry per cross-section parameter, corresponding directly to the final scalar values of the physical sensitivity terms: $\phi^{\dagger T} \frac{\partial \mathbf{A}}{\partial \Sigma_j} \phi$ and $\phi^{\dagger T} \frac{\partial \mathbf{F}}{\partial \Sigma_j} \phi$. By computing these final scalar outputs for all j simultaneously, the method completely bypasses the creation of explicit full Jacobian matrices.

3.3.3 Gradient verification of the custom vJp

Having established the structure of the custom backward rule, its mathematical accuracy must be validated before it can be reliably deployed for the neural network training. To conduct this verification, the 1D cylindrical reactor case study detailed in Chapter 4, is used as a representative test environment. Because the backtracking is implemented analytically, the resulting gradients must be checked to ensure they are correct. A standard method for this validation is to compare the analytical VJP outputs against a numerical finite difference (FD) approximation [34]. For a small perturbation ε applied to a single cross-section component Σ_j , the central finite difference approximation of the gradient is given by

$$\left. \frac{\partial r}{\partial \Sigma_j} \right|_{\text{FD}} \approx \frac{r(\Sigma_j + \varepsilon) - r(\Sigma_j - \varepsilon)}{2\varepsilon}, \quad (3.15)$$

where $r(\Sigma)$ is the inner-product residual associated with the eigenvalue problem, function of the cross-section tensor over N_r material regions and M cross-section types per group $\Sigma \in \mathbb{R}^{N_r \times M}$:

$$r(\Sigma) = \langle \phi^\dagger, A(\Sigma)\phi - \frac{1}{k} F(\Sigma)\phi \rangle, \quad (3.16)$$

evaluated at the same converged flux used by the custom backward pass. Since the components of Σ span several orders of magnitude, the perturbation is scaled to each component individually, $\varepsilon_j = \max(10^{-2} |\Sigma_j|, 10^{-5})$, so that it remains proportional to the magnitude of the cross section while never dropping below a numerical floor set by the precision of the solver.

The check is carried out on representative training samples, comparing both the per-sample and the batched (training) implementation of the solver, to ensure that the padding and vectorization used during training do not alter the computed sensitivities. The results are presented for one geometry in Table 3.1, where the cross sections that are structurally fixed to zero by the physical model, such as $\nu\Sigma_f$ and χ in the control rod and moderator regions, are excluded.

Table 3.1: Gradient verification comparing the analytical VJP against the central finite differences (FD) estimate across all material regions of the selected geometry, for all cross-section components with a non-zero value. Subscripts 1 and 2 denote respectively the fast and thermal energy groups.

Region	XS	Value	∇_{VJP}	∇_{FD}	Rel. Err.	Status
Absorber material	D_1	9.942e-01	4.605e-02	4.603e-02	3.62e-04	pass
	D_2	1.727e-01	-2.032e-02	-2.032e-02	6.51e-06	pass
	$\Sigma_{a,1}$	1.549e-02	-6.612e+00	-6.613e+00	1.41e-04	pass
	$\Sigma_{a,2}$	2.869e+00	-1.815e-04	-1.815e-04	6.41e-07	pass
	S_{11}	3.317e-01	0.000e+00	-8.208e-07	1.00e+00	noise
	S_{12}	1.189e-04	-6.450e+00	-6.432e+00	2.74e-03	pass
	S_{21}	7.861e-03	1.771e-03	2.092e-03	1.54e-01	pass
	S_{22}	1.005e+00	0.000e+00	0.000e+00	0.00e+00	noise
Fuel zone	D_1	1.095e+00	-3.670e-02	-3.668e-02	4.74e-04	pass
	D_2	3.842e-01	2.242e-02	2.242e-02	8.14e-06	pass
	$\Sigma_{a,1}$	2.309e-02	-3.109e+01	-3.109e+01	1.04e-06	pass
	$\Sigma_{a,2}$	4.703e-01	-1.083e+00	-1.083e+00	2.42e-06	pass
	$\nu\Sigma_{f,1}$	2.624e-02	2.591e+01	2.591e+01	2.06e-06	pass
	$\nu\Sigma_{f,2}$	9.232e-01	5.629e-01	5.629e-01	1.16e-06	pass
	S_{11}	2.726e-01	0.000e+00	-3.404e-05	1.00e+00	noise
	S_{12}	9.009e-03	1.883e+01	1.884e+01	4.18e-04	pass
	S_{21}	1.172e-03	-4.074e-01	-4.054e-01	5.01e-03	pass
	S_{22}	3.981e-01	0.000e+00	-1.060e-06	1.00e+00	noise
Moderator	χ_1	1.000e+00	1.200e+00	1.200e+00	1.29e-06	pass
	D_1	8.249e-01	-5.343e-02	-5.343e-02	8.44e-06	pass
	D_2	2.569e-01	5.436e-02	5.437e-02	6.20e-05	pass
	$\Sigma_{a,1}$	5.301e-04	-1.107e+00	-1.108e+00	1.27e-03	pass
	$\Sigma_{a,2}$	1.908e-02	-1.149e+00	-1.149e+00	1.67e-05	pass
	S_{11}	3.485e-01	0.000e+00	4.868e-07	1.00e+00	noise
	S_{12}	5.491e-02	6.498e-02	6.495e-02	3.89e-04	pass
	S_{21}	7.224e-05	1.260e-01	1.247e-01	1.04e-02	pass
	S_{22}	1.279e+00	0.000e+00	-4.029e-07	1.00e+00	noise

The self-scattering cross sections S_{gg} , indicated as “noise”, do not appear in the removal or in-scattering terms of $A\phi$, since self-scattering cancels in the diffusion formulation. Both the custom VJP and the

reference gradient are therefore exactly zero for these entries, and the small nonzero finite difference residuals (on the order of 10^{-7} to 10^{-5}) are attributed to single precision numerical noise rather than a genuine discrepancy. Among the valid components, the custom VJP agrees closely with the finite difference estimate, with discrepancies remaining small enough to be attributed to the limited numerical precision of the finite difference approximation itself rather than to an error in the custom gradient.

As an additional cross-check, the same components were also compared against a gradient obtained by applying automatic differentiation directly to Equation 3.16, which is not affected by the numerical noise inherent to finite differences. This comparison confirms the positive verification, with 21 out of the 36 total cross-section components being active and passing the check, with a mean relative error of about 1.00×10^{-6} .

3.3.3.1 Verification of the inclusion criterion

The inclusion criterion introduced in Section 2.1 requires that the low-fidelity solver can, in principle, reproduce the relevant high-fidelity responses when supplied with suitable admissible inputs. This condition was verified for selected configurations individually by directly updating the cross sections input to the diffusion solver using the adjoint-based gradient directly, without the use of the neural network. Near exact agreement with the corresponding OpenMC eigenvalue was obtained in all tested cases, demonstrating that the diffusion solver can reproduce the high fidelity response with optimal cross-section corrections.

Figure 3.2 compares the predictions obtained across the sampled configurations before and after the cross-section corrections learned by PEDS. The agreement with the OpenMC distribution is improved after correction, although exact agreement is not obtained for every sample. This remaining discrepancy reflects the challenge of learning suitable corrections simultaneously across the full batch, rather than a limitation of the diffusion solver itself. The result therefore supports the suitability of the diffusion solver as the embedded low-fidelity model in the PEDS framework.

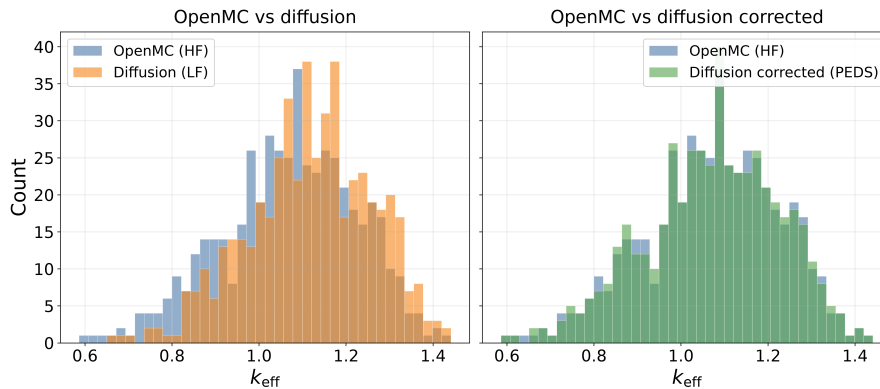


Figure 3.2: Comparison of the effective multiplication factor, k_{eff} , predicted by the OpenMC high-fidelity solver and the low-fidelity diffusion solver over the sampled configurations. The left panel shows the results using the diffusion solver only, whereas the right panel shows the results after the PEDS applied its cross-section correction.

3.3.4 Network architecture

Having established the differentiable physics solver, the remaining component of the PEDS framework is the surrogate network responsible for predicting the cross-section inputs to that solver. The objective of this architecture is to map the raw geometric and material parameters of a specific reactor design configuration to a set of refined cross-sections, reducing the discrepancy between low-fidelity and high-fidelity solutions.

As introduced previously in Section 2.2 and illustrated in Figure 2.2, the prediction of cross-sections proceeds in two steps. A fast polynomial regression provides an initial, geometry-dependent estimate of the baseline cross-sections. The generator network then refines this baseline by predicting localized corrections that better match the high-fidelity target. By separating broad geometric dependencies from fine-tuned physical corrections, the network is not forced to learn the cross-sections from scratch, resulting in more stable and effective training.

The following subsections detail complete architecture: the generation of baseline cross-sections through polynomial regression (Subsubsection 3.3.4.1), the internal structure of the generator network (Subsubsection 3.3.4.3), and the model integration and training methodology (Subsubsection 3.3.4.3).

3.3.4.1 Baseline cross-sections

The polynomial regression provides the initial estimate of the cross-sections used as input to the diffusion solver. It plays the same role of the downsampled geometry in previous PEDS applications: a coarse but physically grounded starting point that the network subsequently improve to match the target. Each sample is characterized by a distinct design vector $\mathbf{p}$ containing the geometric and material parameters, which the regression model map to a sample-specific multigroup cross-section estimate.

Given this role, a standard least squares fit is entirely sufficient. The regression does not need to be perfectly accurate; it simply needs to provide a rapid, reasonable initial guess. This approach keeps computational overhead practically negligible. Because fitting a second-order polynomial requires a direct linear solve rather than an iterative neural network optimization, this baseline setup is a one-time operation that takes just seconds to complete. Once trained, the saved regression model is embedded within the PEDS pipeline to instantaneously generate a baseline cross-section tensor for every new configuration, completely independent of further Monte Carlo simulations.

The MGXS used as regression targets are obtained from the same high-fidelity simulations already generated for the neural network training data, so no additional Monte Carlo runs are required. Because accuracy is not the priority, fewer training samples are needed than for the neural network. This polynomial regression mapping was empirically verified to accurately capture the macroscopic cross-section behaviour across the relevant physical regimes of the design space. While the fit quality naturally varies depending on the specific material region and energy group, this does not compromise the framework’s objective. The regression’s primary role is simply to anchor the surrogate in the correct physical regime, leaving the neural network responsible for the precise, solver-informed corrections. Detailed performance metrics for this baseline regression are provided in Appendix A.

The regression predicts parameters across n_{regions} material regions, with n_{XS} cross-section values per region, yielding a fully assembled baseline cross-section tensor of shape $n_{\text{regions}} \times n_{\text{XS}}$. For the fission spectrum χ_g , an additional constraint is added since this quantity must sum to one across all energy groups, by definition as a probability distribution. Letting a regression fit it directly would risk assigning it arbitrary intermediate values with no physical meaning.

3.3.4.2 Generator network

The generator network (implemented as `GeneratorNN` in the codebase) is trained to predict the targeted corrections for the initial baseline cross-sections. Its primary objective is to map the sample’s design parametrization vector $\mathbf{p}$ to a set of refined adjustments, minimizing the discrepancy between the low-fidelity solver’s output and the high-fidelity target.

To facilitate a more informed and stable training process, the network is supplied with two additional physics-based inputs, both of which are readily available at inference time. The first is the baseline cross-sections themselves, which are converted to log-space and standardized:

$$z_j = \frac{\log(\Sigma_j^{\text{baseline}}) - \mu_j}{\sigma_j}, \quad (3.17)$$

where μ_j and σ_j are the mean and standard deviation of the log-baseline values for cross-section entry j , computed over the entire training set. These standardized entries are then assembled into the normalized log-baseline vector $\mathbf{z}$. This log-space conversion is necessary because nuclear cross-sections differ by many orders of magnitude. If these raw values were fed directly into the neural network, the large differences in scale would lead to gradients instability. Using log-space compresses the data to a uniform scale. Furthermore, it allows the network’s output to act as a multiplicative correction rather than an additive one, inherently preventing the prediction of non-physical negative cross-sections.

The second additional input is a set of average flux values over the material regions n_{regions} , obtained from a single diffusion solve using the baseline cross-sections. For every energy group g and every material region r , a volume-weighted average flux is computed as:

$$\bar{\phi}_{g,r} = \frac{1}{\sum_{i \in r} V_i} \sum_{i \in r} V_i \phi_{g,i}, \quad (3.18)$$

where V_i is the volume of cell i and the sum runs over all cells belonging to region r . These averages are gathered into a vector $\bar{\phi}$ of length $G \times n_{\text{regions}}$ and normalized before being passed to the network.

This input acts as a crucial physical regularizer. In early development, it was observed that applying unconstrained cross-section corrections could cause the resulting spatial flux to drift noticeably from the physical baseline, even if the final eigenvalue k_{eff} matched the target perfectly. Supplying the baseline flux as an input encourages the network to seek corrections that preserve a physically consistent flux shape. This improves training stability and output quality; an example of the resulting flux profiles is presented later in Chapter 4.

The network architecture is structurally divided into a trunk and a head. The trunk takes the normalized geometry parameters $\tilde{\mathbf{p}}$ and the normalized average flux vector $\bar{\phi}$ as inputs. Since the raw geometry and material parameters p_k that define a sample can differ from one another by orders of magnitude, each is rescaled to a common range before being passed to the network. This is a standard practice in machine learning known as *feature scaling*, and the specific rescaling used here is min-max normalization, mapping each parameter to the interval $[0, 1]$ [35]:

$$\tilde{p}_k = \frac{p_k - p_k^{\min}}{p_k^{\max} - p_k^{\min}}, \quad (3.19)$$

where $p_k^{\min}$ and $p_k^{\max}$ denote the lower and upper bounds of the design range for that parameter. This keeps every input to the generator network within a comparable numerical scale, improving the conditioning of training and preventing parameters with larger raw magnitudes from dominating the gradient updates. It processes them through a stack of fully connected layers with ReLU activations to produce a dense feature vector $\mathbf{h}$. This feature vector is then concatenated with the normalized log-baseline vector $\mathbf{z}$ and passed through the head, a single fully connected output layer. This final layer produces an unbounded log-ratio correction value, denoted as ℓ_j^{raw} , for every cross-section entry j :

$$\ell_j^{\text{raw}} = \text{head}_j([\mathbf{h}, \mathbf{z}]). \quad (3.20)$$

To ensure training stability, the weights of this final head layer are initialized to zero, meaning at the very start of training the network outputs exactly $\ell_j^{\text{raw}} = 0$ for all parameters. Because $e^0 = 1$, the network initially acts as a perfect identity function, leaving the baseline cross-sections entirely unchanged. As training progresses, the optimization algorithm gradually shifts the output away from zero, allowing the network to learn the necessary adjustments without erratic initial guesses that could destabilize the solver. Before being applied, each correction value is constrained to a fixed empirical range:

$$\ell_j = \text{clip}(\ell_j^{\text{raw}}, -2.0, 0.5). \quad (3.21)$$

These bounds strictly limit the maximum total deviation the network can apply to the baseline. Converted from log-space, this asymmetric range allows the cross-sections to decrease to roughly $e^{-2.0} \approx 0.135$ times the baseline (an 86.5% reduction) or increase to a maximum of $e^{0.5} \approx 1.649$ times the baseline (a 64.9% increase). This remains a comfortably wide margin, as the expected final correction typically falls well below 50% in either direction. The specific clipping range is an arbitrary choice rather than a strict physical limit; it is implemented simply to prevent gradient explosions and uncontrolled weight updates early in training from pushing a cross-section to an unrecoverable, non-physical value.

A binary mask $m_j \in \{0, 1\}$ is further applied to enforce regions where a cross-section must remain exactly zero to retain a physical meaning, as is the case for fission cross-sections in every region outside the fissile material. The fully corrected cross-section tensor $\Sigma^{(\text{final})}$ is then assembled by combining the baseline, the exponentially scaled correction, and the physical mask:

$$\Sigma_j^{(\text{final})} = \Sigma_j^{(\text{baseline})} e^{\ell_j} m_j. \quad (3.22)$$

Similar to the physical constraints placed on the baseline generation (Subsubsection 3.3.4.1), the corrected fission spectrum χ_g undergoes a dedicated normalization step at the very end of the forward pass. After the neural network applies its continuous corrections, the updated χ_g values are systematically rescaled so that they strictly sum to one across all energy groups in fissile materials, and are forced to exactly zero in non-fissile regions. This translates to discrete values of 1 for the fast group and 0 for the thermal group in a standard two-group setup, but the implemented code is fully generalized to accommodate any number of energy divisions.

The network uses hidden layers of size [128, 256, 128]; this configuration was chosen after comparing several depths and widths, and gave better performance than smaller alternatives tested.

3.3.4.3 Model integration and training

The generator network and the low-fidelity solver are combined into a single end-to-end differentiable model, referred to as `PEDSModel` in the code. Both components operate on the same physical sample, but each requires the design vector $\mathbf{p}$ in a different form: the solver uses the raw geometric and material parameters directly to build the mesh and region layout on which the diffusion equations are assembled, while the generator network receives the normalized parameters $\tilde{\mathbf{p}}$ described in Subsubsection 3.3.4.2. Given a batch of raw and normalized sample parameters, the baseline cross-sections are generated by polynomial regression, and corrected by the generator network. These are then passed to the solver described in Subsection 3.3.1, which solves the resulting matrices to obtain the predicted eigenvalue for each sample.

The network is trained by minimizing the mean squared error between the predicted and the reference k_{eff} , computed over each training batch. Other loss formulations, including the mean absolute error and the Huber loss, were also tested, but the mean squared error produced the most effective training behaviour and was adopted as the final choice.

Before training starts, the dataset of normalized geometry parameters and reference eigenvalues is split into training, validation, and test sets. Samples are first sorted by k_{eff} and divided into ten quantile bins of equal size, from which a fixed number of samples is then drawn. For every sample, the baseline cross-sections and the average flux features are precomputed once, and the flux features are normalized using the mean and standard deviation measured on the training set, so that the same scale is applied consistently across samples during training.

Training proceeds over a fixed number of epochs, although early stopping can be activated to avoid losing computational time when no further improvement is observed. At the start of every epoch, the training set is shuffled and split into fixed-size batches. Since every sample in a batch requires an independent call to the external eigenvalue solver, these calls are distributed across multiple worker processes running in parallel on a HPC, removing the eigenvalue solve as a bottleneck during training, as it is the most computationally expensive step of the pipeline. For each batch, the model computes the predicted eigenvalues using the solver over the cross-sections corrected by the network, the loss is evaluated, and the gradients of the loss with respect to the network parameters are obtained through the custom VJP rule described in Subsection 3.3.2. These gradients are clipped by their global norm to a fixed value

of 0.5: if the gradient exceeds this threshold, it is uniformly rescaled so that its total magnitude is brought back to 0.5. This does not change the direction of the update but prevents unusually large gradients, particularly early in training or after difficult batches, from producing unstable parameter jumps, especially helpful early in training or after difficult batches. The clipped gradients are then used to update the network parameters with the Adam optimizer, following a cosine-decay learning rate schedule with linear warmup, where the maximum and minimum learning rates are respectively 5×10^{-4} and 5×10^{-6} .

After each epoch, the same procedure is repeated on the validation set without parameter updates, tracking the reactivity error in pcm over training. An exponential moving average of this validation error is used to select the best-performing checkpoint, saved once the model has passed the initial epochs of oscillation, and training is stopped once this smoothed error shows no further improvement over a fixed number of epochs. A scheme of the training process summarizing all the details described in this chapter is presented in Figure 3.3.

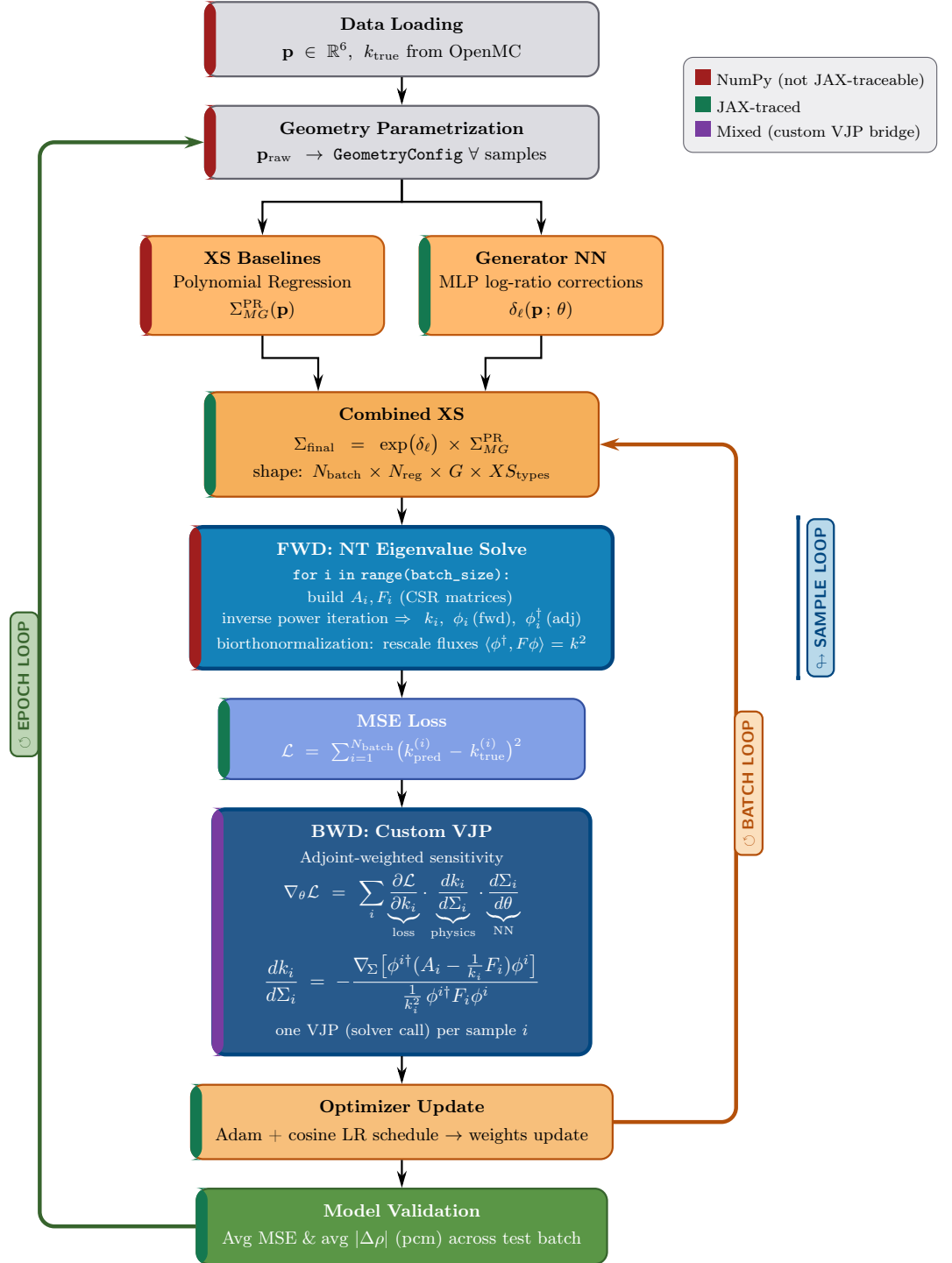


Figure 3.3: Flowchart of the PEDS training pipeline, showing the flow from the raw geometry parameters to the eigenvalue prediction, the loss evaluation, and the parameter update through the custom VJP rule.

Chapter 4

Results and discussion

The PEDS setup described in Chapter 2 employs the 1D neutron diffusion solver presented in Subsection 3.3.1 as its low-fidelity component. While the surrogate framework is intended to be generalized to complex multi-dimensional geometries in the future, it is important in these early stages of research and development to have a simple proof-of-concept system that can be fully controlled and verifiable. Since this is the first time building such a network for neutron transport applications, there is no guideline or known case study to use as a testing example. It was therefore decided to formulate a simple but non-trivial problem, consisting of a reactor core reduced to a one-dimensional cylinder. This chapter presents the evaluation of the PEDS model on this representative case study. First, the specific geometric parameters are defined in Section 4.1, followed by a description of the high-fidelity training data generation through OpenMC in Section 4.2. The preliminary performance and accuracy of the initial version of the surrogate model are then evaluated in Section 4.3. Sensitivity studies are subsequently presented in Section 4.4 to examine the sources and physical characteristics of the remaining prediction errors. The influence of the training data distribution and geometric parameters is assessed to identify the operational limits of the framework. Finally, Section 4.5 demonstrates the practical use of PEDS in a forward reactor design search.

4.1 Geometry description and implementation

The case study mimics a simplified reactor core. It is a 1D cylindrical geometry with three concentric regions ordered from the center outward: a B_4C control rod, a fuel annulus, and a water moderator/reflector. To approximate an infinite 1D cylinder in the high-fidelity 3D solver, reflective boundary conditions are applied in the axial direction, while a vacuum boundary condition is imposed at the outer radial surface. Correspondingly, the 1D diffusion solver enforces a reflective zero-current condition at the center to satisfy cylindrical symmetry ($d\phi_g/dr|_{r=0} = 0$), and a simplified zero-flux Dirichlet condition at the outer perimeter ($\phi_g(R) = 0$).

This specific configuration was chosen to intentionally challenge the diffusion theory approximation of the low-fidelity solver. Fick’s law assumes a slowly-varying flux and weak neutron absorption; these assumptions inherently break down near strong absorbers like the control rod, and at the interface between the multiplying fuel material and the strongly-moderating water region. The severity of these

diffusion errors depends on the specific sample of the parameter vector (reactor design), which dictates the magnitude of the local flux gradients. This dynamic discrepancy between the low-fidelity diffusion solution and the high-fidelity Monte Carlo solution provides an ideal testbed to investigate PEDS.

Two energy groups are employed: a thermal group with energies below 0.625 eV, and a fast group with energies above it, to properly capture the energy dependence of the neutron population in the system. Both OpenMC and the diffusion solver implement the geometry with the same three regions and material parameters. In OpenMC, using the methodology described in Section 3.2, the regions are specified as concentric Z-cylinders with an axial half-height of 50 cm and reflective boundaries on the upper and lower axial faces. The central control rod region is represented by a homogenised mixture of boron carbide and iron. Its absorber fraction controls the relative amount of B_4C and therefore the absorption strength of the region. The fuel annulus is represented by a homogenised mixture of uranium and water, with the uranium enrichment and moderation fraction treated as variable parameters. The outer reflector region consist of pure light water at nominal density. Figure 4.1 shows a schematic of the geometry for a representative case.

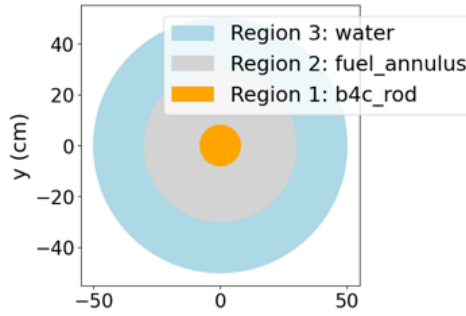


Figure 4.1: Schematic cross section of the three region cylindrical geometry: B_4C control rod, fuel annulus, and water moderator, centre to outside.

The diffusion solver uses the same radial region ordering and material parameters as the OpenMC model. The region boundaries are defined by their outer radii, and the geometry is discretised into radial mesh cells. Multigroup cross sections are assigned to each cell according to its material region. Within the PEDS framework, the baseline multigroup cross sections are modified by the neural network correction and subsequently provided as input to the diffusion solver. The system matrices are then assembled on the spatial mesh and solved following the method described in Section 3.1.

A uniform radial mesh width of 1cm was selected to enable rapid low fidelity evaluations, with an average solver time of approximately 4ms. Since the total radial extent of the geometry remains below 100cm, this resolution results in a limited number of mesh cells to process while retaining the main variations in the flux profile. The computational cost was found to increase rapidly with mesh refinement; a preliminary test showed that a tenfold increase in the number of mesh cells leads to an approximately twentyfold increase in solution time. Keeping this duration minimal is essential for the training process, where the solver is evaluated for every data sample at each epoch. By adopting this coarser mesh, the low fidelity model achieves the execution speed required for efficient training and to carry out extensive design studies, while the neural network correction compensates for the physical approximations and reduced numerical precision.

OpenMC was also set to use the same spatial mesh size of 1 cm per cell for the flux mesh tally, ensuring direct comparability between flux profiles. For a representative sample of the geometry, Figure 4.2 shows the flux profile from the OpenMC reference solution and Figure 4.3 provides the flux profile of the same reactor configuration from the diffusion solver. The interfaces between regions show high flux variations and the greatest disagreement between the two solvers.

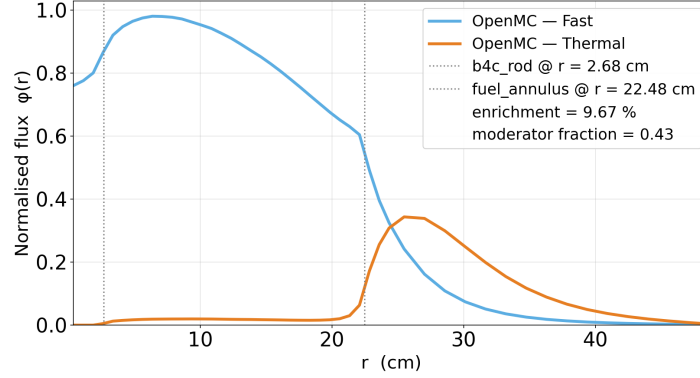


Figure 4.2: OpenMC computed normalised two-group flux profile, with dotted lines marking the control rod and fuel annulus outer boundaries.

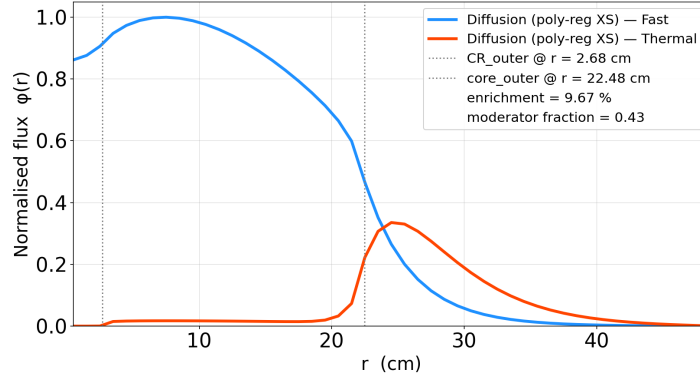


Figure 4.3: Diffusion solver computed normalised two-group flux profile for the same reactor configuration as Figure 4.2, with dotted lines marking the control rod and fuel annulus outer boundaries.

4.2 Training data generation

The training dataset for the surrogate model is produced by repeatedly calling the high-fidelity OpenMC solver described in Section 3.2 at different points of the design space. Because each Monte Carlo run is expensive, computing time is spent efficiently by spreading the sampled configurations as evenly as possible across the parameter space, rather than sampling them purely at random. This is achieved using Latin Hypercube Sampling (LHS), a method that divides each parameter range into equally sized intervals and guarantees that every interval is represented exactly once across the full set of samples. Compared with uniform random sampling, LHS reduces the risk of leaving large gaps or forming clusters in the design space.

For each sample, a set of parameter values is drawn from the LHS grid and rescaled to the physical bounds. Six design parameters together define one reactor configuration and are varied during training-data generation: the control rod outer radius, the control rod absorber fraction, the fuel annulus outer radius, the fuel enrichment, the fuel moderation fraction, and the water region outer radius. Their sampling bounds are listed in Table 4.1.

Table 4.1: Ranges considered for the six geometric and material design parameters.

Parameter	Lower bound	Upper bound
Control rod outer radius (cm)	1.0	6.0
Control rod absorber fraction	0.0	1.0
Fuel annulus outer radius (cm)	10.0	40.0
Uranium enrichment (% U-235, atomic)	1.5	10.0
Fuel moderation fraction	0.40	0.80
Water region outer radius (cm)	40.0	80.0

The control rod outer radius sets the physical size of the absorbing rod at the centre of the geometry, while the control rod absorber fraction controls its homogenised material composition and, consequently, its absorption strength. A value of 1 corresponds to pure boron carbide (B_4C), whereas a value of 0 corresponds to pure iron (Fe). The fuel annulus outer radius sets the outer edge of the fuel-bearing region, and the uranium enrichment specifies the atomic percentage of U-235 within the uranium component, with the remaining uranium assumed to be U-238. The fuel moderation fraction controls the relative amounts of water and uranium in the homogenised fuel region. Low values correspond to a uranium-rich mixture, whereas high values correspond to a more strongly moderated mixture. The water region outer radius sets the extent of the surrounding moderator and reflector region.

These bounds were chosen to keep the geometry within the range of a simplified, single-assembly-like reactor model, while still producing a wide enough spread of eigenvalues to be useful for design studies. The control rod and fuel annulus radii are kept small enough that the three regions remain physically distinct under any combination of sampled values, and the water region radius is kept large enough to provide adequate reflection. The enrichment and moderation fraction bounds were selected to cover fuel compositions representative of light water reactors, avoiding extreme values that would push the configuration far outside the range the surrogate is intended to be useful for.

Every simulated configuration uses the same fixed Monte Carlo settings: 20,000 particles per batch, 50 inactive batches followed by 150 active batches (200 batches total). With these settings, a single simulation takes approximately 65 seconds to complete, and the standard deviation of the resulting k_{eff} values is, on average, below 50 pcm.

Figure 4.4 shows the full distribution of k_{eff} obtained when sampling uniformly across the bounds of Table 4.1. The sampled configurations produce eigenvalues spanning a wide range, including many combinations of parameters that correspond to strongly subcritical or strongly supercritical systems and are not representative of practical reactor operating conditions. For this reason, the training set used to fit the surrogate model has been restricted to configurations with k_{eff} between 0.8 and 1.2, discarding samples outside this window, to match the domain of interest for the current application, constituting a

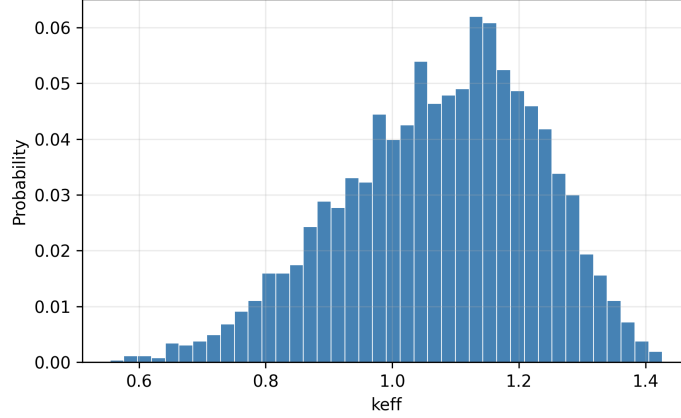


Figure 4.4: Distribution of the effective multiplication factor (k_{eff}) for the complete generated dataset, resulting from uniform Latin Hypercube Sampling across the defined geometric and material parameter bounds.

good representation of possible scenarios in reactor core design studies.

This restriction does not reduce the quality of the training. Instead, it focuses the training process on configurations within the operating range of interest, rather than on extreme configurations that are outside the scope of the present design study. Because the surrogate is trained to be accurate where it will actually be used in a PEDS design framework, removing the far tails of the k_{eff} distribution improves, rather than degrades, its performance.

4.3 Preliminary results

A baseline for the PEDS framework was established by evaluating a reference study, using the dataset generation methodology detailed in Section 4.2. A total of 500 samples were generated and partitioned into an 80% training and 20% validation split. The neural network architecture described in Subsection 3.3.4 served as the starting point for this baseline configuration. Training was conducted for 70 epochs with a batch size of 25 samples, as preliminary tests indicated that both training and validation metrics stopped improving noticeably beyond this point.

The training performance was first evaluated using the Mean Absolute Error (MAE) and Mean Squared Error (MSE), standard measures of prediction error applied here to quantify the difference between the high-fidelity OpenMC k_{eff} target and the low-fidelity solver prediction. The evolution of these metrics over the training epochs is shown in Figure 4.5.

While statistical error metrics confirm mathematical convergence, they do not provide a direct physical interpretation of the results. In nuclear reactor applications, the actual physical state of a system is conventionally characterized by its reactivity (ρ), which represents its fractional departure from criticality [31]:

$$\rho = \frac{k_{\text{eff}} - 1}{k_{\text{eff}}} \quad (4.1)$$

To properly quantify the physical discrepancy between two separate calculations, a reactivity difference

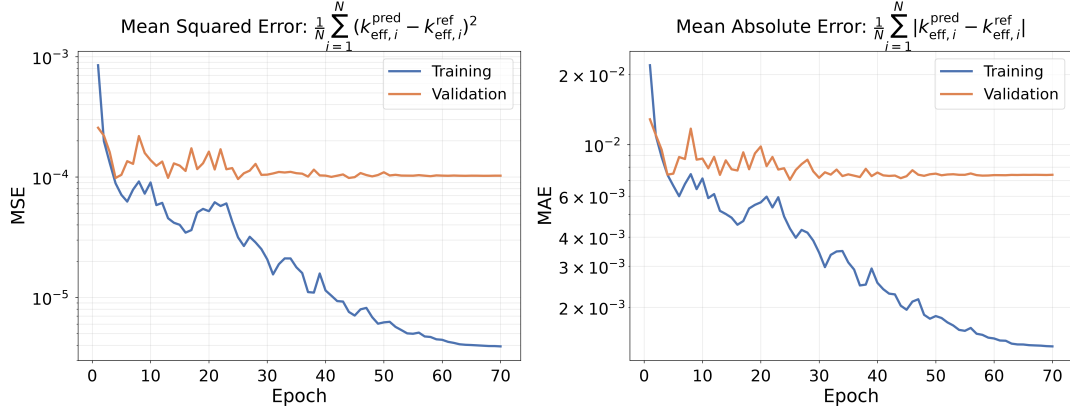


Figure 4.5: Evolution of MAE and MSE metrics over 70 training epochs for the baseline PEDS configuration.

($\Delta\rho$) is utilized. Hence, when comparing the target reference value (k_{ref}) with the model prediction (k_{pred}), the reactivity difference is derived by subtracting the two states:

$$\Delta\rho = \rho_{ref} - \rho_{pred} = \left(\frac{k_{ref} - 1}{k_{ref}} \right) - \left(\frac{k_{pred} - 1}{k_{pred}} \right) = \frac{k_{ref} - k_{pred}}{k_{ref} \cdot k_{pred}} \quad (4.2)$$

To contextualize the computational cost of the PEDS framework, execution times for the primary operations were tracked during the baseline training run. The relevant timing metrics, accumulated over the 70 epochs, are summarized in Table 4.2.

The data highlights the efficiency of the framework, with the low-fidelity eigenvalue solve requiring an average of only 9.3 ms and the custom analytical backward pass (VJP) derived in Subsection 2.2.3 evaluates gradients in just 2.1 ms. Ultimately, a complete training step for a single batch of 25 samples (comprising the neural network forward pass, the parallel physics solver execution, the backward pass, and the optimizer weight update) is executed in approximately 360 ms. Consequently, a full training epoch (400 samples) is completed in 5.76 seconds, scaling against the 1.0 second required to validate the 100 holdout samples, where no gradient calculation or optimizer update is performed.

Table 4.2: Summary of mean execution times for key PEDS operations during the baseline training run.

Operation	Calls	Mean Time
Low-fidelity eigenvalue solve (fwd)	2115	9.3 ms
Custom VJP backward pass (bwd)	1120	2.1 ms
Full training step (batch of 25)	1120	360.0 ms
Full training step (epoch)	70	5.76 s
Validation step (epoch)	70	1.01 s

4.3.1 Training-validation gap and correction attempts

Because nuclear core design requires extreme precision, discrepancies that appear negligible in standard machine learning metrics can actually be substantial. Differences in reactivity of a reactor are conventionally measured in units of percent mille (pcm), where one pcm is equivalent to 10^{-5} . To define a

physically meaningful error threshold for the neural network, the effective delayed neutron fraction, β_{eff} , is used as a reference. This quantity corresponds to the reactivity insertion beyond which a reactor would become prompt critical, triggering very rapid power transients. Although β_{eff} varies slightly with the reactor configuration, a value of 650 pcm was adopted in this work as a representative value for a system fuelled primarily by U-235. Hence, the objective of PEDS is to achieve low-fidelity predictions whose reactivity difference from the high-fidelity target remains below this threshold.

A comprehensive evaluation of the model’s statistical performance is presented in Table 4.3 and Table 4.4 for the training and validation sets, respectively. Alongside the standard MSE and MAE loss functions, secondary metrics including the mean, median, and 95th percentile (P95) of the reactivity difference are reported to give broader statistical context, together with the fraction of samples falling below the 650 pcm accuracy threshold and a stricter 100 pcm limit. Epoch 1, representing the state immediately after the first weight update, distinct from the uncorrected Epoch 0 baseline, is compared against the converged state at epoch 70 to illustrate the overall learning dynamics.

Table 4.3: Training metrics at selected epochs. Epoch 1 represents the state after the first network update, distinct from the uncorrected baseline.

Epoch	MSE	MAE	Mean (pcm)	Median (pcm)	P95 (pcm)	Frac $< \beta_{\text{eff}}$	Frac < 100
1	1.8×10^{-3}	3.5×10^{-2}	3374	2809	8113	0.12	0.01
70	6.9×10^{-6}	1.9×10^{-3}	186	122	598	0.97	0.43

Table 4.4: Validation metrics at selected epochs, highlighting the lower fraction of samples meeting the 650 pcm threshold compared to the training set.

Epoch	MSE	MAE	Mean (pcm)	Median (pcm)	P95 (pcm)	Frac $< \beta_{\text{eff}}$	Frac < 100
1	8.3×10^{-4}	2.4×10^{-2}	2229	2076	5268	0.14	0.02
70	6.4×10^{-5}	6.1×10^{-3}	640	416	1937	0.65	0.13

These statistics highlight a disparity between the training and validation performance. On the training data set, 97% of samples successfully meet the 650 pcm threshold, with the average error dropping to 186 pcm. On the validation set, the mean error settles at 640 pcm, with 65% of cases below β_{eff} . It is important to note that a degree of performance degradation on unseen data, commonly referred to as the generalization gap, is a fundamental and expected characteristic of neural network training [36, 37].

Despite this expected gap, the network achieves a substantial overall improvement. The average validation error drops significantly from the initial uncorrected baseline of approximately 5000 pcm down to 640 pcm. However, analyzing individual sample trajectories reveals that this residual error is not distributed uniformly: certain geometries are intrinsically harder for the network to evaluate. As illustrated in Figure 4.6, for a subset of validation geometries (shown here using indexes 20 to 40 as a representative example), some predictions stall at elevated reactivity differences. This early performance plateau suggests that for these specific physical domains, the model may be converging to a suboptimal local minimum without achieving further error reduction. These cases fail to converge below the target accuracy threshold, illustrating the practical limits of the generalization capabilities of the network.

Since the stalling behavior on specific physical geometries suggested a potential limitation in the model

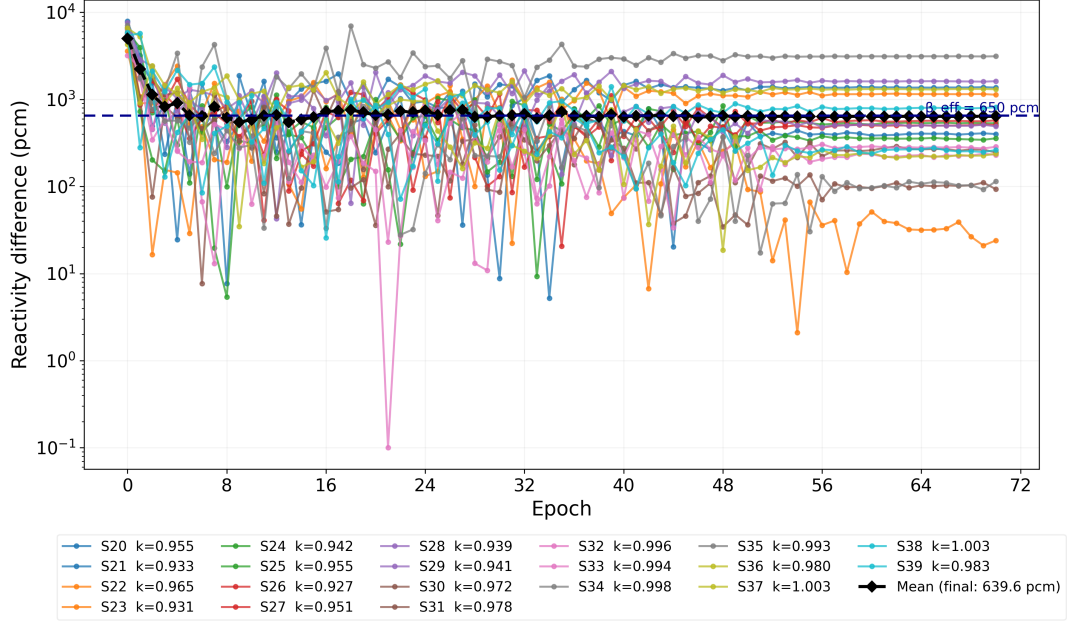


Figure 4.6: Evolution of the reactivity difference, expressed in pcm, over training epochs for selected validation samples. The dashed black line shows the mean error across samples, while the horizontal blue line marks the reference value $\beta_{\text{eff}} = 650$ pcm.

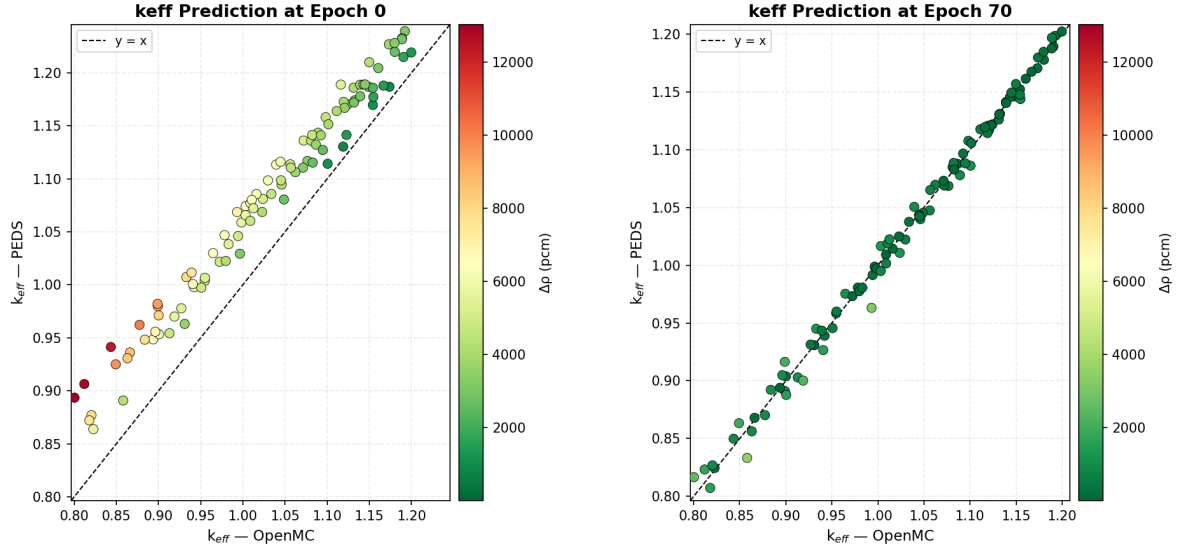
setup, a wide range of alternative network configurations was tested. The overall learning capacity and architecture of the network were modified by varying hidden layer sizes, restructuring the shared trunk and output heads, and adjusting the physical input features they receive. Furthermore, various standard techniques designed to improve generalization to unseen data and stabilize the learning process were applied. These included methods intended to prevent the model from memorizing the training data (dropout and L_2 penalties), strategies to maintain stable mathematical updates during training (gradient clipping), and optimization algorithms intended to help the network escape potential local minima (AdamW optimizer).

Despite these efforts, none of the standard mitigation techniques meaningfully reduced the validation gap or eliminated the persistent stalling. Consequently, the NN architecture was returned to the configuration described in Subsection 3.3.4. This behavior is therefore reported here as an open limitation of the current setup. The inability of standard machine learning regularization methods to resolve this issue might indicate that the observed validation gap is not fully explained by the forms of overfitting targeted by these methods. It may also be related to limited training coverage in some regions of the design space, to a more complex relationship between the design parameters and the required cross section correction, or to a discrepancy between the diffusion and high fidelity models that is more difficult to represent for some configurations. It is therefore left as a direction for future work.

4.3.2 Physical validation and overall framework efficacy

The initial state, prior to the application of PEDS (hereafter referred to as “epoch 0”), corresponds to the baseline diffusion solver evaluation using the initial cross-section estimates, derived from the fast

polynomial regression. Among this initial pool of cases, none of the samples fall below the β_{eff} threshold. This indicates that the uncorrected diffusion output cannot be considered reliable for applications such as design studies. This baseline discrepancy is primarily attributed to the inherent physical approximations of the diffusion solver on the geometrical cases studied, representing a perfect application for the neural network model to correct.



(a) Solver predictions before applying the NN.

(b) Solver predictions at end of training.

Figure 4.7: Comparison of the predicted k_{eff} against the OpenMC reference for the validation samples. The initial diffusion solver results are shown in (a), while the neural network corrected predictions after 70 epochs are presented in (b). The point colors represent the reactivity difference $\Delta\rho$ expressed in pcm.

Figure 4.7 shows the distribution of the validation samples before (epoch 0) and after the neural network correction is applied to the cross-sections used by the diffusion solver. It is clear from this figure that the PEDS setup successfully generates a set of useful cross-section adjustments that bring the model prediction significantly closer to the target value.

A verification of the cross-section outputs is necessary to ensure the model is learning valid physics rather than overfitting mathematically. Figure 4.8 illustrates the magnitude of the cross-section adjustments applied by the neural network groups for a representative validation sample across both fast (Group 1) and thermal (Group 2) energy groups. The relative adjustments remain physically plausible and tightly bounded, demonstrating that the surrogate does not exploit non-physical parameter spaces simply to minimize the loss function.

The spatial flux distribution is also examined to verify that the shape does not significantly vary when the baseline input is corrected by the NN. This is expected from the implementation of the physical regularization in the network architecture described in Subsubsection 3.3.4.2. Figure 4.9 compares these normalized flux profiles for a representative validation sample. As intended, the PEDS-corrected flux closely follows the baseline diffusion solver, exhibiting only minor deviations from the OpenMC target due to the inherent physical approximations of the low-fidelity solver. This confirms that the network

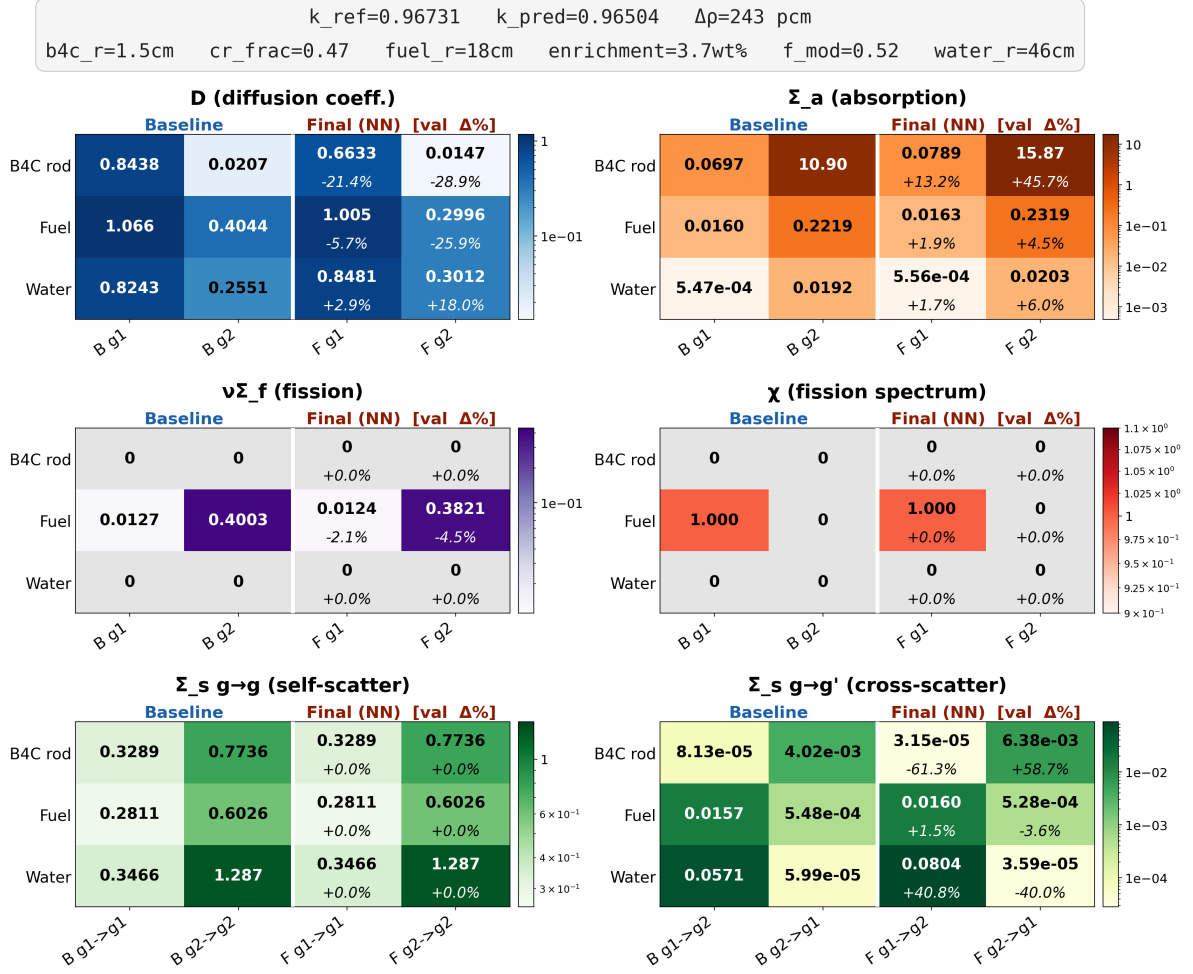


Figure 4.8: Nominal values and relative magnitude of neural network corrections applied to the baseline macroscopic cross-sections (from the polynomial regression) for a representative sample at epoch 70.

successfully preserves a consistent spatial distribution while minimizing the eigenvalue error.

A final mapping of the error distribution provides deeper insight into the framework's operational boundaries. Figure 4.10 presents the distribution of the validation samples in terms of their reactivity difference, explicitly mapping these errors against the magnitude of their target k_{eff} . A distinct correlation emerges between the final prediction accuracy and the criticality state of the system: supercritical configurations ($k_{eff} > 1$) converge to a much lower final reactivity difference compared to subcritical configurations ($k_{eff} < 1$). It is important to note that the supercritical geometries already exhibited smaller initial discrepancies before applying PEDS; this suggests that the diffusion solver model itself is inherently more valid for the configurations found in the supercritical regime, providing PEDS with a physically closer starting point that the network must then correct. Conversely, the network's comparative struggle to fully minimize errors in the subcritical regime highlights a specific physical domain where the model approximations are most challenging to correct.

This behavior is actually favourable from a reactor design perspective: reactor cores are normally designed

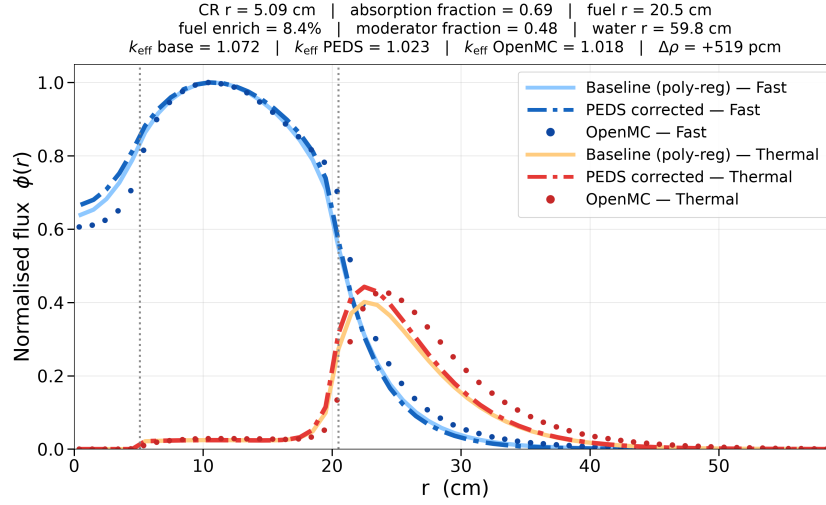


Figure 4.9: Normalized spatial flux distributions for the fast and thermal energy groups of a representative validation sample, whose parameters are annotated within the figure. The plot compares the high-fidelity OpenMC target, the uncorrected baseline diffusion solution, and the diffusion solution utilizing PEDS-corrected cross-sections.

including excess reactivity to account for fuel depletion over the operating cycle. Greater accuracy is therefore most valuable near and above criticality, while for deeply subcritical configurations a larger discrepancy in reactivity prediction is less significant for preliminary design studies.

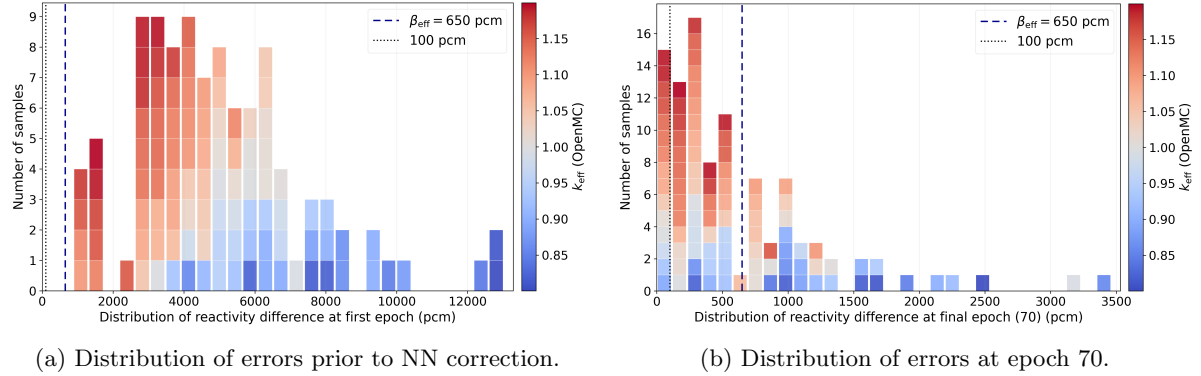


Figure 4.10: Distribution of validation samples evaluated by their reactivity difference ($\Delta\rho$) and mapped against the magnitude of the target k_{eff} . The plots compare the error distribution prior to the neural network correction (a) and at the final epoch 70 (b).

A natural question raised by this domain-dependent behavior is whether it stems from a deficiency in the baseline polynomial-regression cross-sections used to initialize the framework, rather than from the diffusion solver itself. A natural question raised by this domain dependent behavior is whether it stems from inaccuracies in the baseline cross sections provided by the polynomial regression, rather than from the low fidelity diffusion model itself. To examine this, the diffusion solver was evaluated for every geometry using the multigroup cross sections directly extracted from the corresponding high fidelity OpenMC calculation, previously stored to train the polynomial regression. The resulting k_{eff} values were compared with the regressed cross-sections, against the same OpenMC target.

Both cross-sections sources produce nearly identical reactivity error against the OpenMC target: a mean $|\Delta\rho|$ value of 5044 pcm with the exact MGXS versus 5033 pcm with the regression. To isolate the contribution of the regression itself, the k_{eff} predictions obtained with the two XS sources were compared directly against each other: the median absolute difference is only 170 pcm. This means the regression’s own inaccuracy accounts for at most about 3% of the total, or equivalently is $26\times$ smaller than the full discrepancy. It can consequently be concluded that the observed discrepancy is dominated by the low fidelity model rather than by the polynomial regression initial guess. This includes the physical limitations of the diffusion approximation and the use of a coarse spatial mesh, which are shared in the two diffusion calculations.

This proves that the underlying discrepancy is driven almost entirely by the diffusion approximation, and not by the quality of the cross-sections. Consequently, generating exact cross-sections for every geometry (requiring an OpenMC run) would completely defeat the computational purpose of a fast surrogate without actually fixing the physics. The polynomial model serves its intended purpose perfectly: it provides an instant, quantitatively reliable starting point, leaving the role of bridging the physical gap between diffusion and transport theory entirely to the neural network.

Although the baseline PEDS configuration achieves a significant overall error reduction, the distinct performance disparity between geometrical domains demands a deeper investigation. To better characterize the framework’s capabilities and boundaries, a series of targeted parametric studies was conducted. Rather than attempting to definitively isolate a single root cause, these analyses explore how specific training choices and dataset manipulations influence the model’s performance limits. These sensitivity studies, detailed in the following section, are designed to better identify the capabilities of the framework and inform future development.

4.4 Model tuning and sensitivity analysis

The preliminary results presented in Section 4.3 demonstrate the overall efficacy of the PEDS framework, proving that the neural network successfully corrects the baseline diffusion solver approximations and significantly reduces the global reactivity error. However, this initial evaluation also revealed distinct training limitations, most notably a persistent performance plateau and a non-uniform generalization gap, particularly concentrated on subcritical geometric configurations. To better characterize the framework’s capabilities and operational boundaries, this section presents a comprehensive series of sensitivity studies. These analyses systematically investigate the origins of the residual discrepancies to isolate whether they stem from the neural network architecture, the training data structure, or the inherent limits of the diffusion approximation, defining the limits of the current setup.

In the following subsections, the model’s sensitivity to the choice of training data is first explored by evaluating the impact of dataset size, the isolated training of subcritical and supercritical regimes, and the enforcement of balanced target distributions. Following this, the numerical integrity of the underlying physical solvers is verified. Finally, a detailed exploration of the geometric parameter space is conducted to identify the specific physical configurations that challenge the surrogate’s correction capabilities, thereby mapping the current limits of the framework and informing future developments.

4.4.1 Influence of training dataset size

A scaling study was conducted to evaluate how PEDS performance responds to varying training dataset sizes. This is essential from a practical standpoint, since a surrogate model is only useful if its initial training cost is sufficiently low. If generating the necessary high-fidelity data requires a computational effort comparable to that of a standard design study, the framework defeats its purpose.

All samples were drawn from a common pool of approximately 2000 high-fidelity OpenMC simulations, restricted to $k_{\text{eff}} \in [0.8, 1.2]$. To ensure an unbiased baseline for comparison, the validation and test sets were fixed at 100 samples each across all experiments, consisting exclusively of unseen sample geometries with no overlap between any of the data splits. It is important to emphasize that the validation set is utilized solely for model selection and checkpointing during training, whereas the test set provides a strictly unbiased final evaluation. By maintaining an identical validation and test set across all sizes and random seeds, any observed performance variations can be exclusively attributed to changes in the training data volume and random initialization, rather than discrepancies in the evaluation data.

The training set size was incrementally varied from 250 to 1500 samples. For each size, the training procedure was repeated across 10 independent random seeds to account for statistical variance in data shuffling and initial network weights. Because the random seeds select from the same fixed pool, the training subsets are nested by construction; the samples used in a smaller training set are always included in any larger training set drawn with the same seed.

Figure 4.11 illustrates this sampling procedure for a representative training size of 1000 samples, utilizing Kernel Density Estimation (KDE) to visualize the data distribution. Because the validation and test sets are fixed across all runs, their distributions remain identical. Conversely, the training set distribution fluctuates slightly across the 10 seeds due to the random sampling of the 1000-sample subsets from the full data pool. It should be noted that these KDE curves represent continuous probability densities rather than absolute probabilities. Because the total area under each curve is normalized to one over the k_{eff} domain, the relative height at any specific k_{eff} value serves as a direct visual measure of the sample concentration at that point.

All models were trained for 70 epochs, as in the previous case. While the network was optimized using the mean squared error on k_{eff} , model selection was instead based on a separate evaluation metric: the mean absolute reactivity difference (in pcm), computed on the validation set. This metric was chosen because the main goal of PEDS for this neutron transport application is to reduce the reactivity difference, making it a more physically meaningful indicator of performance than the training loss itself. Because this metric is susceptible to transient noise and epoch-to-epoch training oscillations, an Exponential Moving Average (EMA) was applied to the validation curve to determine the optimal checkpoint. A smoothing factor of 0.85 was assigned to the historical average and 0.15 to the current epoch, creating an effective averaging window of approximately seven epochs. The network weights corresponding to the minimum of this smoothed validation curve were saved in a checkpoint as the final model and subsequently evaluated on the isolated test set.

The aggregated test set metrics for each training size are presented in Table 4.5, with the corresponding

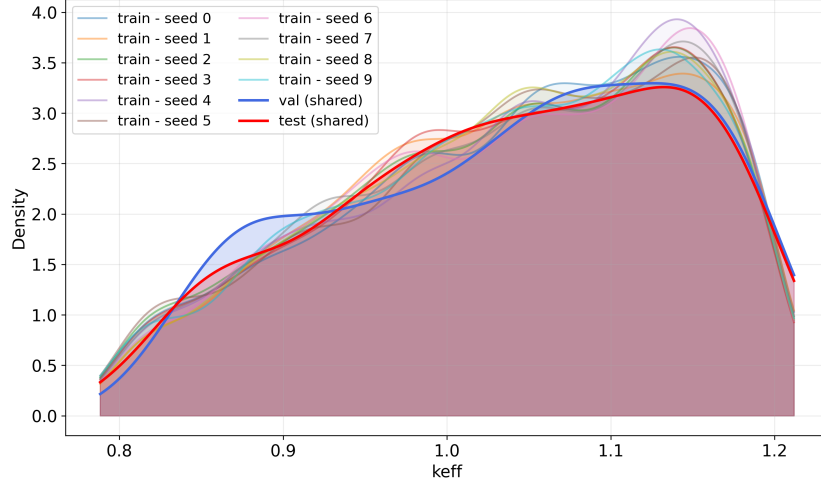
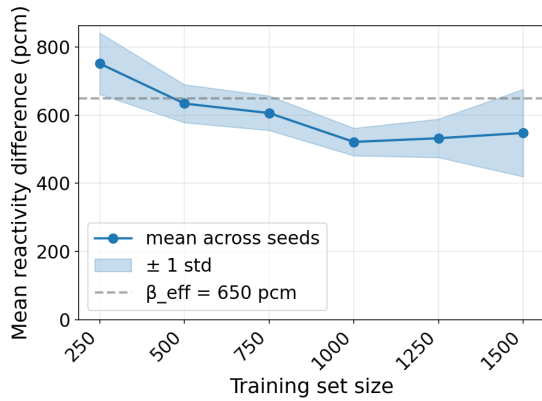


Figure 4.11: Kernel Density Estimation (KDE) of the target k_{eff} distribution for a 1000-sample training set across 10 independent random seeds. The validation and test distributions remain fixed, while the training distribution exhibits minor statistical fluctuations.

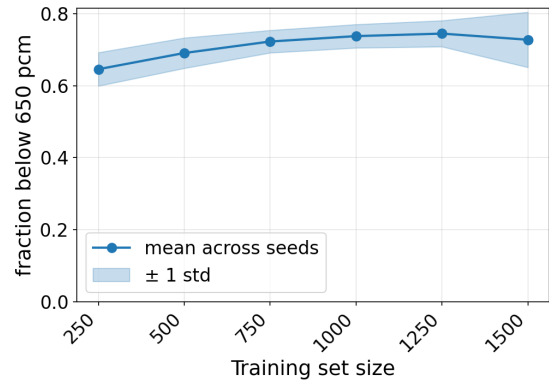
visual scaling trends shown in Figure 4.12.

Table 4.5: Test metrics as a function of training dataset size. The mean error includes the standard deviation across 10 random seeds.

Size	MSE	MAE	Mean (pcm)	Median (pcm)	P95 (pcm)	Frac $< \beta_{\text{eff}}$	Frac < 100 pcm
250	1.12×10^{-4}	7.26×10^{-3}	752 ± 90	443	2451	0.65	0.13
500	7.57×10^{-5}	6.10×10^{-3}	634 ± 56	396	2002	0.69	0.15
750	7.22×10^{-5}	5.82×10^{-3}	607 ± 51	364	1942	0.72	0.14
1000	5.02×10^{-5}	5.02×10^{-3}	522 ± 40	308	1757	0.74	0.20
1250	5.65×10^{-5}	5.10×10^{-3}	533 ± 57	322	1632	0.75	0.20
1500	6.09×10^{-5}	5.33×10^{-3}	548 ± 128	336	1614	0.73	0.18



(a) Evolution of the mean reactivity difference.



(b) Fraction of test samples below 650 pcm.

Figure 4.12: Scaling of test set performance metrics with respect to training dataset size. Data points represent the average across 10 random seeds, with shaded regions indicating the standard deviation.

The data demonstrate a clear saturation in model performance. The mean reactivity difference improves steadily as the dataset grows from 250 to 1000 samples. However, beyond 1000 samples, the addition of more data yields diminishing returns. Furthermore, at 1500 samples, the standard deviation across the 10 seeds noticeably increases. This elevated variance is attributed to specific random seeds becoming trapped in degenerate local minima during early training iterations; these anomalous runs fail to stabilize, polluting the aggregated mean and increasing the overall standard deviation.

This performance plateau at 1000 samples represents a highly favorable outcome for the framework. If the neural network required an excessively large dataset to achieve acceptable accuracy, the computational expense of generating the high-fidelity training data would exceed the cost of running the high-fidelity solver directly for the entire design study, undermining the purpose of a surrogate model. Demonstrating that best performance is reached with only 1000 simulations ensures that the upfront training cost remains minimal and can be easily amortized. This establishes the trained surrogate as a powerful and efficient tool for repetitive applications such as iterative core design and optimization.

4.4.2 Subdivision of training data into subcritical and supercritical regimes

As observed in the preliminary baseline (Section 4.3) and confirmed across all dataset sizes in Subsection 4.4.1, geometries with subcritical k_{eff} values consistently exhibit the highest prediction discrepancies. This recurrent pattern suggests a systemic bias in how the framework learns across the target domain. To determine whether this behavior originates from training the network on a combined pool of subcritical and supercritical cases, or if it reflects an inherent limitation specific to one criticality regime, the dataset was separated and analyzed independently.

The training and evaluation methodology detailed in Subsection 4.4.1 was maintained, utilizing the fixed 70-epoch limit and the EMA-smoothed validation metric for checkpoint selection. As illustrated by the dataset distribution in Figure 4.4, the pool of high-fidelity simulations contains fewer subcritical cases than supercritical ones. Consequently, the subcritical configuration was evaluated using a training set of 500 samples to allow the construction of a balanced subset. The previous scaling study showed that this number of samples still yield robust model convergence. Because the supercritical domain benefits from higher data availability, it was evaluated at both 500 and 1000 samples to assess if additional data would further improve its isolated performance. Each configuration was trained across 10 independent random seeds.

Table 4.6 summarizes the test metrics for each regime before and after the neural network correction. The “before” values represent the baseline accuracy of the uncorrected low-fidelity diffusion solver; because this solver is deterministic and independent of the training process, these values carry no seed variance. The “after” values report the post-correction metrics, presented as the mean and standard deviation across the 10 random seeds.

The results demonstrate that PEDS reduces the prediction error in both regimes; however, the correction is notably more effective for supercritical configurations. After correction with 500 samples, the subcritical model achieves an 84.3% reduction in mean error. In contrast, the supercritical model achieves a 90.3% error reduction at 500 samples, further improving to a 91.5% reduction with 1000 samples, consistent

Table 4.6: Test metrics for isolated subcritical and supercritical configurations, before and after NN correction. The mean error includes the standard deviation across random seeds ($n = 10$ seeds for the corrected results; single evaluation for the uncorrected baseline).

Configuration	Train Size	NN Correction	Mean (pcm)	Median (pcm)	Frac $< \beta_{\text{eff}}$	Frac < 100 pcm	P95 (pcm)
Subcritical	500	Before	7780	7409	0.00	0.00	13099
Subcritical	500	After	1224 ± 55	906 ± 70	0.38 ± 0.04	0.07 ± 0.02	3490 ± 567
Supercritical	500	Before	3406	3287	0.01	0.00	5880
Supercritical	500	After	330 ± 27	246 ± 29	0.87 ± 0.03	0.24 ± 0.03	953 ± 130
Supercritical	1000	Before	3406	3287	0.01	0.00	5879
Supercritical	1000	After	289 ± 32	210 ± 29	0.90 ± 0.03	0.26 ± 0.05	864 ± 111

with what observed in the baseline case. The subcritical configuration stalls at an average error exceeding 1200 pcm, with a drastically lower fraction of samples satisfying the 650 pcm and 100 pcm accuracy thresholds.

It is worth noting that the gap between the two regimes is already present before any correction is applied: the uncorrected baseline error for the subcritical domain (7800 pcm) is approximately 2.3 times worse than that of the supercritical domain (3406 pcm). This confirms that a significant portion of the initial discrepancy originates from the low-fidelity diffusion solver itself, rather than a failure of the machine learning optimization. After correction, the gap between the two regimes widens rather than narrows: at 500 samples, the subcritical mean error (1224 pcm) is approximately 3.7 times worse than the supercritical mean error (330 pcm). While the surrogate framework successfully reduces the error in both regimes, it cannot fully compensate for the larger inaccuracy of the low-fidelity model in the subcritical regime, and the relative disadvantage of the subcritical configuration becomes more pronounced after training.

4.4.3 Impact of a uniform k_{eff} training distribution

PEDS performed worse in the subcritical region, which also coincides with the region that is underrepresented in the dataset due to the skewed k_{eff} distribution produced by the Latin Hypercube Sampling (LHS). This raised the question of whether the weaker performance was caused by the lack of training data in that region, rather than by an intrinsic limitation of the model. To test this, a fixed-range binning strategy was introduced, dividing the k_{eff} domain into linearly spaced bins and enforcing an equal number of samples per bin across the training, validation, and test splits, so that every k_{eff} region is evenly represented.

This approach has some similarity with active learning, where new samples are collected in the areas where the model output struggles the most, in order to improve the coverage of the training data where it is currently weak. However, the scale here is much smaller: no new data is created, since the same fixed pool of samples is simply reorganised to give more weight to the underrepresented k_{eff} region that performs worse.

Due to a lack of available data below $k_{\text{eff}} = 0.85$, the target range was slightly reduced from the original $[0.8, 1.2]$ to $[0.85, 1.15]$ for this study. A baseline model, trained on this same reduced range but without

bin balancing, was used for comparison. Because the raw dataset contains enough subcritical samples to support a balanced training size of only 500, the balanced-split results were compared against a non-balanced baseline using the identical training size of 500 samples. This isolates the effect of balancing from any confounding variations in dataset size. Figure 4.13 shows a KDE plot of the new balanced sets, which opposes to the shape introduced in Figure 4.11, now adapted to the tighter dataset.

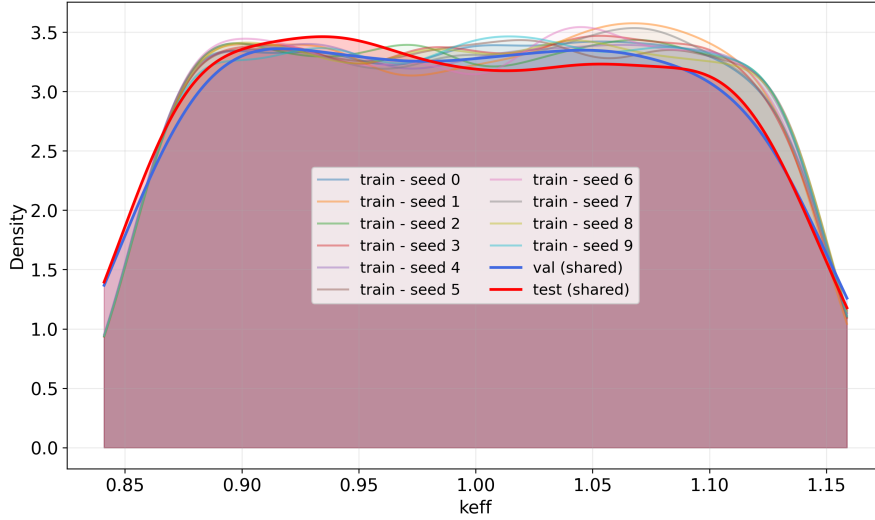


Figure 4.13: Kernel Density Estimation (KDE) of the k_{eff} distribution for the balanced dataset splits, constrained to the $[0.85, 1.15]$ range.

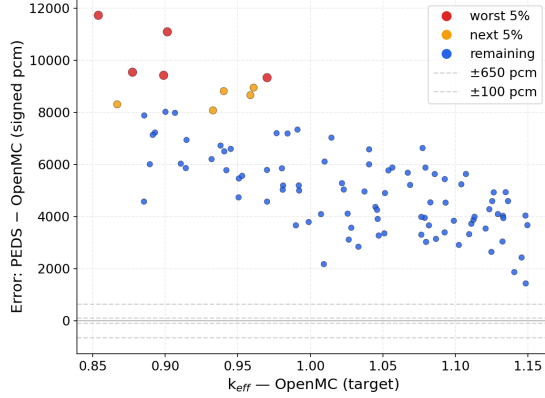
The relevant test metrics are reported in Table 4.7, following the same set of metrics used throughout the studies in this chapter, to allow a direct comparison between the balanced and unbalanced configurations.

Table 4.7: Test metrics for unbalanced and balanced k_{eff} bins, training size 500, before and after NN correction. Corrected results report mean $\pm$ standard deviation across 10 seeds; uncorrected values are single-seed, with no variance.

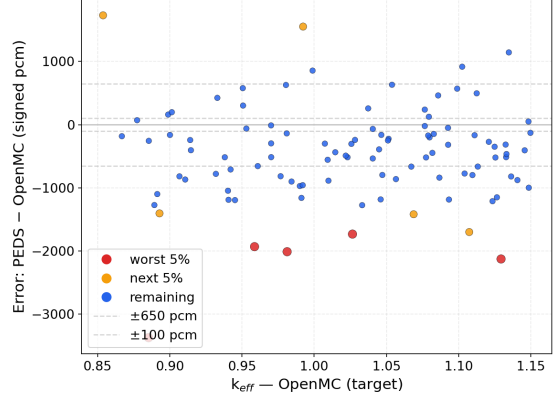
Configuration	NN Correction	Mean (pcm)	Median (pcm)	Frac $< \beta_{\text{eff}}$	Frac < 100	P95 (pcm)
Unbalanced	Before	5303	5016	0.00	0.00	8971
	After	686 ± 48	468 ± 51	0.61 ± 0.04	0.12 ± 0.03	2070 ± 265
Balanced	Before	5594	5484	0.00	0.00	8949
	After	681 ± 72	479 ± 71	0.61 ± 0.05	0.14 ± 0.04	2032 ± 259

The comparison shows that balancing the training bins across the k_{eff} range does not produce a substantial improvement in the overall test metrics: the mean and median errors, as well as the fraction of samples below 650 pcm, remain similar between the two configurations, with the balanced case only marginally ahead in the fraction of samples below 100 pcm. This suggests that simply re-weighting the sampling density in k_{eff} is not sufficient to fix the accuracy gap in the subcritical region, since the underlying difficulty of that region persists regardless of how densely it is sampled.

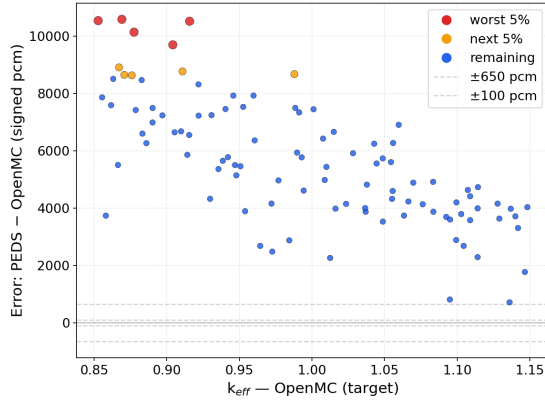
Figure 4.14 shows the signed prediction error in the test set against the target k_{eff} for both the unbalanced and balanced cases. In the unbalanced case (b), the worst-performing samples (highlighted in red) after the NN correction are scattered across the entire k_{eff} range. Looking at the balanced case (d), overall the



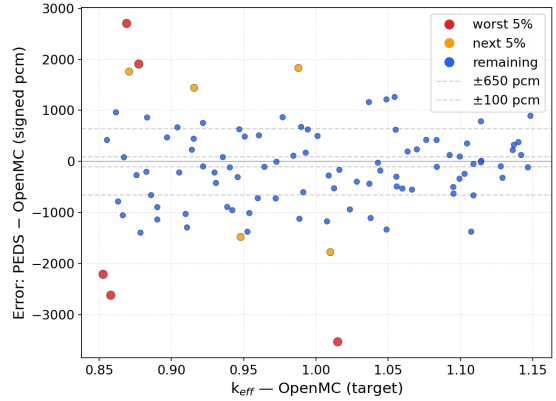
(a) Unbalanced, before correction.



(b) Unbalanced, after correction.



(c) Balanced, before correction.



(d) Balanced, after correction.

 Figure 4.14: Prediction error against target k_{eff} found in the test set, for the unbalanced and balanced configurations, before and after the NN correction.

dataset points are visibly more uniform across the k_{eff} axis, following the binning strategy. However, the worst 5% of samples now collect at the lower end of the k_{eff} range. This indicates that the subcritical region continues to be the most challenging for the model to predict accurately; increasing its representation in the training set merely reshuffles where the largest errors occur rather than reducing their magnitude.

These findings confirm that the discrepancies observed in the subcritical regime are not caused by insufficient or representation of these configurations in the training data. An additional verification was conducted to determine whether the observed discrepancies could be attributed to numerical limitations in the physical solvers. The convergence of the OpenMC reference calculations and the influence of the diffusion solver spatial mesh were assessed for selected configurations with large residual errors. The complete analysis is provided in Appendix C. Neither source was found to be the primary cause of the observed discrepancies. The remaining error therefore appears to be associated with physical characteristics of specific geometries that are more difficult for the neural network to correct.

4.4.4 Study over the parameters space

The previous results indicate that the large discrepancies observed for some surrogate predictions cannot be explained solely by limited local coverage of the training data. Instead, they suggest a broader limitation in the current PEDS framework, which may depend on the physical characteristics of the sampled configurations.

An improved data splitting methodology was introduced to investigate this rigorously, refining the baseline approach described in Subsubsection 3.3.4.3. The original method partitioned the dataset into k_{eff} quantile bins based on equal sample counts. However, enforcing equal quotas means that naturally sparse criticality regimes must span much wider numerical ranges, diluting the geometric representation in those specific areas. Furthermore, because geometry parameters are not explicitly stratified, random splits can match in k_{eff} distribution while differing significantly in their underlying physical compositions.

To facilitate a more robust comparison across the splits, the dataset was expanded and partitioned using a nested stratification scheme. The test set was increased to 300 samples and kept fixed to provide a consistent evaluation benchmark across independent runs, while the validation set was expanded 500 samples, and redrawn at every run like the training dataset, which was kept at 1000 samples. Samples were first strictly stratified by their fuel radius into exactly five equal quantile bins, guaranteeing uniform coverage of this primary driving parameter (which sensitivity studies identified as the most influential geometric feature). Within each of these primary geometric strata, the remaining samples were allocated to explicitly mirror the underlying k_{eff} distribution proportionally across all three splits.

By locking the primary geometric driver and matching the criticality profiles, the secondary geometric parameters are allowed to distribute naturally while maintaining highly representative coverage. This rigorous sampling strategy ensures a more balanced baseline, allowing any observed error reduction to be confidently attributed to the model’s actual correction capabilities across different physical regimes, rather than arbitrary dataset imbalances. Figure 4.15 shows the overall performance of the PEDS correction across the stratified k_{eff} ranges. The figure reports the relative distribution of samples, the mean absolute reactivity discrepancy before and after correction, and the fraction of samples for which the residual discrepancy is below the effective delayed-neutron fraction, $|\Delta\rho| < \beta_{\text{eff}} = 650$ pcm. The corresponding detailed numerical breakdown is provided in Appendix Appendix D (Table D.1).

Evaluating the model across this stratified space confirms the physical trend suspected earlier: the predictive error decreases smoothly as the target k_{eff} increases. This pattern holds robustly across the data splits, both before and after the neural network correction is applied. Similarly, the fraction of corrected samples satisfying $|\Delta\rho| < \beta_{\text{eff}}$ increases considerably with k_{eff} . Interestingly, comparing these results against the isolated training scenarios previously presented in Table 4.6 reveals an underlying regularization mechanism. When evaluated exclusively on subcritical samples, the model trained on the mixed domain outperforms the model trained solely on subcritical data (achieving a mean error of 933 pcm compared to 1224 pcm). Training the model across the full k_{eff} spectrum provides a broader physical context that acts as a natural regularizer. By including both regimes, the model learns a more generalized physical representation, transferring useful geometric structure across the entire domain to improve overall accuracy.

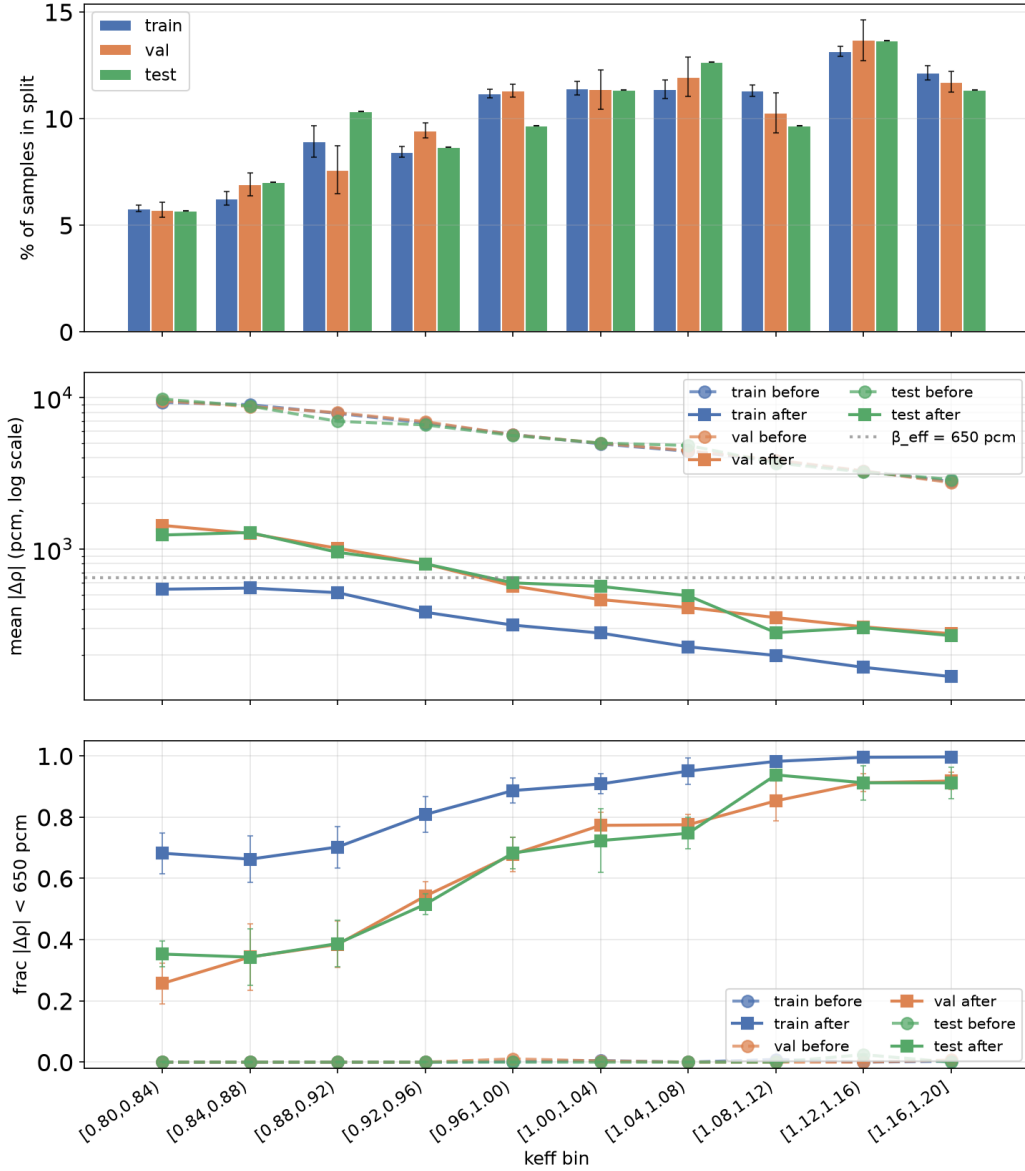


Figure 4.15: PEDS performance across stratified k_{eff} bins for the training, validation and test datasets. The upper, middle, and lower panels show the sample distribution, mean absolute reactivity discrepancy before and after correction, and fraction of samples satisfying $|\Delta\rho| < \beta_{\text{eff}} = 650$ pcm, respectively. The middle panel uses a logarithmic scale; error bars denote the standard deviation over five runs.

However, the bin-wise results reveal a distinct, non-uniform generalization gap. While the network accurately generalizes to unseen geometries in the supercritical regime, the discrepancy between the training error and the validation/test error grows substantially in the lowest subcritical bins. This behaviour is also reflected by the fraction of samples satisfying $|\Delta\rho| < \beta_{\text{eff}}$. This indicates a generalization limitation within the neural network itself. The model appears to memorize localized mathematical corrections for highly subcritical configurations during training, which fail to generalize effectively to novel subcritical geometries. Conversely, the physical rules learned for supercritical domains are more robust when applied to unseen data.

Although the criticality condition alone is clearly a strong predictor of error magnitude, it does not fully explain why certain geometries are harder to correct than others within the same k_{eff} range. Further studies investigated whether this behavior is driven by specific input parameters, either individually or in combinations. As an initial exploratory step, a parallel-coordinates plot was constructed to visualize the six geometry parameters across the 200 test samples, with each line colored by its absolute reactivity error $|\Delta\rho|$. While this single configuration provides only a qualitative snapshot of the parameter space rather than a statistically exhaustive proof, it can highlight immediate structural trends. Figure 4.16 shows the error before training (representing the uncorrected diffusion solver), while Figure 4.17 shows the error after the neural network correction.

Before correction, the largest errors are spread across several parameter combinations, but the geometries with a low fuel annulus outer radius and a high CR fraction stand out among the worst cases. After correction, the overall error scale drops sharply, down to about 2000 pcm at most, and most of these radius-driven cases are resolved. A smaller group of high-error lines remains after correction, again focusing on the same parameters, suggesting that this specific combination is harder for the network to learn even after training.

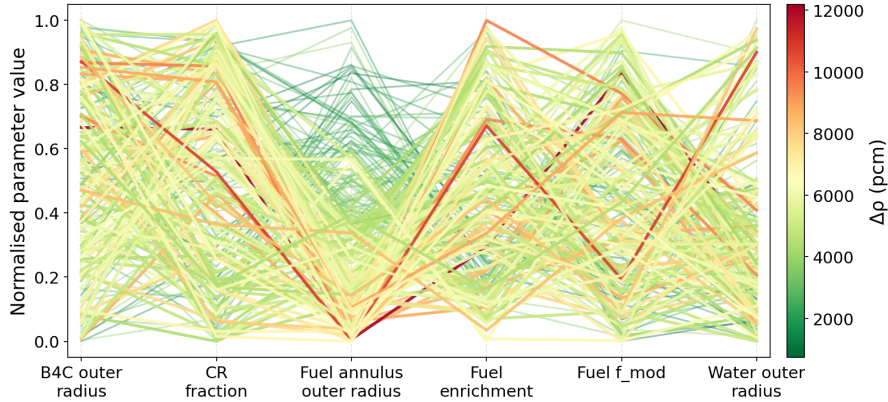


Figure 4.16: Parallel-coordinates plot of the six geometry parameters for a 200 test-set case, colored by absolute reactivity error before neural-network correction (reference case, 1000 training samples).

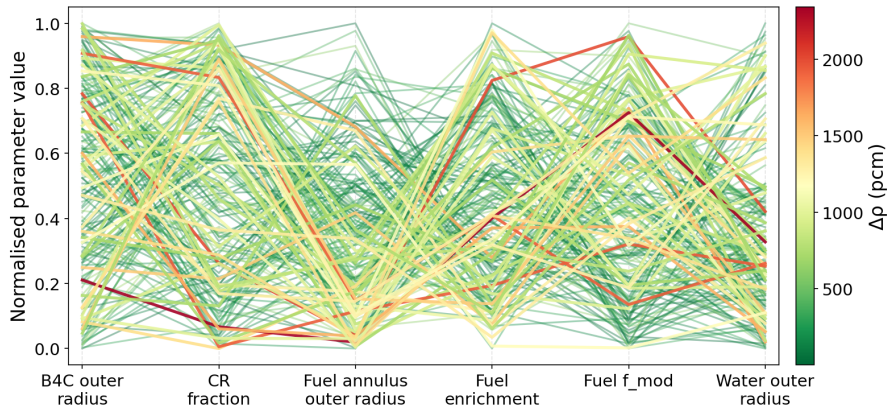
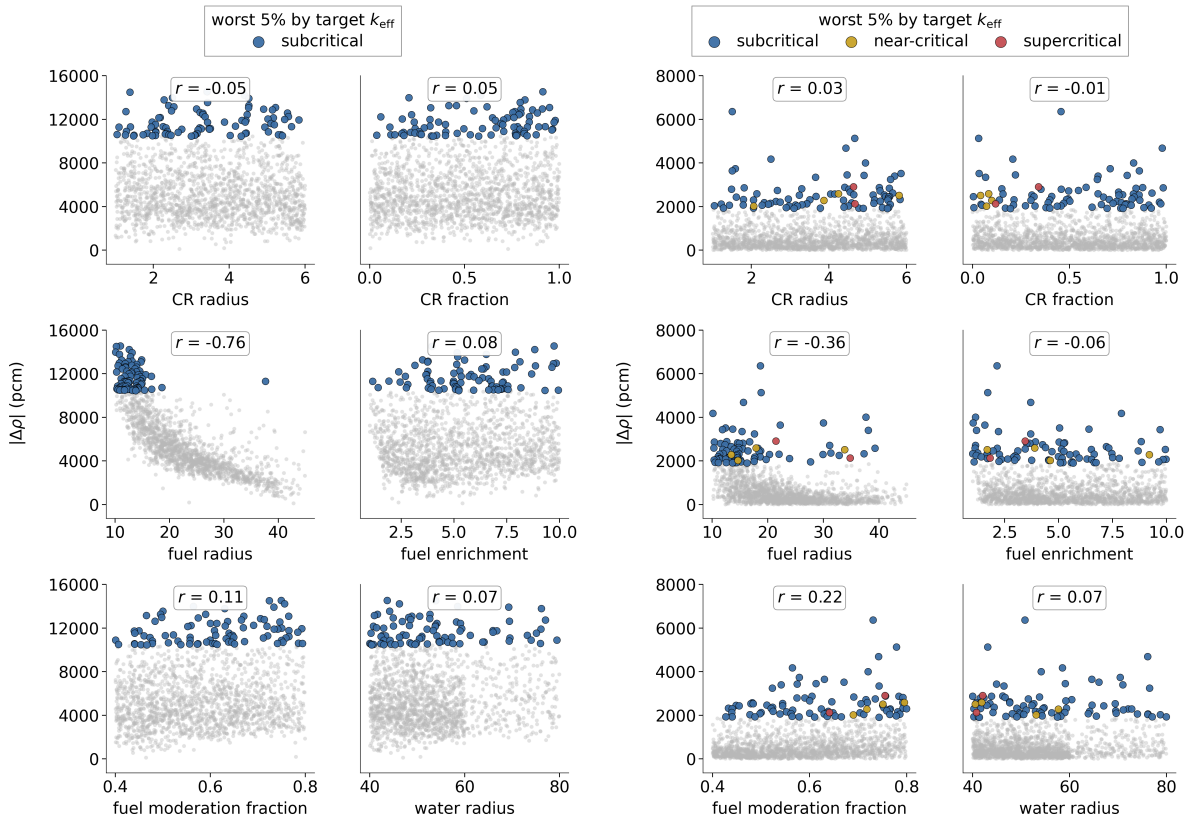


Figure 4.17: Parallel-coordinates plot of the same six geometry parameters and test cases as Figure 4.16, after neural-network correction.

To check whether the trend holds more broadly, the same six geometry parameters were compared individually against the absolute reactivity error using scatter plots and Pearson correlation coefficients (r). The parameter ranges evaluated correspond to the bounds previously defined in Table 4.1. This analysis was expanded across the training (1000 samples), validation (500 samples), and test (300 samples) datasets to thoroughly evaluate the performance and error distribution of the framework.

Figure 4.18 presents this individual parameter analysis for the validation split, before and after the neural network correction. To track how the error distribution shifts across different physical regimes, the worst 5% of samples (representing the highest mismatch) are explicitly highlighted and categorized by their target criticality state: subcritical, near critical, and supercritical.



(a) Diffusion solver error (before NN correction).

(b) Residual error (after NN correction).

Figure 4.18: Scatter plots comparing the absolute reactivity error against individual geometry parameters for the validation set. The Pearson correlation coefficient (r) is presented for each parameter subplot. The worst 5% of samples are highlighted to illustrate the dominant criticality regimes driving the highest errors before (a) and after (b) correction.

Prior to training (Figure 4.18a), the error strictly reflects the baseline diffusion solver approximations. The fuel annulus outer radius clearly stands out from the other five parameters, exhibiting a consistent strong negative correlation across all splits: $r \approx -0.77$ for training, -0.76 for validation, and -0.75 for testing. A distinct physical trend is visible: smaller fuel radii are heavily associated with larger computational discrepancies. Furthermore, the worst performing 5% of configurations are exclusively subcritical geometries. The remaining five parameters all demonstrate negligible correlations ($|r| \leq 0.16$

across all splits), indicating that none of them individually drive the initial diffusion error.

Following the application of the neural network correction (Figure 4.18b), the overall error pattern changes significantly. The global error scale drops sharply, and the fuel radius correlation weakens globally, settling between $r = -0.36$ and -0.39 across the three data splits. This confirms that the neural network successfully learns to mitigate the majority of this radius driven bias on both seen and unseen geometries. While subcritical geometries still dominate the residual errors, the worst performing 5% of samples now include a small fraction of near critical and supercritical cases, indicating a more evenly distributed error floor.

A deeper inspection of the remaining geometric parameters reveals a subtle generalization discrepancy regarding the fuel moderation fraction. While the correlation remains exceptionally weak in the training set ($r = 0.12$), it increases noticeably in the unseen validation ($r = 0.22$) and test sets ($r = 0.29$). This divergence suggests a slight overfitting behavior: the network successfully learns to balance the moderation fraction during training, but it struggles to perfectly generalize that specific correction to novel geometries, leaving a minor residual bias on unseen data.

To further investigate whether the residual errors are driven by coupled geometric effects, a two-parameter interaction study was also conducted. This analysis evaluated the statistical interaction scores between all possible pairs of the six geometric parameters to determine if specific combinations uniquely degrade the model’s accuracy. This evaluation was restricted to the training and validation datasets, as the 300-sample test set lacked sufficient data density across the two-dimensional parameter grids to yield reliable interaction metrics. Throughout both the initial baseline evaluation and following the neural network correction, interaction scores involving the dominant fuel annulus outer radius remained notably weak, peaking at 0.19. This lack of strong coupling reinforces that the initial diffusion error is driven by the fuel radius as an independent variable, and it confirms that no specific parameter pair emerges with a consistently dominant score to definitively explain the remaining residual error.

Ultimately, through this initial parametric exploration, no clear geometric correlation, either individual or coupled, was found that fully captures the distribution of the residual errors after the neural network correction. While the surrogate model successfully reduces the overall discrepancy, the remaining errors continue to exhibit a wider spread across subcritical configurations. This complex behavior suggests that further investigation is required beyond simple geometric parameter dependencies. Future work should explore other potential sources of discrepancy, such as the inherent physical approximations of the diffusion solver itself, while defining a more precise range of applicability for the overall framework.

4.5 Forward design study

Reactor forward design-space search is precisely the class of problem where a fast surrogate like PEDS is most valuable: rather than simulating a single fixed geometry, the engineer must search across many candidate configurations to find one that satisfies a set of performance targets, requiring the solver to be evaluated repeatedly across a broad design space. This section aims to demonstrate that benefit concretely, using the six-parameter reactor core geometry introduced in Section 4.2 as a case study. For this demonstration, the goal of the search is to identify configurations that are simultaneously

near-critical ($k_{\text{eff}} \approx 1$) and have a low radial power peaking factor (PPF), the ratio of the maximum to the average power density within the fuel [31, 38]. PPF is a standard design and safety metric in reactor engineering, since it determines the margin to fuel thermal limits: a design with a low PPF distributes power more evenly across the core, reducing the risk of localized overheating even when the total core power is unchanged.

The first study, Subsection 4.5.1, establishes a baseline in which the design space is searched using only the high-fidelity (HF) solver, replicating a standard nuclear engineering workflow. While other machine-learning methods could in principle accelerate this search, evaluating them is outside the scope of this work, which focuses specifically on the benefits offered by the PEDS methodology.

Subsection 4.5.2 then repeats the same search strategy but replaces the high fidelity evaluations with an ensemble of five independently trained PEDS surrogate models. This allows the search space to be scaled up substantially at a fraction of the computational cost, while simultaneously providing an uncertainty estimate across model predictions to avoid reliance on a single model realization. By comparing the outcomes of the two studies, alongside a test that isolates the contribution of the neural network correction by removing it entirely, the acceleration and reliability that PEDS provides for this class of design problem are quantified.

4.5.1 Baseline design study: High-Fidelity LHS search

The steps involved in a standard forward design problem are: sample the design space broadly, evaluate every candidate with the solver, and filter the resulting pool down to the samples that satisfy the design constraints. This baseline represents both the reference search strategy and the reference computational cost against which the PEDS-accelerated search of Subsection 4.5.2 is then compared.

The Latin Hypercube Sampling (LHS) approach was used to generate the candidate geometries, and the OpenMC solver evaluated each sample. Since the dataset previously generated for PEDS training (Section 4.2) did not store the flux and power in every mesh cell, a new set of 500 OpenMC evaluations was run for this study, judged a sufficient sample size for this demonstration. Note that this represents a single LHS realization; a different seed could produce a slightly different viable yield, as full saturation requires a much larger pool.

The analysis retains a design as feasible if its reactivity deviation from criticality satisfies $|\Delta\rho| \leq 1500$ pcm. Subsequent sorting ranks the selected candidates primarily by their PPF (lowest is best), using $|\Delta\rho|$ as a tie-breaker for designs with similar peaking factors. The full evaluation campaign was executed on the HPC cluster, distributed over 32 CPU cores. The solution of all 500 cases completed after 8 h 58 min 30 s of wall-clock time (approximately 287.2 CPU-hours, corresponding to 11 days 23 h 12 min of allocated CPU time), averaging about 65 seconds per HF evaluation. Each run was stopped once the k_{eff} standard deviation reached the target trigger value of $\sigma_{k_{\text{eff}}} < 40$ pcm.

The search yielded a very narrow set of viable designs: only 28 of the 500 sampled geometries (5.6%) successfully passed the 1500 pcm criticality constraint. Within this feasible subset, k_{eff} ranged from 0.9878 to 1.0131, and PPF values spanned from a minimum of 1.2534 to a maximum of 2.2035, with a

median around 1.5967. Detailed parameters for the top ten feasible designs from this baseline evaluation are provided in Section E.1, while aggregate quality metrics for the full set of 28 feasible designs are compared with the PEDS assisted results in Table 4.8.

4.5.2 Surrogate-assisted LHS screening with a PEDS ensemble

To demonstrate the acceleration provided by the surrogate, the design search was repeated with 5000 LHS-generated core geometries, ten times the HF-only baseline sample size. Candidates were evaluated by an ensemble of five independently trained PEDS models, with the same architecture and training data but different random initializations and train/validation splits, following what was done in [1]. Using an ensemble is common practice to avoid basing the analysis on a single model realization and provides an uncertainty estimate across model predictions in addition to the ensemble mean.

For every candidate, the ensemble mean was used to predict k_{eff} and the diffusion-based radial PPF, while retaining the standard deviation across the five seeds, $\sigma_{k_{\text{eff}}}^{\text{PEDS}}$. Since the surrogate carries a small residual error (approximately 600 pcm) on unseen geometries, quantified in the test set results of previous studies, a conservative pre-verification gate was applied. Only designs with an ensemble-mean k_{eff} within 1000 pcm from the target were classified as feasible. This reduced the set of 5000 candidates to 154 feasible geometries (3.08%). Across these designs, the standard deviation across the ensemble models was examined to quantify the agreement between their predictions. The mean of $\sigma_{k_{\text{eff}}}^{\text{PEDS}}$ was found to be 346 pcm equivalent, with the corresponding PPF standard deviation averaging 0.156.

The feasible geometries were ranked primarily by their predicted proximity to the criticality target, with the PEDS estimated PPF used only as a secondary criterion. Because the diffusion model uses region-homogenized fission cross sections, its PPF captures only coarse flux-driven power variation and does not reliably represent the heterogeneous local power distribution resolved by OpenMC. This inaccuracy makes it unsuitable as a primary ranking criterion, unlike in the HF baseline. To validate the surrogate selections, the top 30 ranked designs were passed to OpenMC for high-fidelity verification. A design was ultimately considered successful if the OpenMC evaluation confirmed it was within 1500 pcm of criticality, matching the benchmark used in the baseline study (Subsection 4.5.1). Of the 30 PEDS proposed candidates, 28 passed this threshold.

The high fidelity quality metrics for these 28 feasible designs are summarized in Table 4.8 and compared to the results obtained from the HF-only baseline. The mean discrepancy between the PEDS ensemble prediction and the OpenMC result was 434.3 pcm for k_{eff} , which is consistent with the expected test accuracy of the surrogate. Conversely, the mean absolute difference for the PPF was 1.275, confirming that the local power peaking factor remains a more challenging quantity for the surrogate to predict accurately. Despite this limitation, the surrogate assisted screening produced designs with a superior average proximity to criticality (a mean of 412.9 pcm compared to 588.3 pcm for the baseline) and a lower mean PPF. Detailed parameters for the ten best geometries identified by the PEDS ensemble are provided in Section E.2.

Comparing these results with the conventional baseline illustrates the potential value of the methodology. In the high fidelity only baseline (Subsection 4.5.1), 500 random geometries were evaluated with OpenMC

Table 4.8: High-fidelity quality metrics for 28 HF-feasible designs ($|\Delta\rho(k_{\text{HF}}, 1)| \leq 1500$ pcm).

Cohort	Metric	Min	Mean	Median	Max
Ensemble PEDS + OpenMC HF	$ \Delta\rho(k_{\text{HF}}, 1) $ (pcm)	42.0	412.9	316.2	1359.0
	PPF_{HF}	1.2353	1.4949	1.3729	2.4518
	$ \Delta\rho(k_{\text{PEDS}}, k_{\text{HF}}) $ (pcm)	38.3	434.3	394.8	1351.3
	$ \text{PPF}_{\text{PEDS}} - \text{PPF}_{\text{HF}} $	0.142	1.275	1.133	4.475
OpenMC-only LHS baseline	$ \Delta\rho(k_{\text{HF}}, 1) $ (pcm)	44.7	588.3	572.7	1289.3
	PPF_{HF}	1.2534	1.6397	1.5967	2.2035

to identify viable designs. By using the PEDS ensemble during the initial screening stage, the search space was expanded tenfold to 5000 candidates with negligible computational overhead, while high fidelity verification was limited to 30 OpenMC evaluations and a comparable number of viable designs was efficiently identified.

To quantify this efficiency and properly assess the computational advantage, a detailed cost analysis must be performed. The complete screening of 5000 candidate geometries required $T_{\text{screen}} = 286.0$ s (4 min 46 s), corresponding to 57.2 ms per geometry for the evaluation across all five ensemble models. The subsequent 30 OpenMC verification calculations required $T_{\text{verif}} = 2267.2$ s (37 min 47 s), or 75.6 s per calculation. Therefore, the marginal computational cost of one PEDS assisted design search was $T_{\text{marginal}} = T_{\text{screen}} + T_{\text{verif}} = 2553.2$ s (42 min 33 s). This marginal cost must be weighed against the initial investment required to develop the surrogate. The one time cost of the PEDS ensemble consisted of 1500 HF calculations used for training (1000 samples) and validation (500 samples), estimated at three times the cost of the 500-samples baseline campaign run $T_{\text{data}} \approx 3T_{\text{baseline}} \approx 26$ h 54 m. The actual training of the five models required $T_{\text{fit}} = 5 \times 1584$ s ≈ 2 h 12 min. The total fixed training cost was therefore $T_{\text{train}} = T_{\text{data}} + T_{\text{fit}} \approx 29$ h 06 min.

Although the first PEDS assisted search includes this substantial upfront cost, the trained ensemble can be reused for all subsequent design searches. To evaluate the practical viability of the methodology, the break even number of equivalent design searches, K_{BE} , was calculated. This was obtained by equating the cumulative cost of repeated high fidelity only searches with the total cost of the PEDS assisted searches:

$$K_{\text{BE}}T_{\text{baseline}} = T_{\text{train}} + K_{\text{BE}}(T_{\text{screen}} + T_{\text{verif}}) \Rightarrow K_{\text{BE}} \approx 3.52. \quad (4.3)$$

This result indicates that the computational investment required to generate data and train the surrogate is fully recovered after conducting four design searches. Consequently, the surrogate approach becomes highly advantageous for extensive or iterative design explorations.

An additional evaluation was performed to isolate the accuracy gained from the neural network correction compared to the baseline low fidelity diffusion solver. When the 154 geometries previously classified as feasible by the surrogate were reevaluated using the diffusion solver alone, none of the 154 designs satisfied the reactivity threshold. Restricting the comparison to the 30 designs with available HF verification, the uncorrected diffusion solver produced a mean discrepancy against OpenMC of 5744 pcm for k_{eff} . This error is approximately 10.6 times larger than the 541 pcm discrepancy achieved by the PEDS ensemble.

In contrast, the mean absolute error for the PPF was comparable, measuring 1.335 for the diffusion solver and 1.263 for PEDS; this is expected, as both methods compute the power peaking factor from the same averaged formulation. This confirms that the low-fidelity diffusion solver on its own is insufficiently accurate on k_{eff} to be useful for this screening task.

These trends are visualized in Figure 4.19, which plots the predicted k_{eff} and PPF against their HF values for both PEDS and the uncorrected diffusion-only solver. The diffusion-only predictions are consistently biased high in k_{eff} relative to HF, while PEDS predictions cluster much closer to the 1:1 line, a trend seen before in Figure 4.7. This confirms that the generator network’s correction to the XS inputs is responsible for PEDS’s improved k_{eff} accuracy.

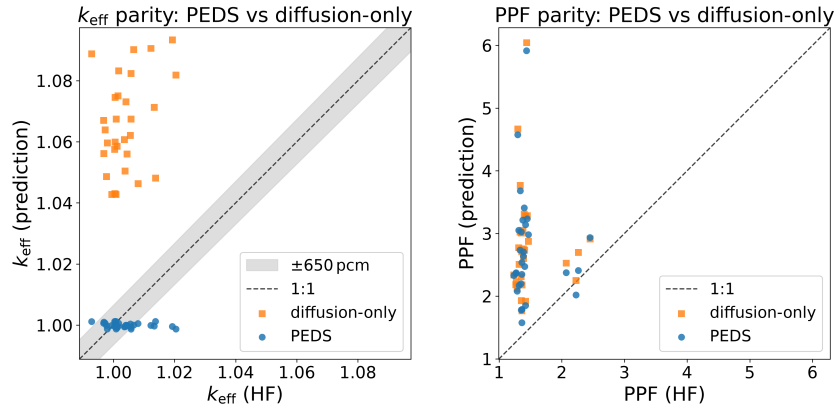


Figure 4.19: Parity plots comparing the predicted k_{eff} and PPF against high fidelity OpenMC results for the 30 verified designs. The performance of the PEDS ensemble is contrasted with the uncorrected diffusion solver.

Chapter 5

Conclusion and Outlook

This thesis investigated the applicability of the Physics Enhanced Deep Surrogate framework to neutron transport calculations. The main objective was to develop and assess a complete PEDS methodology for an eigenvalue based neutron transport problem. This required the implementation of a low fidelity diffusion solver, the formulation of an adjoint based gradient calculation, the integration of the solver within the neural network, and the development of a suitable cross section correction strategy. The methodology was successfully implemented and evaluated for the considered one dimensional cylindrical reactor case.

By embedding a diffusion solver directly within the neural network architecture, the framework ensures that its outputs remain physically consistent. This structural constraint provides a level of interpretability and trust that purely data-driven AI models lack. Rather than generating unconstrained black-box predictions, the surrogate produces the full input cross-sections - combining a fast baseline prediction with a learned correction - that improve the solver's eigenvalue estimation, bringing it substantially closer to the high-fidelity target.

The strength of this methodology lies in its ability to leverage this correction to produce enhanced cross-sections capable of bridging the accuracy gap inherent to the diffusion approximation. These should not be interpreted as exact values, but rather as optimized, effective inputs that allow the low-fidelity solver to succeed where true, geometry-specific MGXS fail. As demonstrated, this correction maintains physical integrity and yields a realistic flux profile. Finally, relying on a mathematical adjoint formulation to compute gradients provides a major structural advantage. It eliminates the need for the underlying solver to be natively differentiable, offering flexibility for future integrations with other established nuclear engineering codes.

The present work was limited to a simplified one dimensional, two energy group cylindrical model. This choice allowed the surrogate implementation to be verified in a controlled setting. The test results demonstrated that PEDS substantially improved the criticality prediction of the diffusion solver. On the locked test set, the mean reactivity error of the diffusion solver was reduced by 88.1%, corresponding to an approximately 8.4 fold improvement. Approximately 69% of the test configurations were predicted within the delayed neutron fraction threshold after correction. The improvement was obtained across

the full considered range of k_{eff} , although supercritical configurations were nearly twice as likely as subcritical ones to fall within this threshold.

The additional studies carried out showed that the lower performance in the subcritical region was not primarily caused by an insufficient representation of these configurations in the training data. Instead, it was associated with the larger baseline error of the diffusion approximation in this regime. Nevertheless, a substantial part of this error was corrected even in the most challenging region of the investigated parameter space. The analysis over the design parameter space further showed that the baseline diffusion error was strongly correlated with the fuel annulus radius. This systematic dependence was substantially reduced after the PEDS correction. The result indicates that the network learned to compensate for a relevant limitation of the diffusion approximation within the considered geometry, rather than only reducing the average error over the sampled configurations.

The objective of the surrogate is not to reproduce the exact precision of a high fidelity Monte Carlo calculation, but to provide rapid and informative evaluations that can guide design choices. The forward design study presented in Section 4.5 demonstrated this use case, showing that the surrogate can be used to explore the design space efficiently and screen out few configurations to then verify with a more accurate model. Once the upfront computational cost of generating training data and optimizing the network is paid, the surrogate can be employed in a fraction of a second with negligible additional cost. This advantage is expected to become even more valuable as the number of design parameters increases. A larger parameter vector requires many more high fidelity evaluations to adequately cover the design space. In this setting, a fast and reliable surrogate such as PEDS would allow to explore much larger design spaces, identifying promising configurations that can then be confirmed with high fidelity calculations.

5.1 Future Work

Several directions could extend the present work. A first useful step would be to place the PEDS results in a broader perspective by comparing the surrogate against simpler machine learning approaches, such as a Gaussian process regression trained directly on the design parameters. Such a comparison would help clarify how PEDS performs relative to other methods in terms of accuracy, inference time, and the amount of high fidelity training data required.

A second set of improvements concerns the training methodology itself. Different loss formulations could be tested, including a Huber loss, or a loss defined directly on the reactivity difference, rather than on k_{eff} itself. Another option is the Gaussian negative log likelihood loss used in [1], which could provide an uncertainty estimate alongside the predicted eigenvalue. In addition, alternative strategies for generating the training data, such as active learning, could be investigated to reduce the number of high fidelity calculations required to reach a given accuracy.

A further direction concerns the remaining generalisation gap between the training and validation sets, and consequently the test set. Individual training trajectories could be examined in more detail to identify configurations for which the network converges to a good result on the training set but fails to generalise, which may help clarify whether this behaviour is related to the network capacity, the

distribution of training data, or the specific geometries involved. Similarly, the systematic difference in accuracy observed across different k_{eff} ranges deserves further investigation, to better understand its origin and to explore possible mitigation strategies.

Finally, the scope of the physical model could be extended. Within the current one dimensional diffusion solver, additional material regions and design parameters, such as further material fraction controls, could be introduced to test the framework on more varied configurations. Beyond this, extending the low fidelity solver to two spatial dimensions would allow the framework to be tested on more realistic and challenging problems, such as the C5G7 benchmark, a standard heterogeneous reactor geometry in which pronounced flux gradients challenge the diffusion approximation.

This work represents a first step rather than a final answer: a proof of concept establishing that physics-enhanced surrogates find an application in neutron transport. This opens considerable freedom for exploration, whether in refining the methodology further or in extending it to new studies.

Data availability

All the data and codes needed to reproduce the results in this work are available in a GitHub repository at https://github.com/catefrau/PEDS_NT.

Appendix A

Polynomial regression evaluation metrics

To evaluate the efficacy of the polynomial regression used for the baseline cross-sections (introduced in Subsubsection 3.3.4.1), the model was fitted to a representative dataset of 500 samples drawn from high-fidelity Monte Carlo simulations. The generation and full context of this dataset are detailed later in Chapter 4.

Table A.1 presents the coefficient of determination (R^2), root mean squared error (RMSE), and chi-squared statistic (χ^2) for each multigroup cross-section (MGXS), sorted in descending order of fit quality. The results demonstrate that the regression captures the behaviour of the fuel and control-rod cross-sections with very high accuracy, while moderator cross-sections exhibit more modest agreement. However, as explained in the main text, this variation in fit quality does not compromise the framework. The regression is intended only to supply a physically meaningful starting point, not to reproduce the reference MGXS exactly. In fact, even if all cross-sections were perfectly matched ($R^2 = 1$ for every quantity), the resulting diffusion solution would still deviate from the Monte Carlo reference, as this deviation originates from the diffusion approximation itself. The role of the polynomial baseline is therefore simply to anchor the surrogate in the correct physical regime. The neural network’s correction phase is then entirely responsible for capturing and bridging this inherent low-fidelity solver discrepancy.

Table A.1: Polynomial regression evaluation metrics for each multigroup cross-section, sorted by R^2 in descending order. RMSE denotes root mean squared error; χ^2 denotes the chi-squared goodness-of-fit statistic. Subscripts 1 and 2 denote the fast and thermal energy groups, respectively, while $S_{gg'}$ denotes the scattering transfer from group g to g' .

Region and Parameter	R^2	RMSE	χ^2
Control rod $\nu\Sigma_{f,1}$	1.0000	0.0000	0.0000
Control rod $\nu\Sigma_{f,2}$	1.0000	0.0000	0.0000
Moderator $\nu\Sigma_{f,1}$	1.0000	0.0000	0.0000
Moderator $\nu\Sigma_{f,2}$	1.0000	0.0000	0.0000
Fuel S_{12}	0.9993	0.0002	0.0007
Fuel S_{22}	0.9993	0.0038	0.0120
Fuel $\nu\Sigma_{f,2}$	0.9993	0.0056	0.0496
Fuel $\Sigma_{a,2}$	0.9993	0.0027	0.0189
Fuel S_{11}	0.9991	0.0003	0.0002
Fuel $\nu\Sigma_{f,1}$	0.9990	0.0002	0.0014
Fuel D_1	0.9987	0.0017	0.0014
Fuel $\Sigma_{a,1}$	0.9986	0.0001	0.0004
Control rod $\Sigma_{a,2}$	0.9940	0.7378	82.8396
Fuel S_{21}	0.9938	1.82×10^{-5}	0.0003
Fuel D_2	0.9927	0.0032	0.0146
Control rod D_1	0.9750	0.0142	0.1181
Control rod $\Sigma_{a,1}$	0.9742	0.0038	0.4838
Control rod S_{11}	0.9548	0.0057	0.0470
Control rod S_{22}	0.9324	0.0362	0.9746
Moderator S_{11}	0.9105	0.0008	0.0010
Control rod D_2	0.8284	0.0201	3.5947
Moderator $\Sigma_{a,1}$	0.7858	5.70×10^{-6}	0.0000
Control rod S_{12}	0.6450	6.21×10^{-5}	0.0164
Moderator D_1	0.6157	0.0042	0.0106
Moderator S_{12}	0.4098	0.0011	0.0124
Control rod S_{21}	0.3174	0.0023	4.28×10^8
Moderator $\Sigma_{a,2}$	0.2781	0.0001	0.0006
Moderator S_{22}	0.2661	0.0104	0.0445
Moderator S_{21}	0.2506	1.99×10^{-5}	0.0020
Moderator D_2	0.1585	0.0025	0.0113

Appendix B

Matrix formulation of the 1D diffusion solver

To provide insight into the discrete terms involved in the low-fidelity solver calculations, a representative matrix expansion for a 1D geometry with two energy groups (fast $g = 1$, thermal $g = 2$) and N spatial cells is outlined below. This also corresponds to what was used for the first implementation of PEDS presented in Chapter 4.

Considering first the fission matrix $\mathbf{F}$, the neutrons born from fission events are at high energy, meaning they are emitted solely into the fast energy group ($\chi_1 = 1, \chi_2 = 0$). The non-zero elements therefore represent the fast neutrons produced by both fast and thermal fissions; no fission neutrons are born directly into the thermal group. Having one element for every cell I of the N cells of the geometry, the matrix takes the following block structure:

$$\mathbf{F} = \begin{bmatrix} \mathbf{F}_{11} & \mathbf{F}_{12} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} = \left[\begin{array}{ccc|ccc} \chi_1 \nu \Sigma_{f,1}^{(1)} & 0 & \dots & \chi_1 \nu \Sigma_{f,2}^{(1)} & 0 & \dots \\ 0 & \ddots & \dots & 0 & \ddots & \dots \\ \vdots & \vdots & \chi_1 \nu \Sigma_{f,1}^{(N)} & \vdots & \vdots & \chi_1 \nu \Sigma_{f,2}^{(N)} \\ \hline 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{array} \right] \quad (\text{B.1})$$

The system matrix $\mathbf{A}$ contains all loss mechanisms for each energy group. It features a block structure in which the diagonal blocks $\mathbf{A}_{gg}$ represent intra-group physics, while the off-diagonal blocks encode inter-group scattering.

Each diagonal block is a tridiagonal matrix of size $N \times N$, combining the leakage operator $\mathbf{L}_g$ (describing the spatial coupling between neighboring cells) and the removal operator $\mathbf{R}_g$, which accounts for absorption and out-scattering from group g . The macroscopic removal cross-section is defined as $\Sigma_{r,g} = \Sigma_{a,g} + \sum_{g' \neq g} \Sigma_{s,g \rightarrow g'}$.

For a single energy group g and a 1D spatial mesh, numbering the spatial cells sequentially, the diagonal block corresponding to Equation 3.2 can be represented as:

$$\mathbf{A}_{gg} = \mathbf{L}_g + \mathbf{R}_g = \begin{bmatrix} \left(\Sigma_{r,g}^{(1)} + \frac{D_{1 \rightarrow 2}}{\Delta r^2} \right) & -\frac{D_{1 \rightarrow 2}}{\Delta r^2} & 0 & \dots \\ -\frac{D_{1 \rightarrow 2}}{\Delta r^2} & \left(\Sigma_{r,g}^{(2)} + \frac{D_{1 \rightarrow 2}}{\Delta r^2} + \frac{D_{2 \rightarrow 3}}{\Delta r^2} \right) & -\frac{D_{2 \rightarrow 3}}{\Delta r^2} & \dots \\ 0 & -\frac{D_{2 \rightarrow 3}}{\Delta r^2} & \ddots & \dots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix} \quad (\text{B.2})$$

The off-diagonal blocks of $\mathbf{A}$, denoted as $-\mathbf{S}_{g' \rightarrow g}$, are diagonal matrices in space, since scattering occurs locally within a single cell with no spatial coupling.

Combining these components, the full system matrix for the two-energy-group problem is assembled as:

$$\mathbf{A} = \begin{bmatrix} \mathbf{L}_1 + \mathbf{R}_1 & 0 \\ -\mathbf{S}_{1 \rightarrow 2} & \mathbf{L}_2 + \mathbf{R}_2 \end{bmatrix} \quad (\text{B.3})$$

where $-\mathbf{S}_{2 \rightarrow 1} = \mathbf{0}$ under the standard thermal reactor assumption that there is no up-scattering from the thermal group to the fast group.

Appendix C

Verification of numerical sources of error

Following the identification of the performance gap in the subcritical regime, it was necessary to verify the integrity of the underlying physical solvers before attributing the bottleneck to PEDS or the training data parameters space. Two potential sources of numerical error were checked: fission source convergence in the high-fidelity OpenMC evaluations, and spatial discretization (mesh size) in the low-fidelity diffusion solver. Specifically, highly subcritical Monte Carlo simulations often require more computational effort to achieve proper source convergence [39]. Simultaneously, the steep flux gradients frequently present in highly absorbing subcritical configurations might not be adequately resolved by the default spatial mesh used for the 1D diffusion solver. The following sections assess whether either effect contributes substantially to the observed residual errors.

C.1 OpenMC convergence

To investigate whether unconverged high-fidelity data were responsible for the surrogate’s failure to correct certain predictions, five distinct geometries yielding the highest prediction errors were selected for detailed analysis. To prevent parameter bias, these cases were explicitly chosen to cover a diverse range of physical configurations. The specific geometric parameters defining these five test cases are summarized in Table C.1.

Table C.1: Physical parameters for the five selected geometries.

Sample ID	CR Radius (cm)	B ₄ C Fraction	Fuel Radius (cm)	Enrichment (%)	Moderator Fraction	Water Radius (cm)
1	1.08	0.03	13.48	4.09	0.63	42.71
2	5.46	0.32	12.55	8.68	0.75	42.88
3	3.54	0.22	44.92	1.71	0.71	51.80
4	4.82	0.12	25.66	1.60	0.76	43.53
5	4.42	0.84	16.67	2.69	0.61	53.36

In eigenvalue simulations, incomplete convergence can manifest as either an unstable effective multiplication factor (k_{eff}) or an unconverged fission source distribution, both of which yield incorrect spatial flux shapes and erroneous multi-group cross-sections [39]. Consequently, the target k_{eff} utilized to train the

neural network would be inaccurate, and the baseline polynomial regression would produce incorrect estimations due to corrupted training data.

To verify convergence, the five selected geometries were re-simulated in OpenMC with an increased number of particle histories, specifically raising the inactive batches from 50 to 100 and the total batches from 200 to 300, together with a total amount of simulated particles increased from 20,000 to 30,000. Throughout the calculations, both the batch-wise k_{eff} and the Shannon entropy of the fission source were monitored to guarantee stability. As illustrated in Figure C.1 for a representative example, the Shannon entropy successfully plateaus early, confirming that the source distribution had fully converged prior to the start of the active batches.

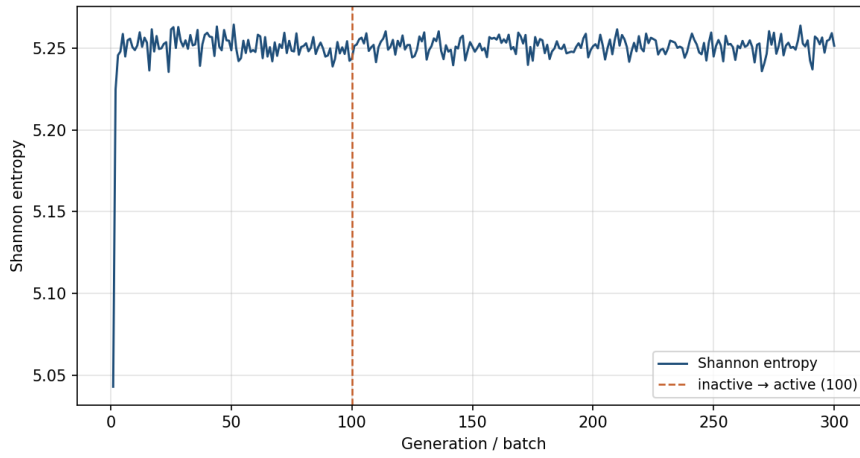


Figure C.1: Shannon entropy over Monte Carlo batches for a representative geometry. A stable plateau is reached before the transition from inactive to active batches at batch 100.

The effective multiplication factors obtained from these supervised converged runs were then compared to the original targets, and the diffusion solver was executed again using the newly extracted multi-group cross sections. The results are detailed in Table C.2.

Table C.2: Comparison of k_{eff} values between original and converged OpenMC runs, including standard deviations and diffusion solver results with updated cross sections.

Sample ID	Orig. OpenMC k_{eff}	Converged OpenMC $k_{\text{eff}} \pm \sigma$	OpenMC Diff. (pcm)	Orig. Diff. (True MGXS)	Converged Diff. (New MGXS)	Diff. Solver Diff. (pcm)
1	0.94517 ± 0.00060	0.94624 ± 0.00037	+120	1.01147	1.01136	-11
2	0.86221 ± 0.00060	0.86217 ± 0.00039	-5	0.94892	0.94863	-32
3	1.07743 ± 0.00059	1.07810 ± 0.00039	+58	1.10020	1.10064	+36
4	0.90596 ± 0.00052	0.90558 ± 0.00035	-46	0.94014	0.94031	+20
5	0.83535 ± 0.00058	0.83660 ± 0.00041	+180	0.92509	0.92534	+30

The difference between the new and original k_{eff} values was found to be minimal, remaining below 180 pcm in all cases. Furthermore, when the diffusion solver was executed using the new cross sections, the resulting k_{eff} deviated by less than 40 pcm from the original diffusion calculations. These negligible differences confirm that the original Monte Carlo datasets were adequately converged and that the extracted cross-sections are physically valid. Consequently, the high prediction errors observed in the subcritical regime cannot be attributed to a failure in the reference data generation, confirming that the

bottleneck resides in either the neural network’s feature extraction or the fundamental limitations of the 1D diffusion equations.

C.2 Influence of diffusion solver spatial discretization

A further analysis was conducted to determine if the largest residual errors could be mainly driven by the low-fidelity solver’s spatial discretization. A uniform 1cm spatial mesh was intentionally selected to prioritise rapid solver evaluations during neural network training and repeated design studies. Hence, the resulting low fidelity prediction included approximations from both the neutron diffusion assumption and the spatial discretisation, which are corrected for through the neural network adjustments of the multigroup cross sections. A dedicated mesh convergence study was therefore performed for the configurations with the largest remaining prediction errors to determine whether further spatial refinement could significantly improve the agreement with the high fidelity OpenMC reference.

The ten test geometries exhibiting the largest absolute reactivity errors ($|\Delta\rho|$) at the stored best training epoch were isolated. For each of these geometries, the PEDS-corrected macroscopic cross-sections were extracted. The geometries were then re-evaluated using the forward diffusion solver on a sequence of progressively finer meshes, spanning from a coarser 2.0 cm mesh down to a highly refined 0.01 cm mesh.

The results, illustrated in Figure C.2, demonstrate that refining the spatial mesh does not resolve the discrepancy with the high-fidelity target. Comparing the solver’s predictions on the 1 cm mesh against the finest 0.01 cm mesh reveals an average internal discretization error of approximately 1400 pcm. However, this is largely unaligned with the overall error against OpenMC. Even at the highest spatial resolutions, the mean absolute reactivity error against the OpenMC reference plateaus between 1600 and 2100 pcm for these worst-case geometries.

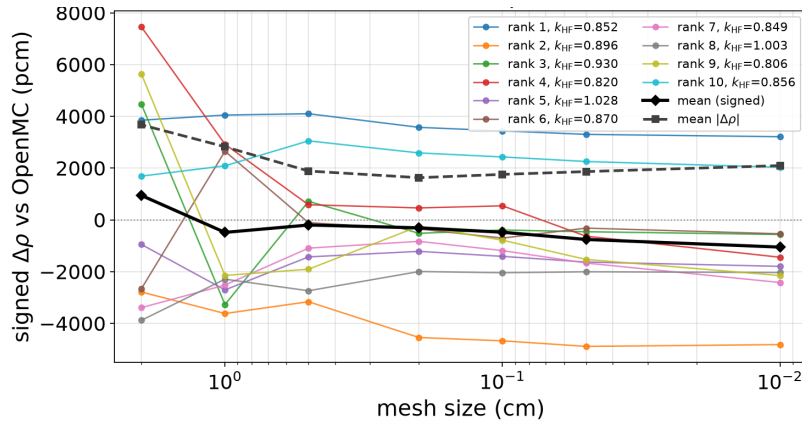


Figure C.2: Signed reactivity differences relative to OpenMC for the ten validation geometries with the largest residual errors after PEDS correction. Results are shown for mesh widths from 2.0 cm to 0.01 cm. Solid and dashed lines indicate the mean signed and mean absolute errors, respectively.

Additionally, the computational cost associated with substantial mesh refinement outweighs any marginal accuracy gains. Refining the mesh from 1 cm to 0.01 cm increases the average solve wall time by three

orders of magnitude (from roughly 0.003 seconds to 5.7 seconds), which undermines the primary objective of building a rapid surrogate model.

For further context, the same mesh refinement analysis was repeated using the uncorrected baseline cross sections produced by the polynomial regression. Figure C.3 shows that the mean reactivity error relative to the OpenMC reference remains approximately 8000 pcm over the investigated range of mesh widths. Refinement down to 0.01cm does not produce a meaningful reduction in this discrepancy, indicating that the error is not primarily caused by the spatial mesh. The reduction to approximately 2000 pcm obtained with PEDS further confirms that the neural network corrects most of the baseline discrepancy, while the remaining error cannot be resolved through spatial mesh refinement alone.

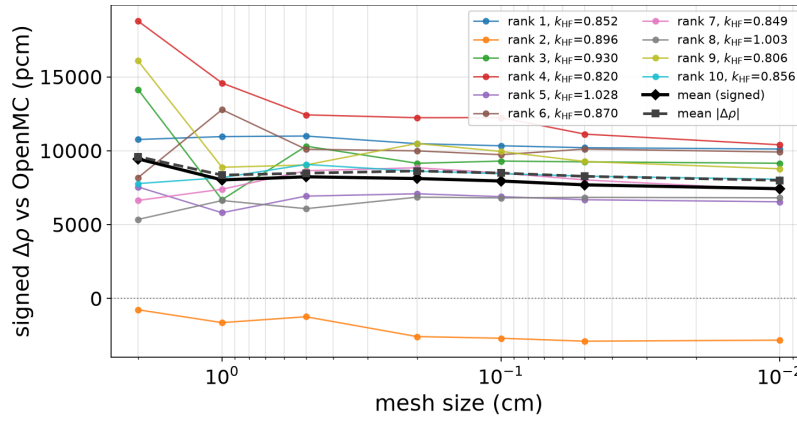


Figure C.3: Signed reactivity differences relative to OpenMC for the same geometries using the uncorrected polynomial cross sections. Results are shown for mesh widths from 2.0 cm to 0.01 cm. Solid and dashed lines indicate the mean signed and mean absolute errors, respectively.

Overall, although the 1cm mesh introduces a finite discretisation error, it is not the main source of the multi thousand pcm discrepancies observed for the worst performing predictions. The remaining error is instead attributed primarily to the inherent physical limitations of the one dimensional diffusion approximation or the neural network’s cross-section mapping, rather than to insufficient spatial resolution.

Appendix D

Parametric study performance metrics

The following section expands upon the parametric-space sensitivity study presented in Section Subsection 4.4.4. Table D.1 provides a detailed numerical breakdown of the PEDS correction performance across stratified k_{eff} ranges. For each bin and data split, the table reports the number and relative proportion of samples, together with the mean absolute reactivity discrepancy, $|\Delta\rho|$, before and after neural-network correction. It also reports the post-correction fraction of samples satisfying $|\Delta\rho| < \beta_{\text{eff}}$, where $\beta_{\text{eff}} = 650$ pcm. Results are reported as the mean and standard deviation across five independent runs; the test set remains locked across these runs.

Table D.1: PEDS correction performance across k_{eff} bins for train, validation, and test splits (train size = 1000 and validation size = 500, averaged over 5 runs; test size = 300 and kept locked). The fraction satisfying $|\Delta\rho| < \beta_{\text{eff}}$ is reported after correction, with $\beta_{\text{eff}} = 650$ pcm. Before correction, this fraction was zero in 22 of the 30 split-bin cases and did not exceed 2.44%, hence it was deemed negligible to report.

Split	k_{eff} range	No. of samples	% of split	Mean $ \Delta\rho $ (pcm)		Improvement	Fraction $ \Delta\rho < \beta_{\text{eff}}$
				Before	After		
Train	0.80–0.84	57.8 ± 1.5	5.78%	9283 ± 228	543 ± 75	94.2%	0.68 ± 0.07
	0.84–0.88	62.6 ± 3.2	6.26%	8983 ± 133	553 ± 92	93.8%	0.66 ± 0.08
	0.88–0.92	89.4 ± 7.4	8.94%	7864 ± 91	516 ± 86	93.4%	0.70 ± 0.07
	0.92–0.96	84.4 ± 2.6	8.44%	6791 ± 86	383 ± 40	94.4%	0.81 ± 0.06
	0.96–1.00	111.8 ± 2.0	11.18%	5713 ± 95	315 ± 47	94.5%	0.89 ± 0.04
	1.00–1.04	114.2 ± 3.3	11.42%	4946 ± 64	279 ± 41	94.4%	0.91 ± 0.03
	1.04–1.08	113.8 ± 4.3	11.38%	4414 ± 94	226 ± 45	94.9%	0.95 ± 0.04
	1.08–1.12	113.0 ± 2.6	11.30%	3819 ± 37	198 ± 33	94.8%	0.98 ± 0.01
	1.12–1.16	131.6 ± 2.3	13.16%	3265 ± 54	166 ± 19	94.9%	1.00 ± 0.00
	1.16–1.20	121.4 ± 3.4	12.14%	2794 ± 42	144 ± 15	94.8%	1.00 ± 0.00
Overall		1000	100.0%	5281	299	94.3%	0.89
Val	0.80–0.84	28.6 ± 1.8	5.72%	9519 ± 496	1433 ± 223	84.9%	0.26 ± 0.07
	0.84–0.88	34.6 ± 2.7	6.92%	8742 ± 165	1267 ± 116	85.5%	0.34 ± 0.11
	0.88–0.92	38.0 ± 5.6	7.60%	7952 ± 271	1012 ± 157	87.3%	0.38 ± 0.08
	0.92–0.96	47.2 ± 1.8	9.44%	6942 ± 170	799 ± 113	88.5%	0.54 ± 0.05
	0.96–1.00	56.6 ± 1.5	11.32%	5688 ± 174	570 ± 70	90.0%	0.68 ± 0.06
	1.00–1.04	56.8 ± 4.6	11.36%	5001 ± 89	464 ± 61	90.7%	0.77 ± 0.04
	1.04–1.08	59.8 ± 4.5	11.96%	4462 ± 132	411 ± 29	90.8%	0.78 ± 0.03
	1.08–1.12	51.4 ± 4.7	10.28%	3867 ± 187	353 ± 74	90.9%	0.85 ± 0.06
	1.12–1.16	68.4 ± 4.8	13.68%	3280 ± 72	307 ± 25	90.7%	0.91 ± 0.03
	1.16–1.20	58.6 ± 2.4	11.72%	2745 ± 48	276 ± 37	89.9%	0.92 ± 0.03
Overall		500	100.0%	5323	599	88.7%	0.70
Test	0.80–0.84	17.0 ± 0.0	5.67%	9782 ± 0	1238 ± 131	87.3%	0.35 ± 0.04
	0.84–0.88	21.0 ± 0.0	7.00%	8804 ± 0	1286 ± 131	85.4%	0.34 ± 0.09
	0.88–0.92	31.0 ± 0.0	10.33%	6968 ± 0	952 ± 84	86.3%	0.39 ± 0.08
	0.92–0.96	26.0 ± 0.0	8.67%	6602 ± 0	800 ± 54	87.9%	0.52 ± 0.03
	0.96–1.00	29.0 ± 0.0	9.67%	5620 ± 0	598 ± 57	89.4%	0.68 ± 0.05
	1.00–1.04	34.0 ± 0.0	11.33%	5011 ± 0	567 ± 63	88.7%	0.72 ± 0.10
	1.04–1.08	38.0 ± 0.0	12.67%	4831 ± 0	492 ± 46	89.8%	0.75 ± 0.05
	1.08–1.12	29.0 ± 0.0	9.67%	3702 ± 0	281 ± 33	92.4%	0.94 ± 0.02
	1.12–1.16	41.0 ± 0.0	13.67%	3223 ± 0	303 ± 41	90.6%	0.91 ± 0.06
	1.16–1.20	34.0 ± 0.0	11.33%	2881 ± 0	268 ± 45	90.7%	0.91 ± 0.05
Overall		300	100.0%	5311	630	88.1%	0.69

Appendix E

Detailed design study geometries

This appendix provides the detailed geometric parameters and evaluation metrics for the top performing designs identified in the forward design space searches discussed in Subsection 4.5.1 and Subsection 4.5.2.

E.1 High Fidelity LHS Baseline

Table E.1 lists the ten highest ranked feasible designs from the baseline evaluation campaign, which relied exclusively on the OpenMC solver. The initial pool consisted of 500 geometries generated via Latin Hypercube Sampling. Designs are ranked primarily by their power peaking factor (PPF), with the absolute deviation from criticality ($|\Delta\rho|$) utilized as a secondary ranking criterion.

Table E.1: Top 10 ranked feasible designs from the high fidelity LHS baseline. All designs share $\sigma_{k_{\text{eff}}} \approx 40$ pcm, the fixed convergence trigger used to stop each run.

Rank	k_{eff}	$ \Delta\rho $ (pcm)	PPF	r_{B4C}	f_{CR}	r_{fuel}	Enr. (%)	f_{mod}	r_{water}
1	0.994325	570.8	1.2534	4.031	0.271	36.493	1.643	0.540	67.145
2	0.994611	541.8	1.2603	2.675	0.086	26.900	1.794	0.694	76.517
3	0.999553	44.7	1.2749	1.622	0.007	25.079	2.129	0.549	76.965
4	1.006173	613.5	1.2922	3.156	0.783	38.218	1.780	0.513	45.783
5	0.990967	911.5	1.2950	3.805	0.457	26.614	2.160	0.587	54.388
6	0.997986	201.8	1.3192	5.913	0.586	25.601	4.645	0.444	40.372
7	1.004101	408.4	1.3654	2.899	0.840	17.637	3.953	0.715	72.811
8	0.994408	562.3	1.3693	2.031	0.904	35.256	1.552	0.777	69.453
9	0.997804	220.1	1.4024	5.056	0.828	19.200	4.847	0.652	43.367
10	0.994242	579.2	1.4249	4.806	0.722	16.884	5.752	0.745	40.425

E.2 Surrogate Assisted LHS Screening

Table E.2 details the ten best geometries identified by the PEDS ensemble from an initial pool of 5000 candidates, after high fidelity verification with OpenMC. The two configurations from the top 30 verification shortlist that exceeded the 1500 pcm criticality limit (ranks 2 and 3) have been excluded from this table. The discrepancy $|\Delta\rho_{\text{P} \leftrightarrow \text{H}}|$ indicates the absolute difference in predicted reactivity between the PEDS ensemble and the high fidelity OpenMC result.

It is observed that the finest designs achieve remarkably low power peaking factors, reaching a minimum of 1.235, while maintaining criticality within tighter margins than in the previous study. The fluctuating discrepancies between surrogate and high fidelity evaluations, confirm the intended use of the model as a broad screening tool rather than a precise estimator.

Table E.2: Detailed parameters and metrics for the ten best geometries identified by the PEDS ensemble, ranked by their high fidelity verified PPF.

HF rank	PEDS rank	k_{eff} PEDS	k_{eff} HF	$ \Delta\rho_{\text{P}\leftrightarrow\text{H}} $ (pcm)	PPF HF	r B4C	f CR	r fuel	Enr. (%)	f mod	r water
1	27	0.9987	1.0009	218	1.235	4.29	0.03	17.25	4.83	0.62	70.65
4	8	0.9997	1.0134	1351	1.291	2.79	0.14	15.34	5.10	0.74	66.28
5	25	1.0012	1.0004	78	1.299	5.17	0.82	18.55	7.85	0.56	60.37
6	19	0.9993	1.0058	646	1.315	3.39	0.52	15.44	8.74	0.60	46.39
7	24	1.0012	1.0138	1240	1.321	2.80	0.88	22.67	2.54	0.70	64.78
8	9	1.0004	1.0056	517	1.335	3.31	0.84	19.58	3.50	0.67	65.48
9	15	0.9994	1.0036	415	1.340	3.41	0.59	20.82	3.76	0.57	53.98
10	5	0.9999	1.0123	1221	1.352	1.37	0.06	13.38	5.64	0.74	61.97
11	7	0.9998	1.0005	65	1.355	1.76	0.16	19.05	2.72	0.72	59.47
12	20	0.9991	1.0012	211	1.356	2.11	0.59	21.25	4.43	0.46	53.49

Bibliography

- [1] Antonio Varagnolo, Giuseppe Romano, and Raphaël Pestourie.
Physics Enhanced Deep Surrogates for the Phonon Boltzmann Transport Equation, November 2025.
- [2] Amjad Ali.
The nuclear power renaissance in the 21st century: Analyzing past trends, present status, and future outlook.
Nuclear Analysis, 5(3):100231, September 2026.
- [3] Jaakko Leppänen, Maria Pusa, Tuomas Viitanen, Ville Valtavirta, and Toni Kaltiaisenaho.
The serpent monte carlo code: Status, development and applications in 2013.
Annals of Nuclear Energy, 82:142–150, 2015.
- [4] R. A. Forster and T. N. K. Godfrey.
Mcnp - a general monte carlo code for neutron and photon transport.
In R. Alcouffe, R. Dautray, A. Forster, G. Ledanois, and B. Mercier, editors, *Monte-Carlo Methods and Applications in Neutronics, Photonics and Statistical Physics*, volume 240 of *Lecture Notes in Physics*. Springer, Berlin, Heidelberg, 1985.
- [5] Paul K. Romano, Nicholas E. Horelik, Bryan R. Herman, Adam G. Nelson, Benoit Forget, and Kord Smith.
Openmc: A state-of-the-art monte carlo code for research and development.
Annals of Nuclear Energy, 82:90–97, 2015.
- [6] Robert Ehrlich and Hyman Hurwitz.
Multigroup methods for neutron diffusion problems.
Nucleonics, 12(2):23–30, 1954.
- [7] Alain Hébert.
Multigroup neutron transport and diffusion computations.
In Dan Gabriel Cacuci, editor, *Handbook of Nuclear Engineering*, volume 2, pages 1031–1180. Springer, New York, NY, 2010.
- [8] Edward W. Larsen and J. E. Morel.
Advances in discrete-ordinates methodology.
In Yousry Azmy and Enrico Sartori, editors, *Nuclear Computational Science: A Century in Review*. Springer, Dordrecht, 2010.

-
- [9] Michalis Frangos, Youssef Marzouk, Karen Willcox, and Bart van Bloemen Waanders. Surrogate and reduced-order modeling: a comparison of approaches for large-scale statistical inverse problems. *Large-Scale Inverse Problems and Quantification of Uncertainty*, pages 123–149, 2010.
- [10] Alison Cozad, Nikolaos V Sahinidis, and David C Miller. Learning surrogate models for simulation-based optimization. *AIChE Journal*, 60(6):2211–2227, 2014.
- [11] George Em Karniadakis, Ioannis G. Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang. Physics-informed machine learning. *Nature Reviews Physics*, 3(6):422–440, June 2021.
- [12] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, February 2019.
- [13] Jeyan Thiyagalingam, Mallikarjun Shankar, Geoffrey Fox, and Tony Hey. Scientific machine learning benchmarks. *Nature Reviews Physics*, 4(6):413–420, June 2022.
- [14] Physics-Informed Neural Networks with Hard Constraints for Inverse Design. <https://epubs.siam.org/doi/epdf/10.1137/21M1397908>.
- [15] Federico Antonello, Jacopo Buongiorno, and Enrico Zio. Physics informed neural networks for surrogate modeling of accidental scenarios in nuclear power plants. *Nuclear Engineering and Technology*, 55(9):3409–3416, September 2023.
- [16] Yuchen Xie, Yahui Wang, and Yu Ma. Boundary dependent physics-informed neural network for solving neutron transport equation. *Annals of Nuclear Energy*, 195:110181, January 2024.
- [17] Perceval Beja-Battais, Alain Grossetête, and Nicolas Vayatis. Enhancing Nuclear Reactor Core Simulation through Data-Based Surrogate Models, November 2025.
- [18] Xiaoqi Li, Youqi Zheng, Xianan Du, and Bowen Xiao. A new surrogate method for the neutron kinetics calculation of nuclear reactor core transients. *Nuclear Engineering and Technology*, 56(9):3571–3584, September 2024.
- [19] Rui-Zhi Zhang, Sheng-Feng Zhu, Kan Wang, Ding She, Jean-Philippe Argaud, Bertrand Bouriquet, Qing Li, and He-Lin Gong. Artificial intelligence in reactor physics: Current status and future prospects. *Nuclear Science and Techniques*, 37(6), jun 2026. Also available as arXiv preprint arXiv:2503.02440.

- [20] Raphaël Pestourie, Youssef Mroueh, Chris Rackauckas, Payel Das, and Steven G. Johnson.
Physics-enhanced deep surrogates for partial differential equations.
Nature Machine Intelligence, 5(12):1458–1465, December 2023.
- [21] Yang Cao, Shengtai Li, and Linda Petzold.
Adjoint sensitivity analysis for differential-algebraic equations: Algorithms and software.
- [22] Steven G Johnson.
Notes on Adjoint Methods for 18.335.
- [23] Paolo Luchini and Alessandro Bottaro.
An Introduction to Adjoint Problems.
Annual Review of Fluid Mechanics, 46(1):493–517, January 2014.
- [24] Weston M. Stacey.
Nuclear Reactor Physics.
Wiley-VCH, Weinheim, Germany, 2nd, completely revised and enlarged edition edition, 2007.
- [25] nuclear-power.com.
Neutron diffusion theory.
<https://www.nuclear-power.com/nuclear-power/reactor-physics/neutron-diffusion-theory/>, 2021.
Accessed 2026-07-21.
- [26] Dan Gabriel Cacuci and Mihaela Ionescu-Bujor.
Sensitivity and Uncertainty Analysis, Data Assimilation, and Predictive Best-Estimate Model Calibration.
In *Handbook of Nuclear Engineering*, pages 1913–2051. Springer, Boston, MA, 2010.
- [27] Paolo Saracco, Sandra Dulla, and Piero Ravetto.
The adjoint neutron transport equation and the statistical approach for its solution, September 2016.
- [28] Elmer E. Lewis and Warren F. Miller.
Computational Methods of Neutron Transport.
John Wiley & Sons, New York, 1984.
- [29] William Boyd, Adam Nelson, Paul K. Romano, Samuel Shaner, Benoit Forget, and Kord Smith.
Multigroup Cross-Section Generation with the OpenMC Monte Carlo Particle Transport Code.
Nuclear Technology, 205(7):928–944, July 2019.
- [30] OpenMC Development Team.
Multigroup cross section generation part i: Introduction.
<https://docs.openmc.org/en/v0.9.0/examples/mgxs-part-i.html>.
Accessed: 2026-07-22.
- [31] James J Duderstadt and Louis J Hamilton.
Nuclear Reactor Analysis.

- John Wiley & Sons, New York, 1976.
- [32] Adilson Costa da Silva, Aquilino Senra Martinez, and Alessandro da Cruz Gonçalves.
Reconstruction of the neutron flux in a slab reactor.
World Journal of Nuclear Science and Technology, 2(4):181–186, 2012.
- [33] Masatoshi Hayashi.
Analytical solution of diffusion adjoint equation for homogeneous bare critical reactor.
Annals of Nuclear Energy, 114:569–570, April 2018.
- [34] Atilim Gunes Baydin, Barak A Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind.
Automatic differentiation in machine learning: a survey.
Journal of Machine Learning Research, 18:1–43, 2018.
- [35] Wikipedia contributors.
Feature scaling.
https://en.wikipedia.org/wiki/Feature_scaling, 2026.
Accessed: 2026-07-10.
- [36] Ian Goodfellow, Yoshua Bengio, and Aaron Courville.
Deep Learning.
MIT Press, 2016.
<http://www.deeplearningbook.org>.
- [37] Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals.
Understanding deep learning requires rethinking generalization.
In *International Conference on Learning Representations (ICLR)*, 2017.
- [38] H. Mazrou et al.
Assessment of the safety impact caused by the variations of the power peaking factors.
Progress in Nuclear Energy, 2023.
- [39] Forrest B Brown.
On the use of shannon entropy of the fission distribution for assessing convergence of monte carlo criticality calculations.
In *Proceedings of the 2006 physics PHYSOR conference*, 2006.